

Escuela Colombiana de Ingeniería Julio Garavito

Arquitecturas de Software (ARSW)

Laboratorio: Escalamiento en Azure con Máquinas Virtuales y Balanceador de Carga

Integrantes:

Esteban Valente Arenas

Keyla Yunuette Serna Illescas

Repositorio: https://github.com/KeySerna/ARSW_LOAD-BALANCING_AZURE

Bogotá, octubre de 2026

1. Introducción

En este laboratorio se evaluó la escalabilidad de una aplicación Node.js (FibonacciApp) que calcula el enésimo número de la secuencia de Fibonacci, desplegada en máquinas virtuales de Microsoft Azure. Se compararon dos estrategias: el escalamiento vertical, que consiste en aumentar los recursos de una sola máquina, y el escalamiento horizontal, que consiste en agregar varias máquinas detrás de un balanceador de carga.

Para cada estrategia se midieron los tiempos de respuesta, el consumo de CPU y la cantidad de peticiones respondidas con éxito bajo carga concurrente generada con Newman, y se comparó el costo de cada infraestructura.

1.1 Escenario de calidad

Escalabilidad: cuando un conjunto de usuarios consulta un enésimo número (superior a 1000000) de la secuencia de Fibonacci de forma concurrente y el sistema se encuentra bajo condiciones normales de operación, todas las peticiones deben ser respondidas y el consumo de CPU del sistema no puede superar el 70 %.

1.2 Condiciones del entorno

El laboratorio se realizó con una suscripción Azure for Students. Esta suscripción tiene restricciones que obligaron a ajustar algunos pasos del enunciado: solo permite desplegar recursos en cinco regiones (West US 2, Spain Central, Mexico Central, France Central y Brazil South), limita a 6 los vCPU por región y a 3 las direcciones IP públicas por región, y no ofrece varios de los tamaños pedidos (B1ls y B4ms). Además, el SKU Básico del balanceador de carga fue retirado por Microsoft en 2025, por lo que se usó el SKU Estándar. Cada ajuste se explica en la sección correspondiente.

2. Parte 1 - Escalabilidad vertical

2.1 Creación de la máquina virtual

Se creó la máquina virtual VERTICAL-SCALABILITY en el grupo de recursos SCALABILITY_LAB, con imagen Ubuntu Server 24.04 LTS, usuario scalability_lab y autenticación por llave pública SSH. El primer intento en la región Chile Central fue rechazado por la política de regiones permitidas de la suscripción (RequestDisallowedByAzure), por lo que la VM se creó en Mexico Central.

El tamaño B1ls que pide el laboratorio no estaba disponible para la suscripción, así que se usó como tamaño inicial Standard B2ats_v2 (2 vCPU, 1 GiB de RAM, 7,52 US$ al mes).



Figura 1. Implementación completada y recursos creados por Azure junto con la VM.

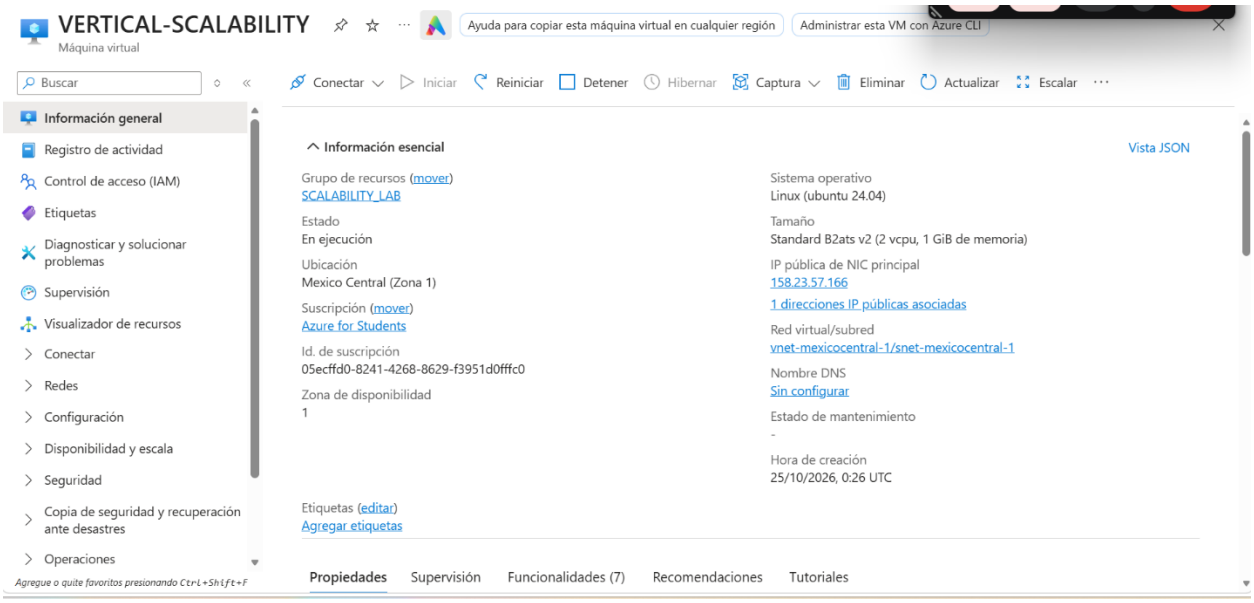


Figura 2. Información general de la VM VERTICAL-SCALABILITY (tamaño B2ats_v2, región Mexico Central).

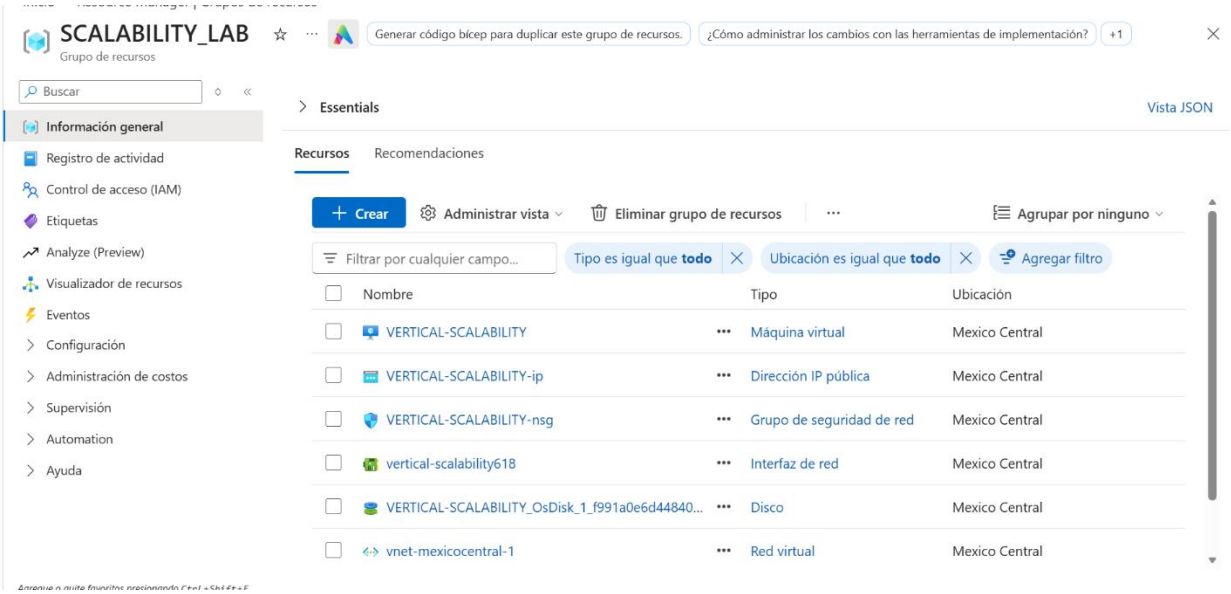


Figura 3. Recursos del grupo SCALABILITY_LAB.

Para poder acceder a la aplicación desde el navegador se creó una regla de entrada en el grupo de seguridad de red que permite el tráfico TCP por el puerto 3000.

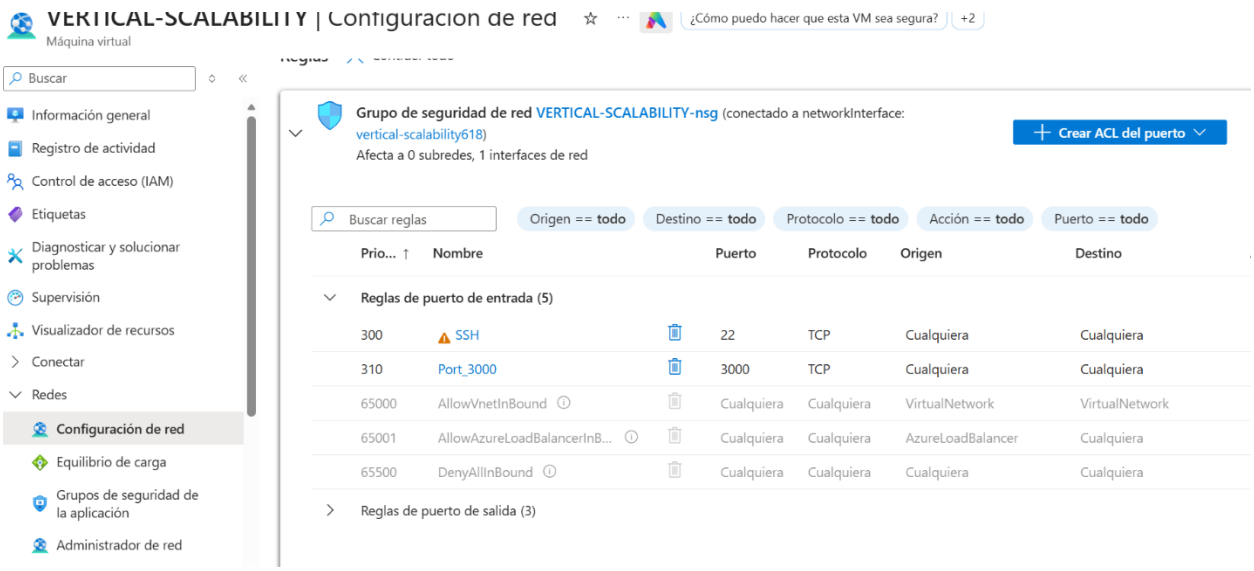


Figura 4. Regla de entrada Port_3000 en el grupo de seguridad de red de la VM.

2.2 Instalación de la aplicación

Se accedió a la VM por SSH, se instaló Node.js v24.22.1 con nvm, se clonó el repositorio, se instalaron las dependencias con npm install y se ejecutó la aplicación con forever para que siguiera funcionando al cerrar la sesión SSH. La aplicación respondió correctamente en el endpoint /fibonacci/6 con el valor 8.

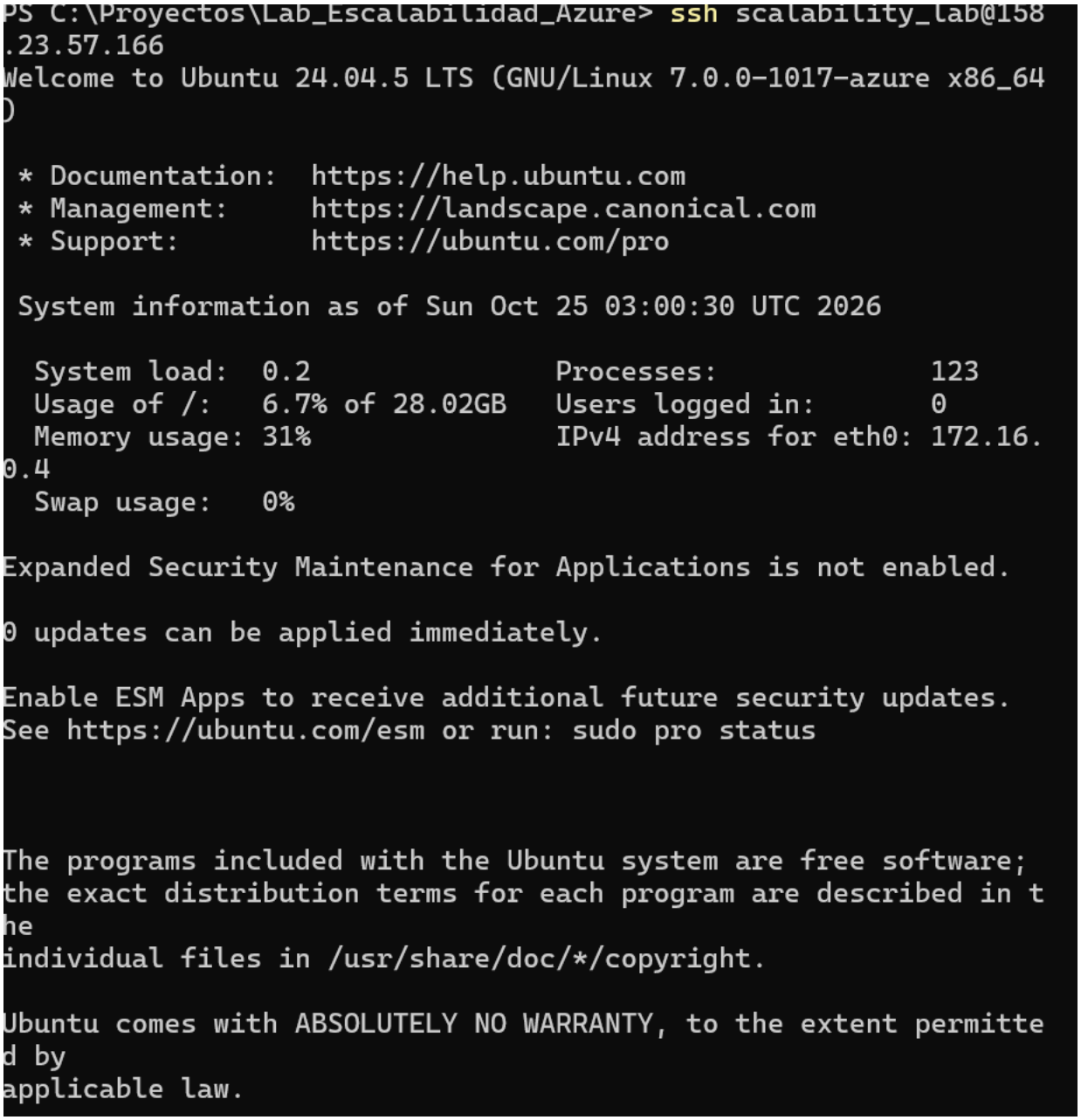


Figura 5. Conexión SSH a la VM VERTICAL-SCALABILITY.

```
added 290 packages in 16s

50 packages are looking for funding
  run `npm fund` for details
scalability_lab@VERTICAL-SCALABILITY:~/ARSW_LOAD-BALANCING_AZURE
/FibonacciApp$ forever start FibonacciApp.js
warn:    --minUptime not set. Defaulting to: 1000ms
warn:    --spinSleepTime not set. Your script will exit if it do
es not stay up for at least 1000ms
info:    Forever processing file: FibonacciApp.js
(node:2912) Warning: Accessing non-existent property 'padLevels'
 of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was c
reated)
(node:2912) Warning: Accessing non-existent property 'padLevels'
 of module exports inside circular dependency
(node:2912) [DEP0060] DeprecationWarning: The `util._extend` API
 is deprecated. Please use Object.assign() instead.
scalability_lab@VERTICAL-SCALABILITY:~/ARSW_LOAD-BALANCING_AZURE
/FibonacciApp$ forever list
(node:2939) Warning: Accessing non-existent property 'padLevels'
 of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was c
reated)
(node:2939) Warning: Accessing non-existent property 'padLevels'
 of module exports inside circular dependency
(node:2939) [DEP0060] DeprecationWarning: The `util._extend` API
 is deprecated. Please use Object.assign() instead.
info:    Forever processes running
data:        uid  command
             script          forever pid  id logfile
                    uptime
data:    [0] IACp /home/scalability_lab/.nvm/versions/node/v24.2
2.1/bin/node FibonacciApp.js 2924    2932    /home/scalability_l
ab/.forever/IACp.log 0:0:0:11.329
scalability_lab@VERTICAL-SCALABILITY:~/ARSW_LOAD-BALANCING_AZURE
/FibonacciApp$
```

Figura 6. FibonacciApp ejecutándose con forever.

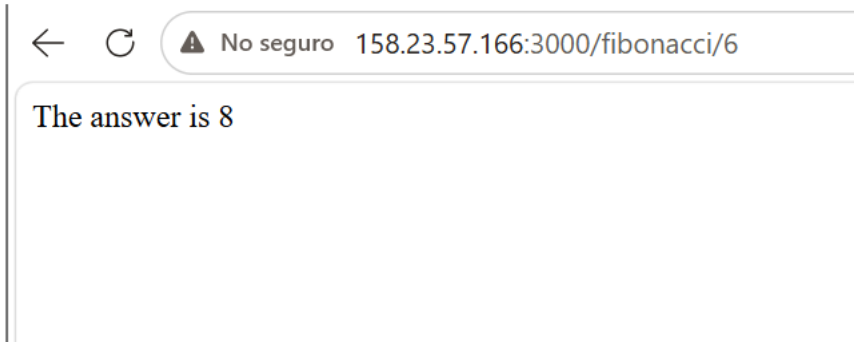


Figura 7. Respuesta del endpoint /fibonacci/6: The answer is 8.

2.3 Tiempos de respuesta

Se midieron desde la consola del navegador los tiempos de respuesta del endpoint para los diez valores pedidos, primero con el tamaño inicial B2ats_v2 y luego con el tamaño escalado D4s_v4. Las peticiones se hicieron una a la vez.

N	B2ats_v2 (2 vCPU)	D4s_v4 (4 vCPU)
1000000	**5.79 s**	**8.52 s**
1010000	**5.90 s**	**8.65 s**
1020000	**5.86 s**	**8.78 s**
1030000	**6.07 s**	**8.96 s**
1040000	**6.19 s**	**9.10 s**
1050000	**6.33 s**	**9.28 s**
1060000	**6.42 s**	**9.48 s**
1070000	**6.44 s**	**9.64 s**
1080000	**6.75 s**	**9.81 s**
1090000	**6.92 s**	**9.99 s**
Promedio	**6.27 s**	**9.22 s**

Tabla 1. Tiempos de respuesta secuenciales del endpoint /fibonacci/N.

```
allow pasting
> for (const n of
[1000000,1010000,1020000,1030000,1040000,1050000,1060000,1070000,1080000,10
90000]) {
    const t0 = performance.now();
    await fetch(`/fibonacci/${n}`);
    console.log(n, ((performance.now() - t0) / 1000).toFixed(2) + " s");
  }
1000000 '5.79 s'                                                    VM45:4
1010000 '5.90 s'                                                    VM45:4
1020000 '5.86 s'                                                    VM45:4
1030000 '6.07 s'                                                    VM45:4
1040000 '6.19 s'                                                    VM45:4
1050000 '6.33 s'                                                    VM45:4
1060000 '6.42 s'                                                    VM45:4
1070000 '6.44 s'                                                    VM45:4
1080000 '6.75 s'                                                    VM45:4
1090000 '6.92 s'                                                    VM45:4
< undefined
>
```

Figura 8. Tiempos de respuesta con el tamaño B2ats_v2.

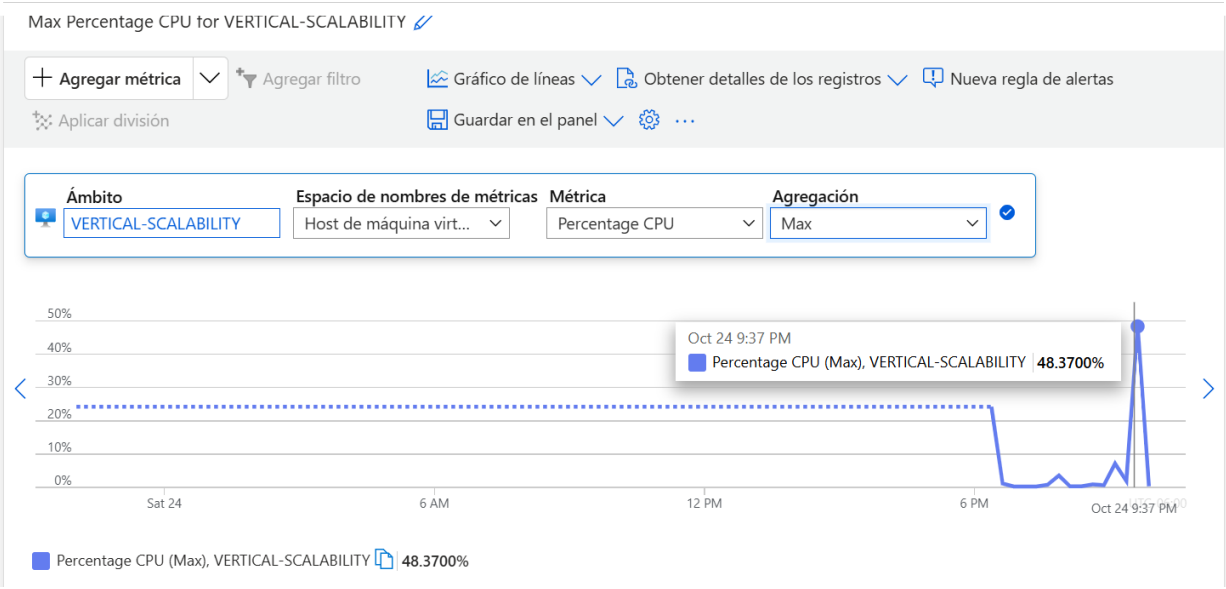


Figura 9. Tiempos de respuesta con el tamaño D4s_v4.

2.4 Consumo de CPU

En la sección Métricas de la VM se consultó el porcentaje de CPU con agregación máxima. Con B2ats_v2 el consumo llegó a 48.37 % durante la medición de tiempos y a

50.02 % durante la prueba con Newman. Con D4s_v4 el consumo se mantuvo alrededor de 25 % en todas las pruebas (25.02 %, 24.35 % y 25.24 %).

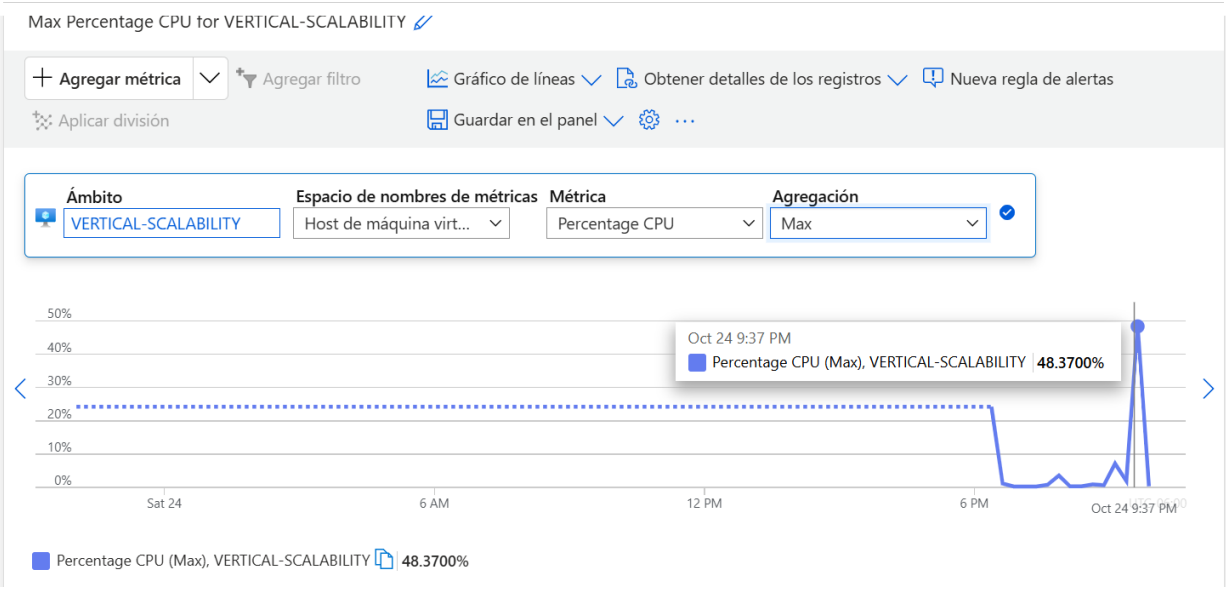


Figura 10. Consumo de CPU con B2ats_v2 durante la medición de tiempos.

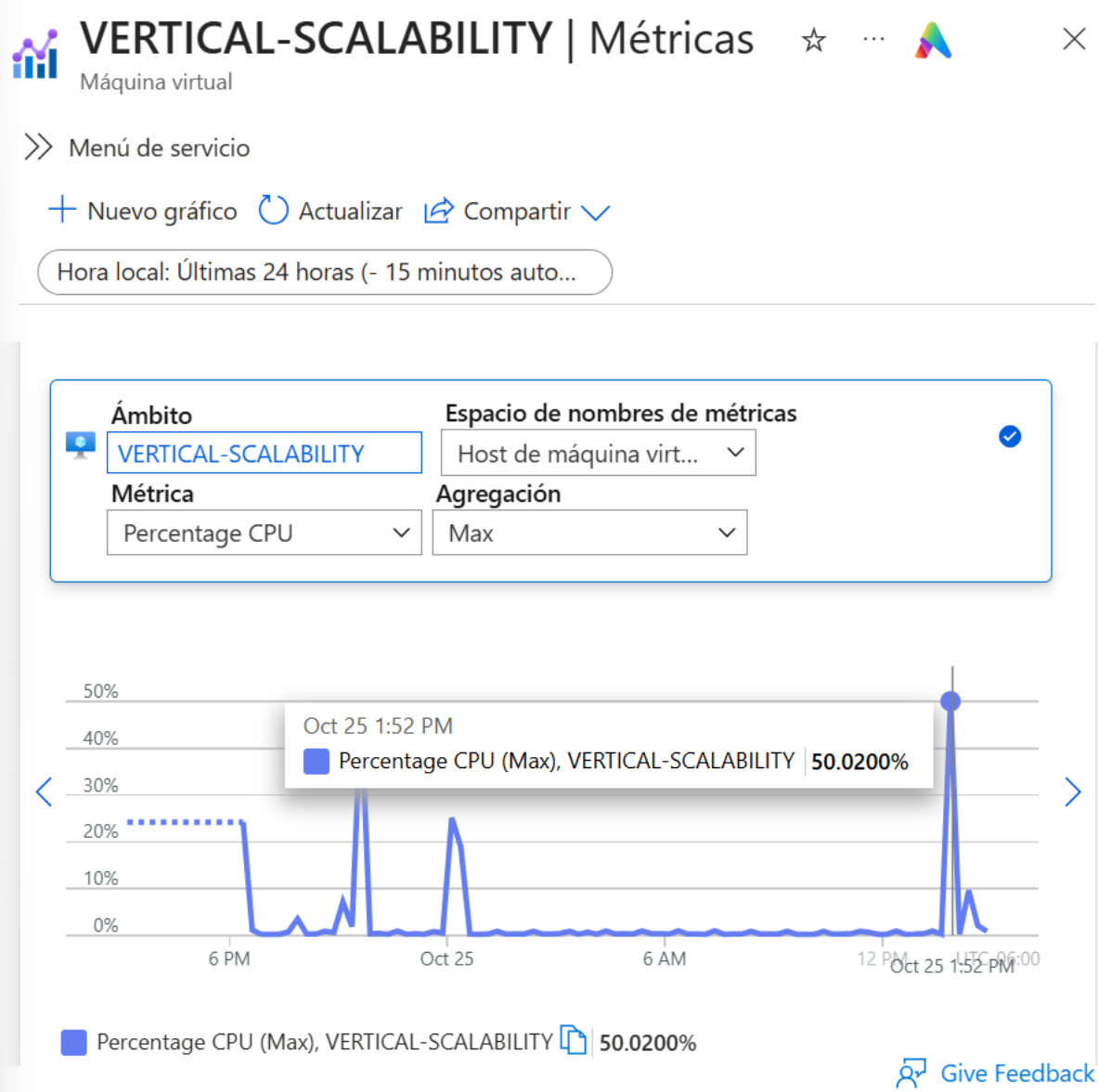


Figura 11. Consumo de CPU con B2ats_v2 durante la prueba de Newman con 2 ejecuciones en paralelo.

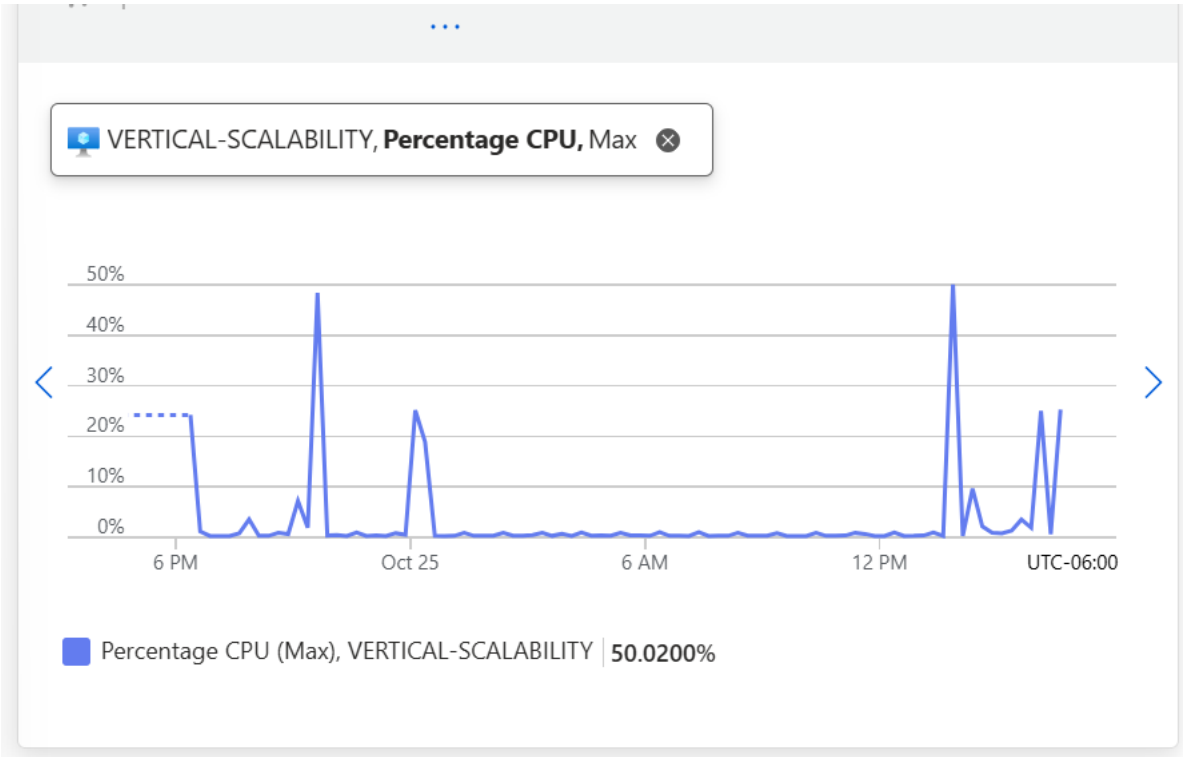


Figura 12. Consumo de CPU de la VM en 24 horas: picos de ~50 % con B2ats_v2 y de ~25 % con D4s_v4.

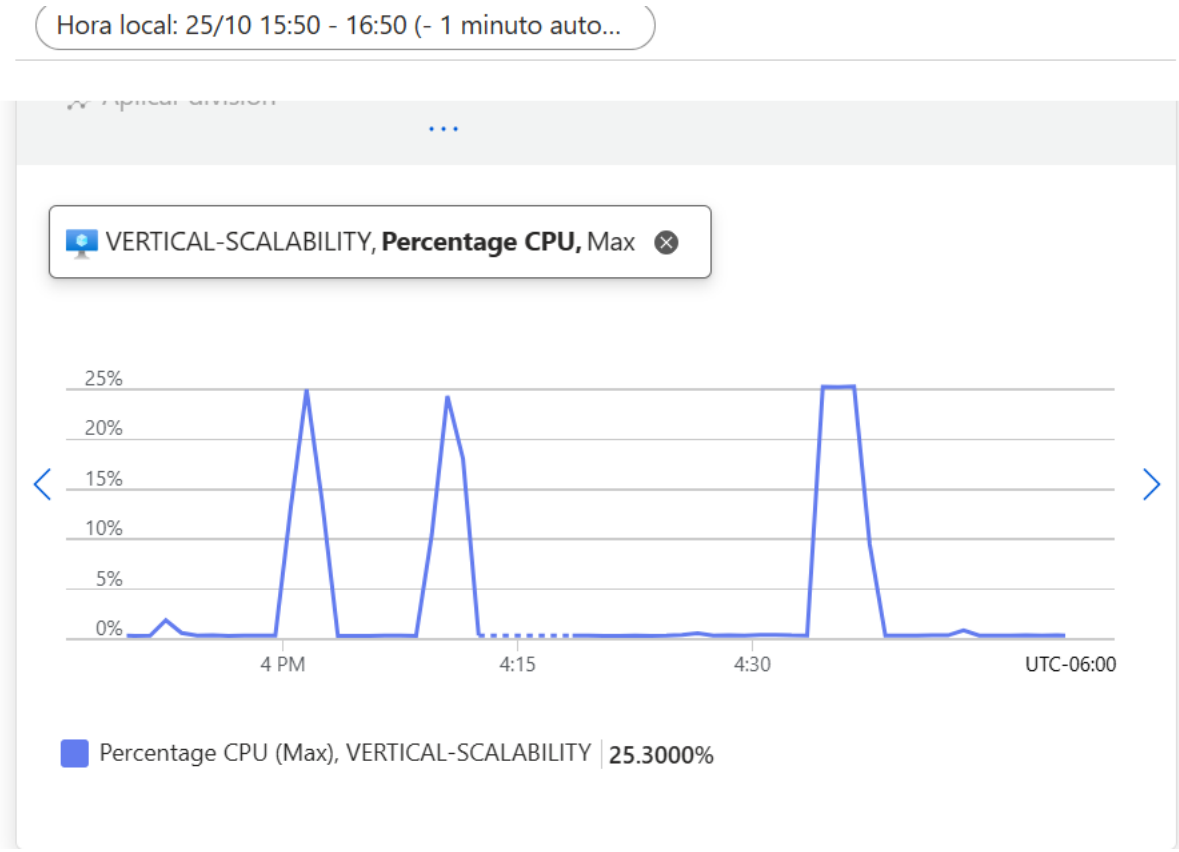


Figura 13. Consumo de CPU con D4s_v4 durante las tres pruebas.

2.5 Pruebas de carga con Newman

Desde un equipo distinto a la VM se ejecutó la colección de Postman con Newman, 10 iteraciones por ejecución, lanzando 2 ejecuciones al mismo tiempo. Con B2ats_v2 las 20 peticiones fueron exitosas, con un promedio de 10.6 s y 11.1 s por ventana. Con D4s_v4 fallaron 9 de 20 peticiones (45 %) con el error ECONNRESET.

```
Iteration 8/10

→ fibonacci
  GET http://158.23.57.166:3000/fibonacci/1000000 [200 OK, 209.24kB, 11s]

Iteration 9/10

→ fibonacci
  GET http://158.23.57.166:3000/fibonacci/1000000 [200 OK, 209.24kB, 10.9s]

Iteration 10/10

→ fibonacci
  GET http://158.23.57.166:3000/fibonacci/1000000 [200 OK, 209.24kB, 11.1s]
```

	executed	failed
iterations	10	0
requests	10	0
test-scripts	10	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 1m 46.8s		
total data received: 2.09MB (approx)		
average response time: 10.6s [min: 5.9s, max: 11.6s, s.d.: 1563ms]		

```
PS C:\Proyectos\ARSW_LOAD-BALANCING_AZURE\FibonacciApp\postman\part1>
```

Figura 14. Resumen de Newman con B2ats_v2, ejecución 1 de 2 en paralelo.

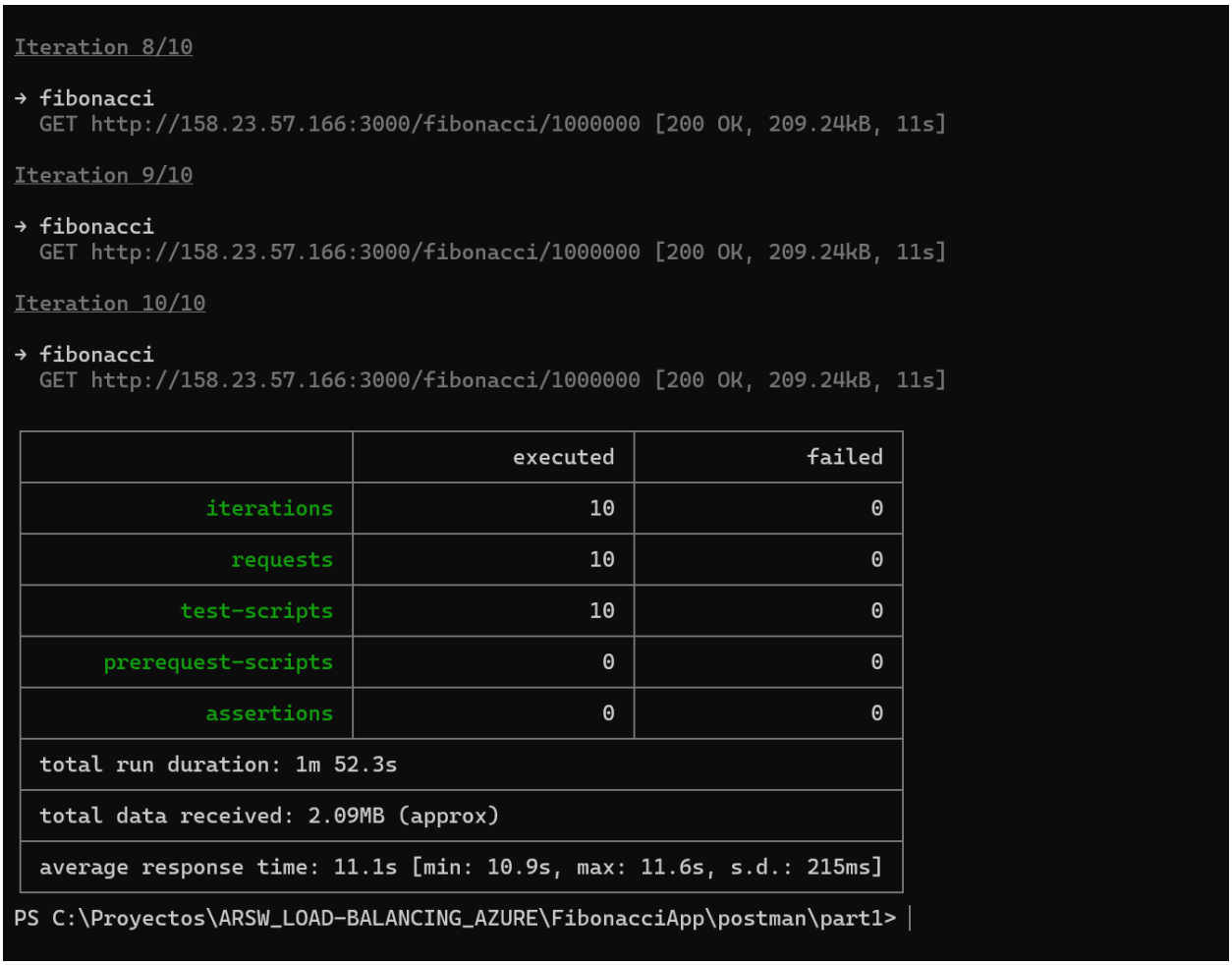


Figura 15. Resumen de Newman con B2ats_v2, ejecución 2 de 2 en paralelo.

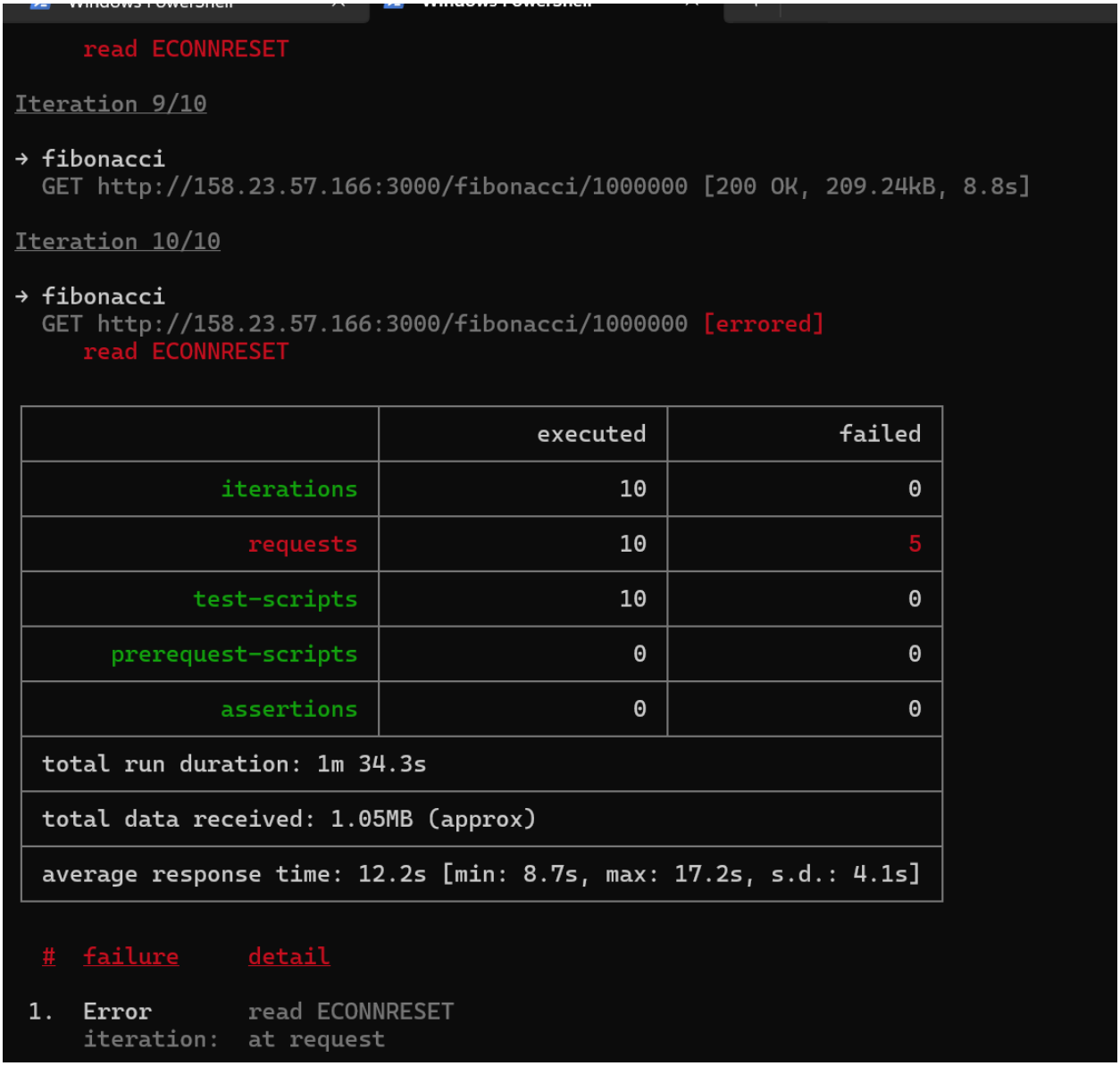


Figura 16. Resumen de Newman con D4s_v4, 2 ejecuciones en paralelo.

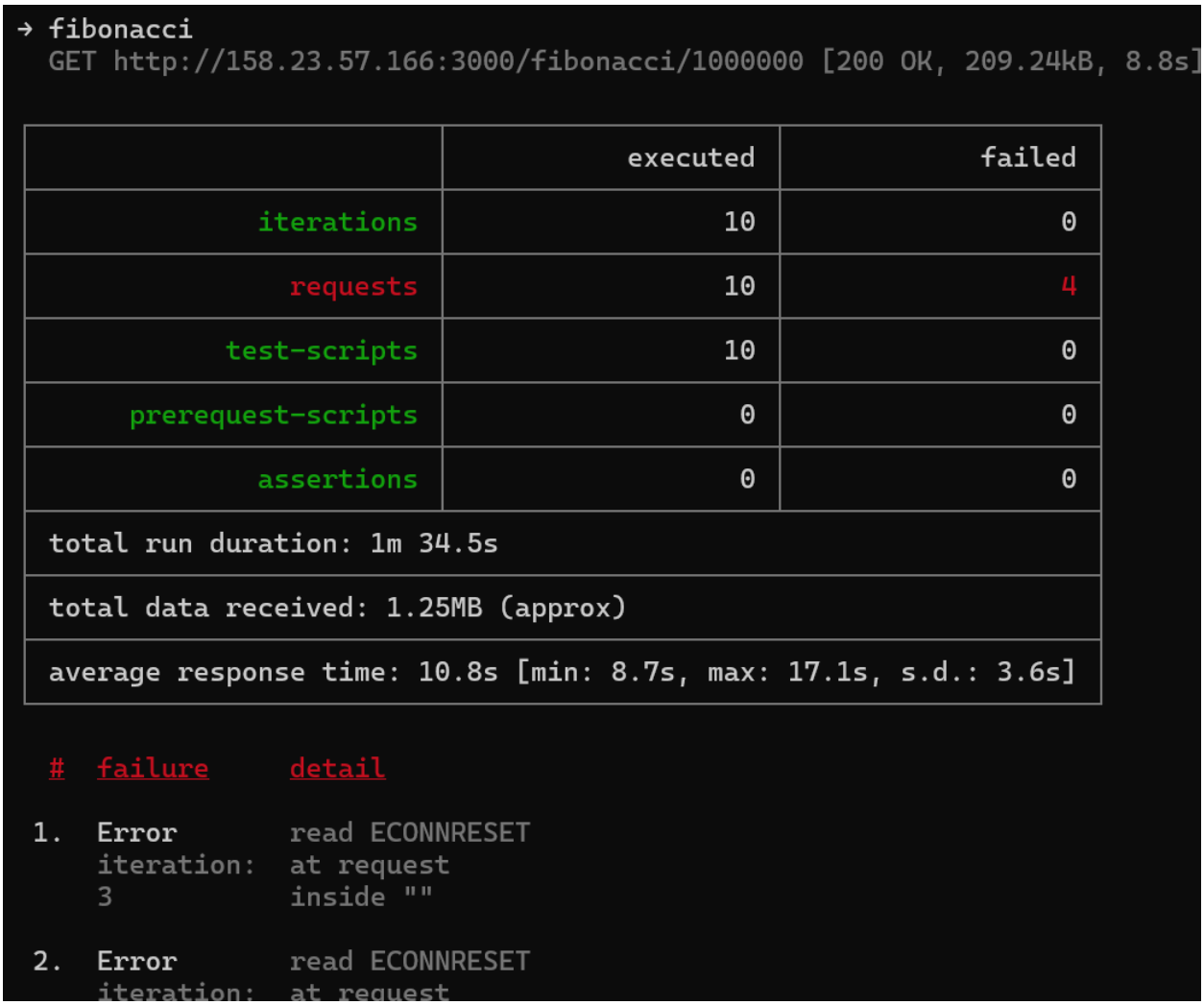


Figura 17. Peticiones fallidas con el error ECONNRESET usando D4s_v4.

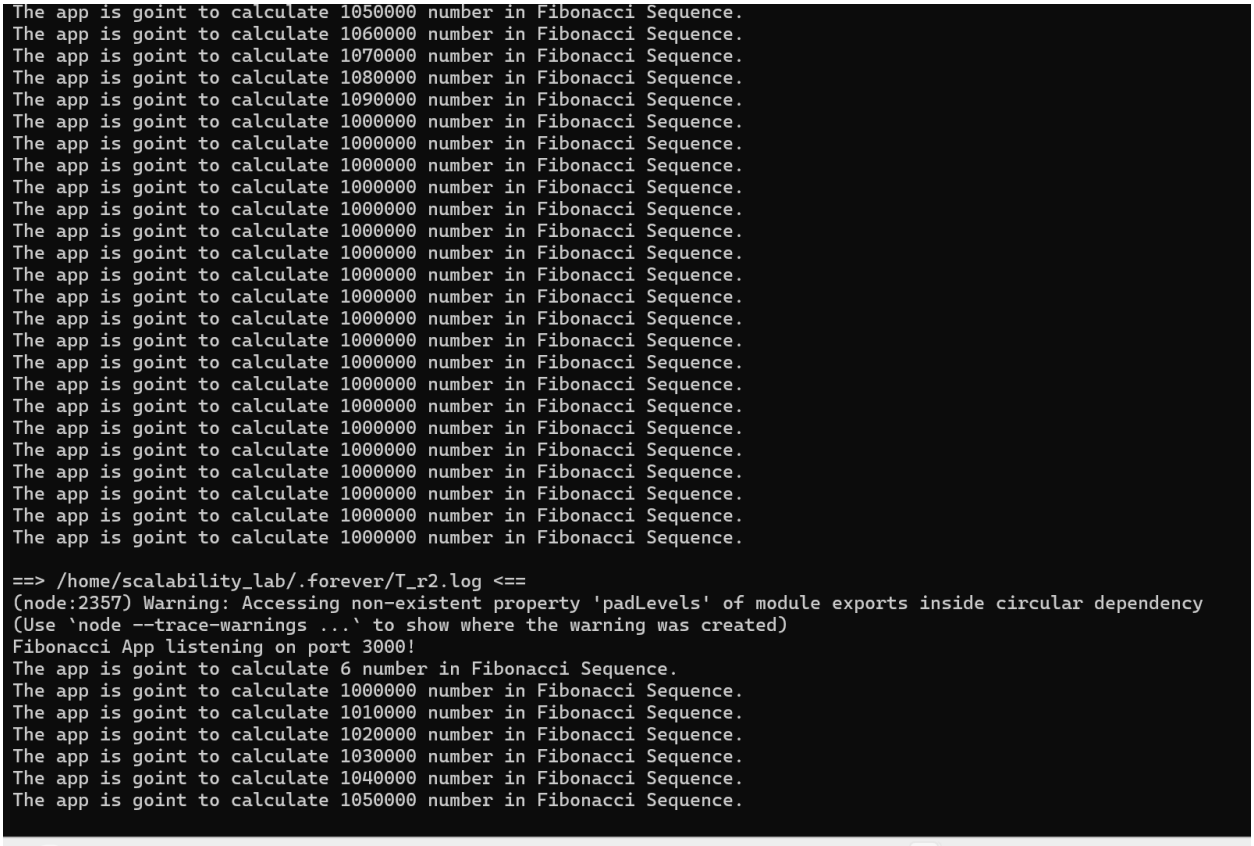


Figura 18. Diagnóstico: forever list y log de la aplicación después de la prueba.

Para entender los fallos se revisó la VM. La aplicación no se había caído (forever mostraba 34 minutos de actividad continua) y en el log solo aparecían cerca de 10 de las 20 peticiones enviadas, es decir, las peticiones fallidas nunca llegaron a procesarse. Node.js atiende con un solo hilo: mientras calcula un Fibonacci no puede atender ninguna otra petición. Newman reutiliza la misma conexión (keep-alive) y Node.js cierra las conexiones keep-alive que no atiende en unos 5 segundos. Con D4s_v4 cada cálculo tardaba cerca de 8.7 s, así que la petición que esperaba en fila superaba ese límite y la conexión se cerraba. Con B2ats_v2 cada cálculo tardaba cerca de 6 s, justo en el límite, y no hubo fallos.

También se ejecutaron 4 ejecuciones de Newman en paralelo sobre D4s_v4. Fallaron 19 de 40 peticiones (47.5 %) y el promedio de respuesta de las peticiones exitosas subió a 25.6 s.

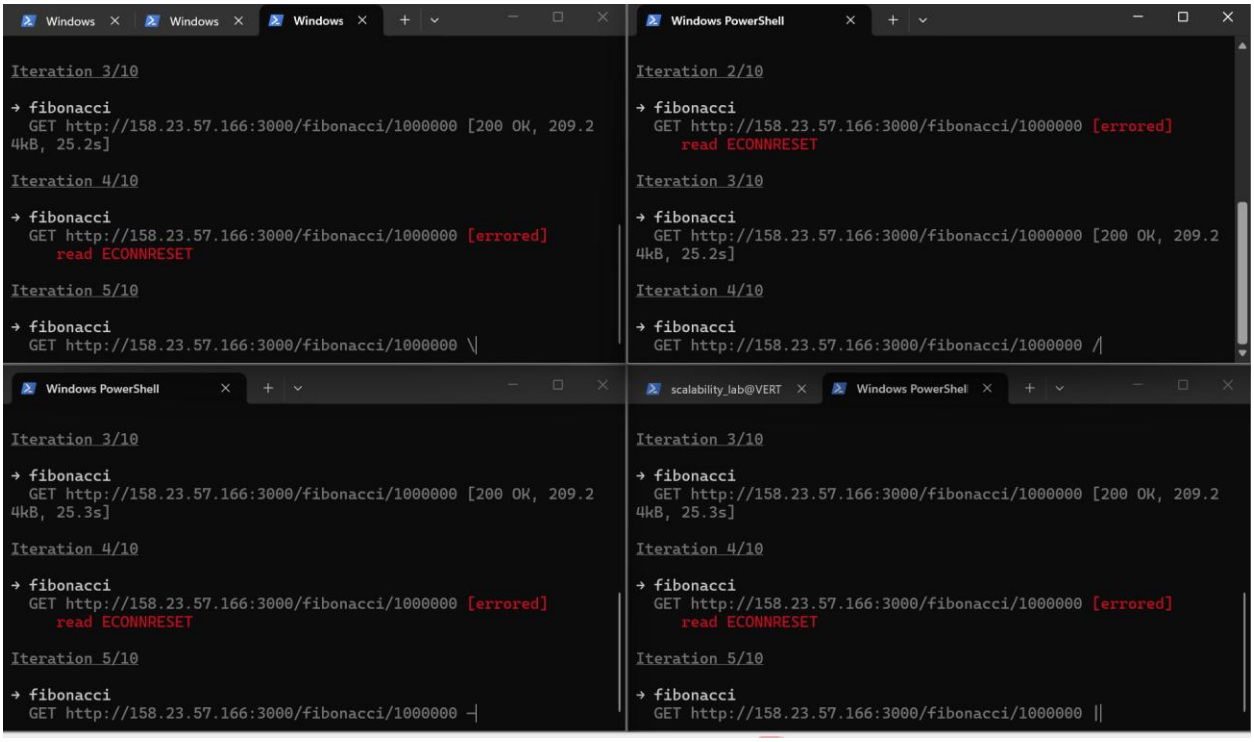
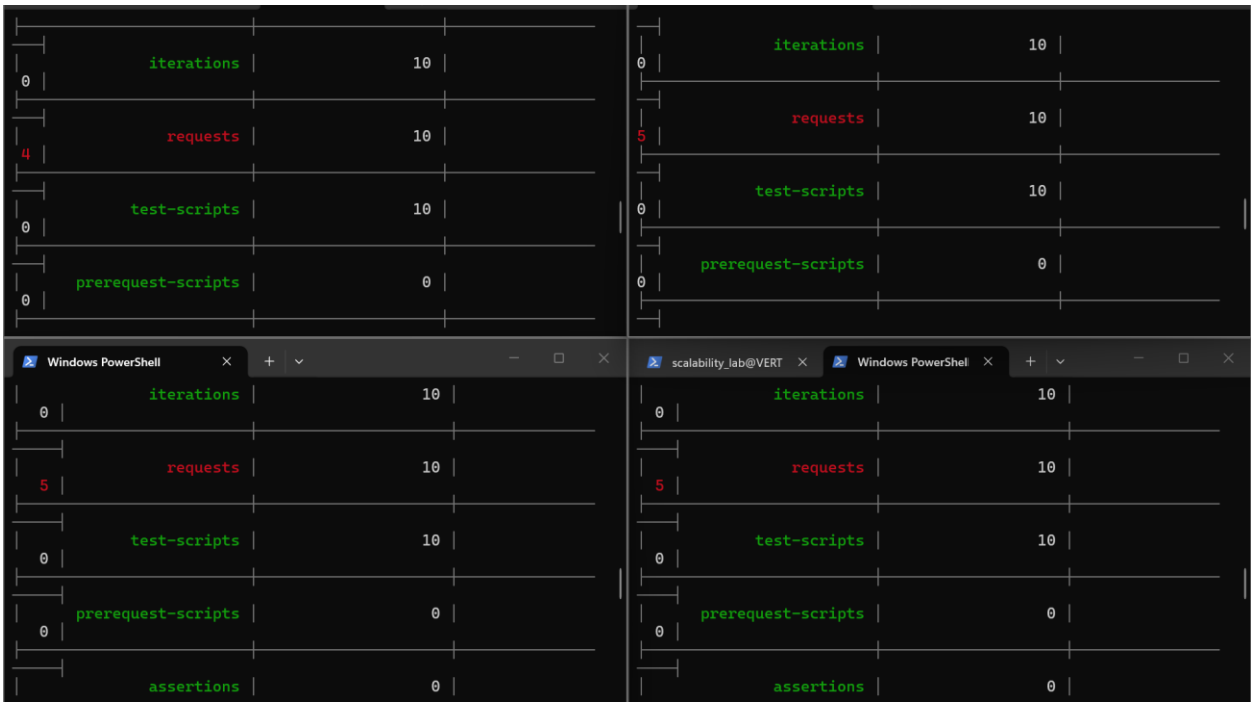


Figura 19. Ejecución de 4 instancias de Newman en paralelo sobre D4s_v4.



20. Resumen de las 4 ejecuciones de Newman en paralelo con D4s_v4.

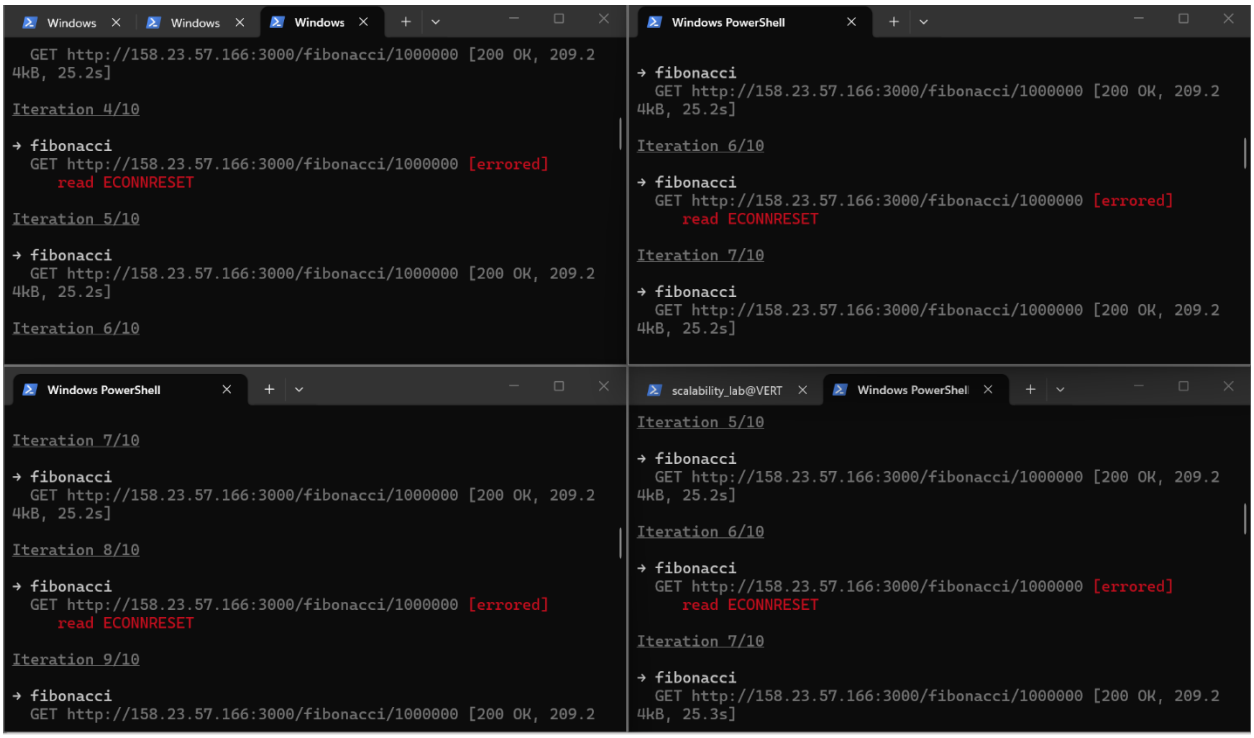


Figura 21. Peticiones fallidas con el error ECONNRESET en las 4 ejecuciones en paralelo con D4s_v4.

2.6 Escalamiento vertical

El laboratorio pide pasar de B1ls a B2ms. Como el tamaño inicial ya tenía 2 vCPU, para que el escalamiento fuera real se buscó un tamaño de 4 vCPU; el B4ms aparecía como no disponible en Mexico Central. Por eso la VM se escaló al tamaño D4s_v4 (4 vCPU, 16 GiB de RAM, 154,03 US$ al mes), unas 20 veces más caro que B2ats_v2. Al cambiar el tamaño, Azure reinició la VM y la aplicación dejó de responder hasta que se volvió a iniciar manualmente con forever. La IP pública no cambió porque es una IP estática de SKU Estándar.

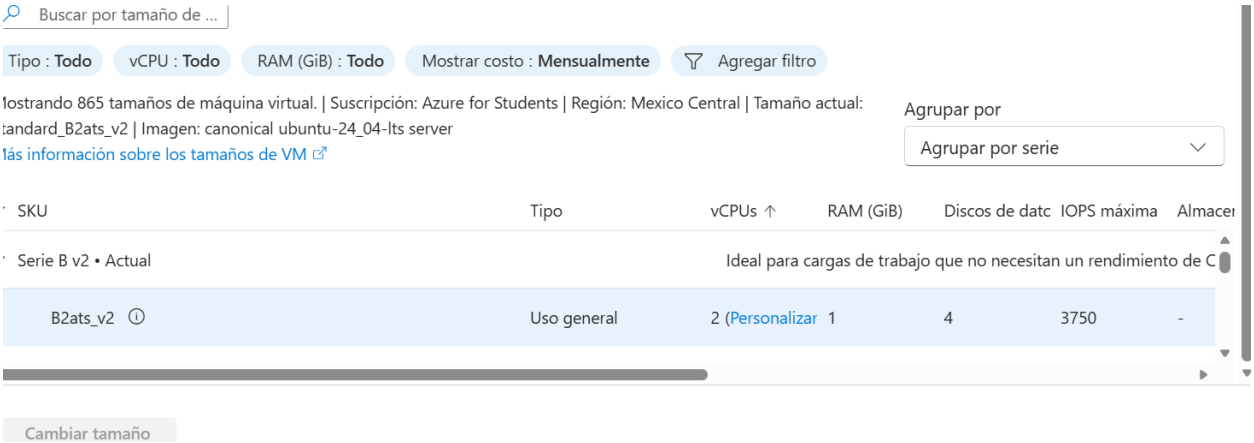


Figura 22. Tamaño actual B2ats_v2 en la pantalla de cambio de tamaño.

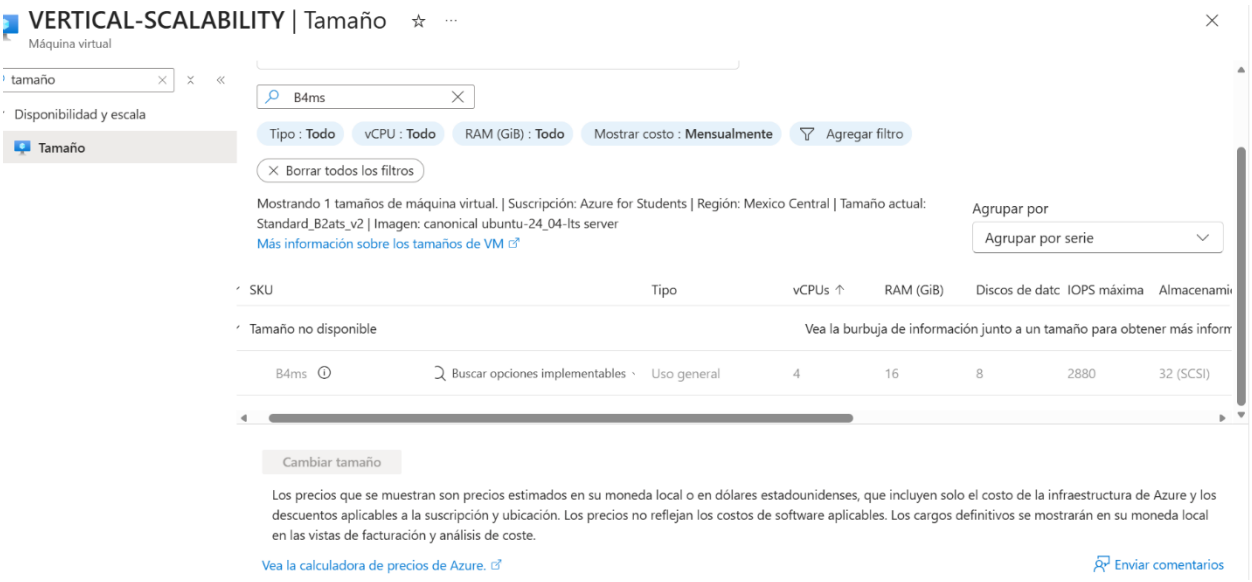


Figura 23. El tamaño B4ms no está disponible en Mexico Central para la suscripción.

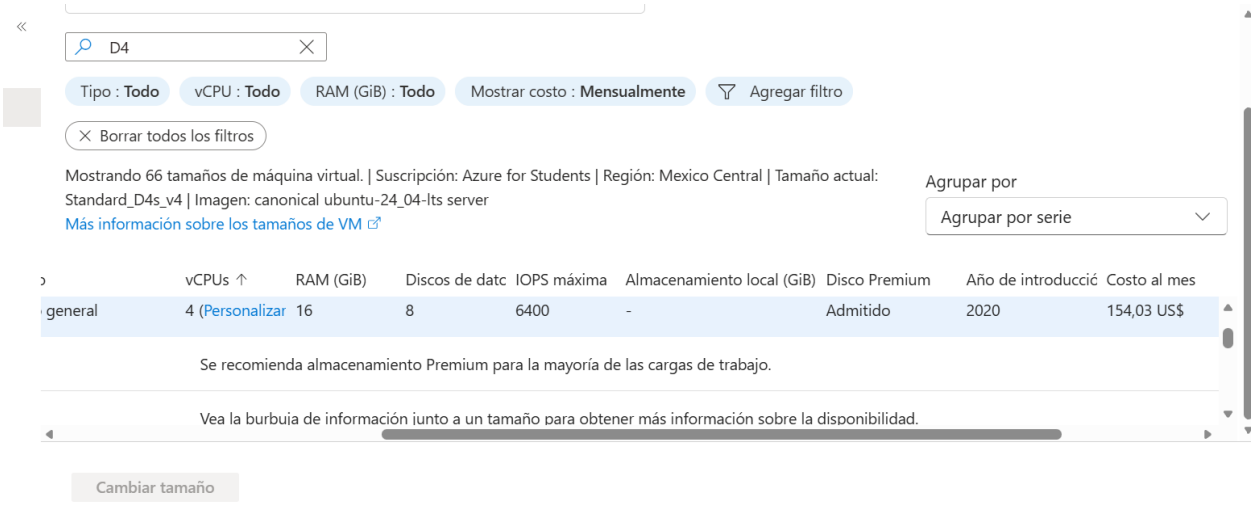


Figura 24. Selección del tamaño D4s_v4.

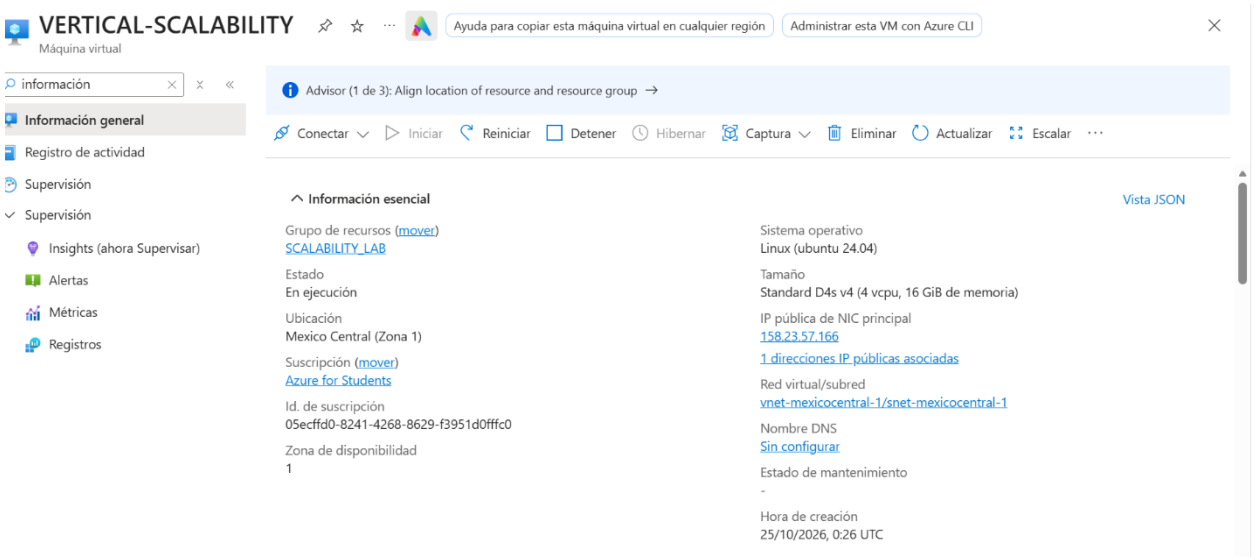


Figura 25. VM con el nuevo tamaño Standard D4s_v4.

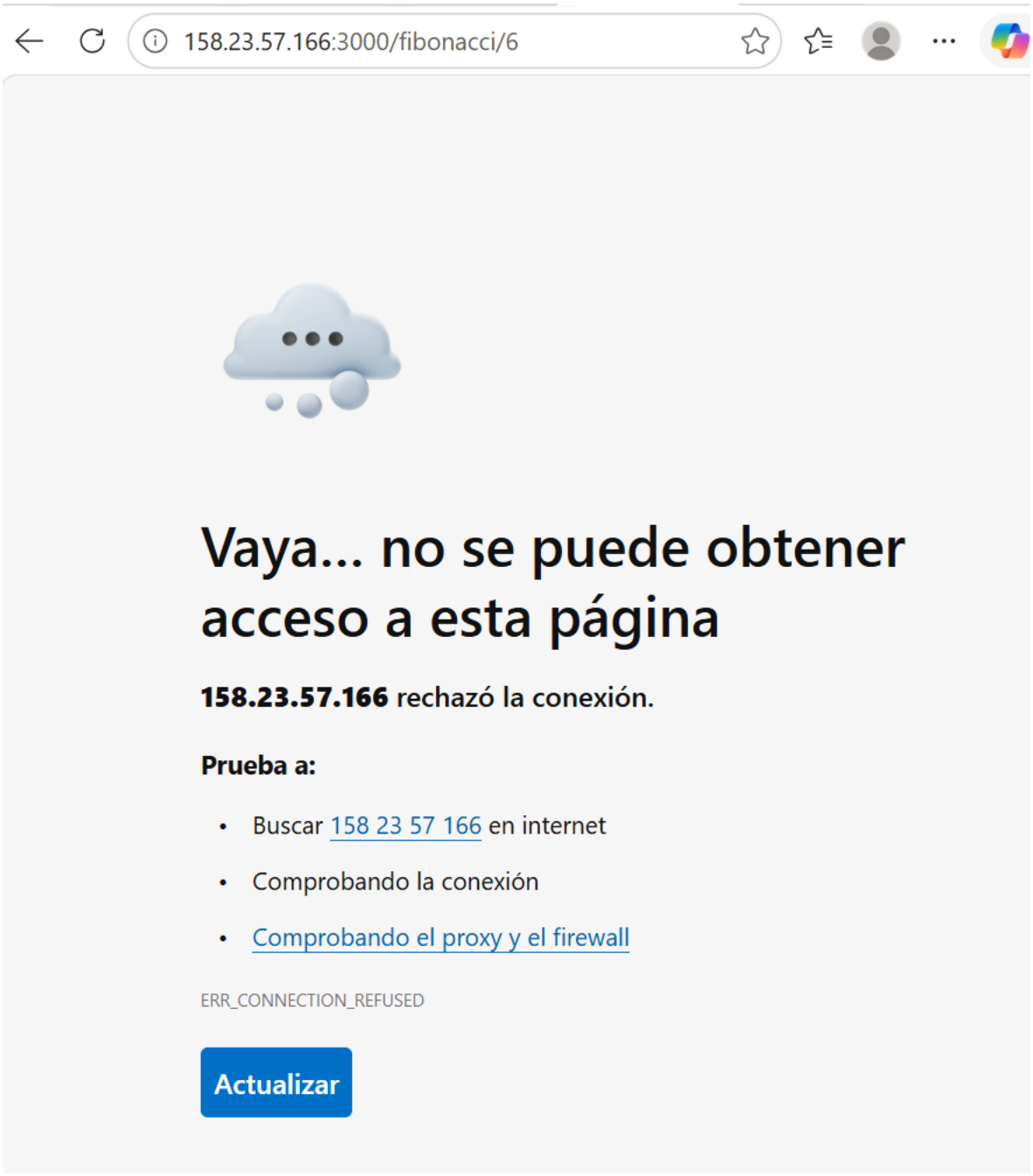


Figura 26. La aplicación deja de responder después del cambio de tamaño.

```
 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

8 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '26.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Sun Oct 25 03:00:31 2026 from 186.84.20.146
scalability_lab@VERTICAL-SCALABILITY:~$    cd ~/ARSW_LOAD-BALANCING_AZURE/FibonacciApp
scalability_lab@VERTICAL-SCALABILITY:~/ARSW_LOAD-BALANCING_AZURE/FibonacciApp$    forever start FibonacciApp.js
warn:    --minUptime not set. Defaulting to: 1000ms
warn:    --spinSleepTime not set. Your script will exit if it does not stay up for at least 1000ms
info:    Forever processing file: FibonacciApp.js
(node:2345) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:2345) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(node:2345) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() instead.
scalability_lab@VERTICAL-SCALABILITY:~/ARSW_LOAD-BALANCING_AZURE/FibonacciApp$    forever list
(node:2379) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:2379) Warning: Accessing non-existent property 'padLevels' of module exports inside circular dependency
(node:2379) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() instead.
info:    Forever processes running
data:        uid  command                                                script          forever pid  id logfile
                 uptime
data:    [0] T_r2 /home/scalability_lab/.nvm/versions/node/v24.22.1/bin/node FibonacciApp.js 2357    2365    /home/scalability_lab/.f
```

Figura 27. Aplicación iniciada de nuevo con forever después del cambio de tamaño.

Al terminar las pruebas la VM se regresó al tamaño inicial B2ats_v2 para evitar cobros adicionales, y la aplicación se volvió a iniciar.

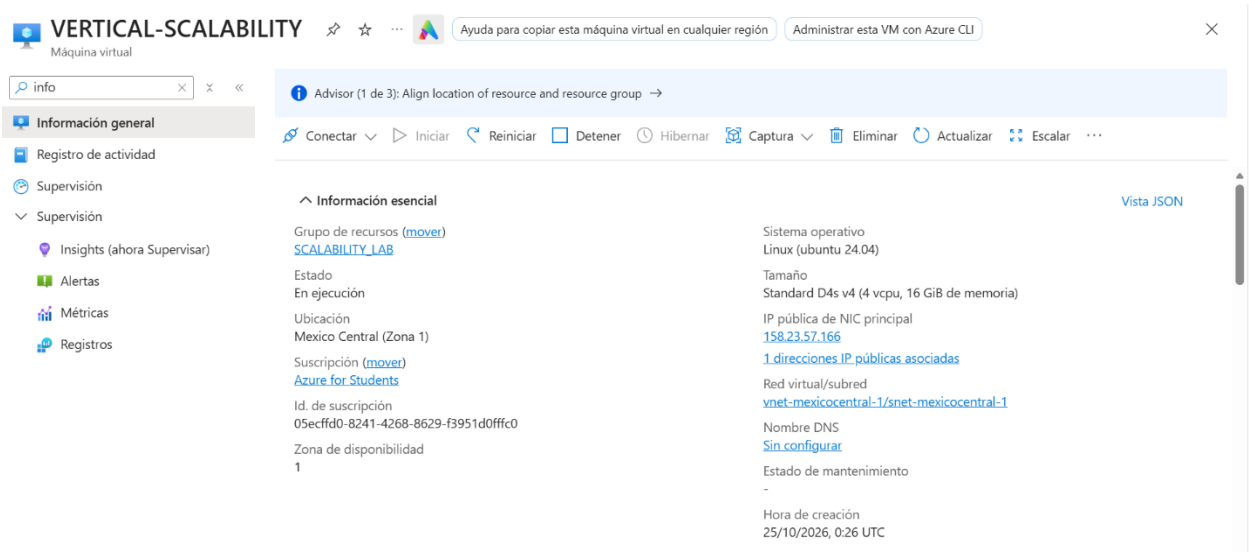


Figura 28. VM regresada al tamaño B2ats_v2.

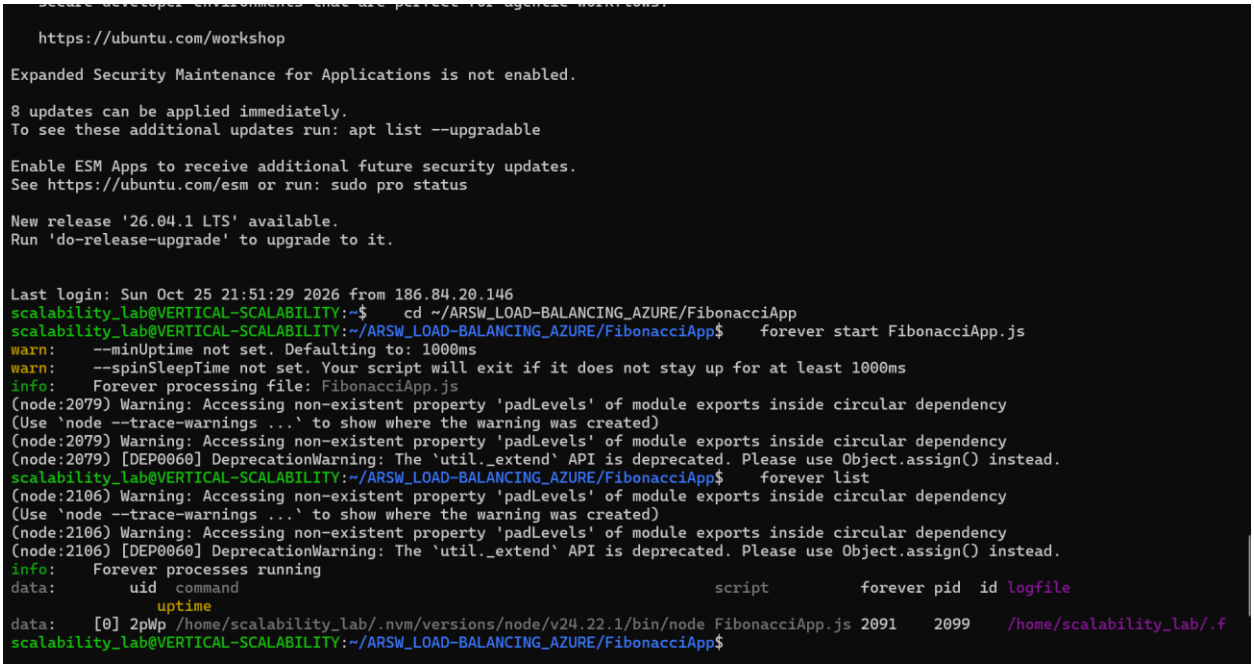


Figura 29. Aplicación funcionando de nuevo con el tamaño inicial.

2.7 Evaluación del escenario de calidad

El escalamiento vertical no cumple el escenario de calidad. La CPU nunca superó el 70 %, pero no porque el sistema tuviera capacidad de sobra, sino porque la aplicación solo puede usar un núcleo: con 2 vCPU el máximo fue cercano al 50 % y con 4 vCPU cercano al 25 %. La condición de responder todas las peticiones no se cumplió con D4s_v4, donde fallaron entre el 45 % y el 47.5 % de las peticiones. Además, la VM más grande fue más lenta, cuesta unas 20 veces más y su cambio de tamaño obligó a reiniciar la máquina.

2.8 Respuestas a las preguntas de la Parte 1

1. ¿Cuántos y cuáles recursos crea Azure junto con la VM?

Azure creó seis recursos: la máquina virtual VERTICAL-SCALABILITY, el disco del sistema operativo, la interfaz de red (NIC), la dirección IP pública, el grupo de seguridad de red (NSG) y la red virtual, que incluye una subred. Se pueden ver en la Figura 1 y en la Figura 3.

2. Describa brevemente para qué sirve cada recurso.

La máquina virtual es el servidor donde corre el sistema operativo y la aplicación. El disco del sistema operativo almacena Ubuntu, Node.js y los archivos de la aplicación de forma persistente. La interfaz de red conecta la VM con la red virtual y tiene asociadas sus direcciones IP. La IP pública permite llegar a la VM desde internet. El grupo de seguridad de red funciona como firewall y define qué tráfico de entrada y salida se permite, por ejemplo el puerto 22 para SSH. La red virtual con su subred es la red privada de Azure donde vive la VM y desde la cual recibe su IP privada.

3. ¿Por qué se cae la aplicación al cerrar la conexión SSH si se ejecuta directamente con node? ¿Por qué hay que crear una Inbound port rule antes de acceder al servicio?

Cuando la aplicación se ejecuta directamente desde la terminal, el proceso queda asociado a la sesión SSH. Al cerrar la conexión, el sistema operativo envía la señal de cierre (SIGHUP) a todos los procesos de esa sesión y la aplicación termina. Por eso se usó forever, que la ejecuta como un proceso en segundo plano independiente de la sesión. La regla de entrada es necesaria porque el grupo de seguridad de red bloquea por defecto todo el tráfico de entrada que no esté permitido explícitamente; al crear la VM solo se abrió el puerto 22, así que el puerto 3000 donde escucha la aplicación tenía que habilitarse.

4. Adjunte la tabla de tiempos e interprete por qué la función tarda tanto tiempo.

Los tiempos se muestran en la Tabla 1. La función calcula el número de forma iterativa, haciendo N - 1 sumas con números de precisión arbitraria (librería big-integer). Para N = 1000000 el resultado tiene más de 200 000 dígitos (la respuesta pesa 209 KB), y cada suma recorre todos los dígitos, así que el trabajo total crece aproximadamente con el cuadrado de N. Por eso el tiempo aumenta a medida que N crece. Además, el resultado no se guarda: cada petición repite todo el cálculo, y como el cálculo es síncrono, bloquea el único hilo de Node.js mientras dura.

5. Adjunte la imagen del consumo de CPU e interprete por qué la función consume esa cantidad de CPU.

Las imágenes están en las Figuras 10 a 13. Durante el cálculo la CPU llega a cerca del 50 % con 2 vCPU y a cerca del 25 % con 4 vCPU. En ambos casos significa que un núcleo está trabajando al 100 %: el cálculo es intensivo en CPU y no hace pausas de entrada y salida, pero Node.js ejecuta el código en un solo hilo, así que nunca usa más de un núcleo, sin importar cuántos tenga la máquina.

6. Adjunte la imagen del resumen de la ejecución de Postman. Interprete los tiempos de ejecución de cada petición y, si hubo fallos, documéntelos y explique.

Con B2ats_v2 y 2 ejecuciones en paralelo, las 20 peticiones fueron exitosas, pero el promedio subió a cerca de 11 s, casi el doble de una petición individual (6 s). Esto se debe a que, como Node.js atiende una petición a la vez, cuando llegan dos al mismo tiempo una espera en fila a que termine la otra. Con D4s_v4 fallaron 9 de 20 peticiones con el error ECONNRESET. La aplicación no se cayó: las conexiones que esperaban en fila más de unos 5 segundos fueron cerradas por Node.js (tiempo de keep-alive) antes de ser atendidas, como se explica en la sección 2.5.

7. ¿Cuál es la diferencia entre los tamaños B2ms y B1ls (no solo busque especificaciones de infraestructura)?

B1ls tiene 1 vCPU y 0,5 GiB de RAM y B2ms tiene 2 vCPU y 8 GiB, pero la diferencia no es solo de hardware. Ambos son de la serie B (burstable), que funciona con créditos de CPU: la máquina tiene un rendimiento base y, cuando está por debajo de él, acumula créditos que después puede gastar para usar la CPU al máximo por un tiempo limitado. El rendimiento base de B1ls es muy bajo (cerca del 5 %), mientras que el de B2ms es mucho mayor (cerca del 60 %), así que B2ms soporta cargas sostenidas por mucho más tiempo. B2ms también admite más discos, más IOPS y más ancho de banda de red, y cuesta aproximadamente 16 veces más. En este laboratorio la comparación equivalente fue B2ats_v2 contra D4s_v4: B2ats_v2 es burstable, con procesador AMD EPYC y muy barato; D4s_v4 es de uso general, sin créditos, con 4 vCPU Intel Xeon y 16 GiB, y cuesta unas 20 veces más.

8. ¿Aumentar el tamaño de la VM es una buena solución en este escenario? ¿Qué pasa con la FibonacciApp cuando cambiamos el tamaño de la VM?

No es una buena solución. La aplicación usa un solo hilo, así que los núcleos adicionales quedan sin usar; lo único que importa es la velocidad de cada núcleo, y en este caso los núcleos de D4s_v4 fueron más lentos que los de B2ats_v2, por lo que la VM grande

respondió peor. Cuando se cambia el tamaño, Azure reinicia la VM y la FibonacciApp se detiene: hubo que entrar por SSH y volver a iniciarla con forever, así que el servicio no estuvo disponible durante varios minutos.

9. ¿Qué pasa con la infraestructura cuando cambia el tamaño de la VM? ¿Qué efectos negativos implica?

Azure detiene la VM, puede moverla a otro servidor físico que tenga el tamaño solicitado y la vuelve a encender. Los efectos negativos son el tiempo sin servicio (downtime) durante el reinicio, la pérdida de los procesos en ejecución y de la información en memoria o en el disco temporal, el riesgo de que cambie la IP pública si es dinámica, la necesidad de que el tamaño esté disponible en la región (B4ms no lo estaba) y un aumento grande del costo. Además, el escalamiento vertical tiene un límite: el tamaño máximo de máquina que ofrece la región.

10. ¿Hubo mejora en el consumo de CPU o en los tiempos de respuesta? Sí/No, ¿por qué?

No. El porcentaje de CPU bajó de cerca de 50 % a cerca de 25 %, pero no es una mejora real: el mismo trabajo ocupa un núcleo entero y solo se divide entre más núcleos totales. Los tiempos de respuesta empeoraron de 6.27 s a 9.22 s en promedio (cerca de 47 % más lentos) porque cada núcleo de D4s_v4 es más lento, y en las pruebas concurrentes aparecieron fallos que con B2ats_v2 no existían.

11. Aumente la cantidad de ejecuciones paralelas del comando de Postman a 4. ¿El comportamiento del sistema es porcentualmente mejor?

No. Con 2 ejecuciones en paralelo falló el 45 % de las peticiones y con 4 falló el 47.5 %. Además, el tiempo promedio de las peticiones exitosas pasó de 11.5 s a 25.6 s, porque ahora hay más peticiones esperando en fila para el único hilo de Node.js. La CPU se mantuvo en 25 %, lo que confirma que la máquina tenía capacidad libre que la aplicación no podía aprovechar.

3. Parte 2 - Escalabilidad horizontal

3.1 Preparación y límites de la suscripción

En Mexico Central la cuota de vCPU estaba en 2 de 6 por la VM de la Parte 1, que debía seguir encendida. Se solicitó subir el límite a 12 vCPU, pero la solicitud fue rechazada. Como la cuota es por región, la Parte 2 se hizo en otra región. Los tamaños de 1 vCPU (B1s, B1ms) no estaban disponibles en ninguna de las regiones permitidas, y en Brazil South solo había tamaños de la serie D, mucho más costosos. Finalmente se eligió Spain Central, donde el tamaño B2ats_v2 estaba disponible (8,91 US$ al mes).

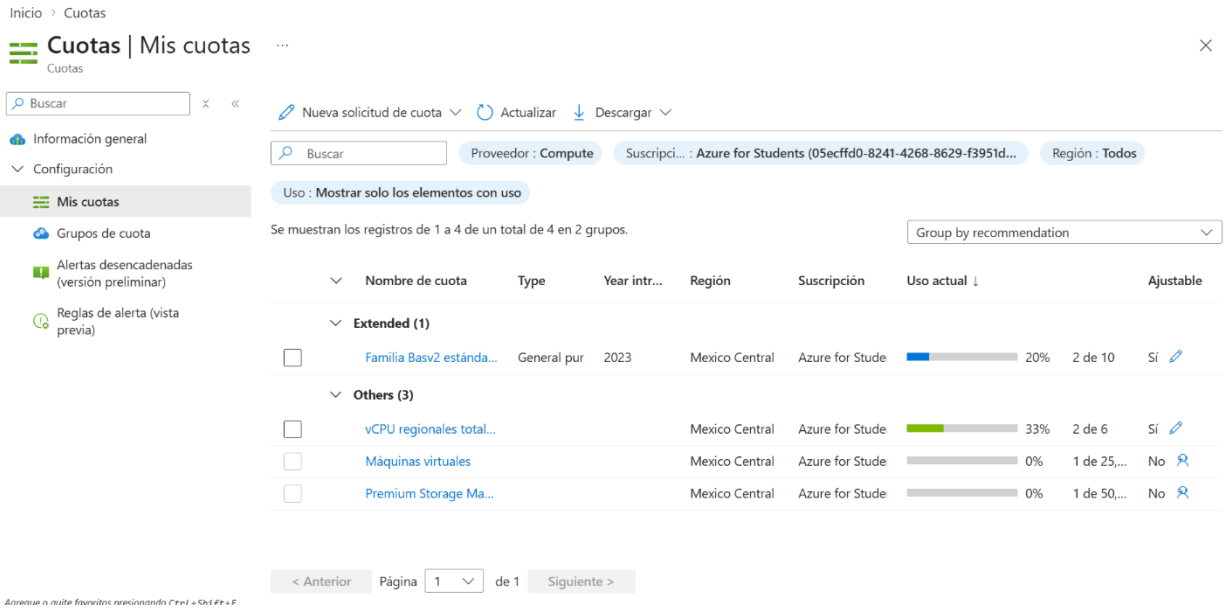


Figura 30. Cuota de vCPU regionales en Mexico Central (2 de 6).

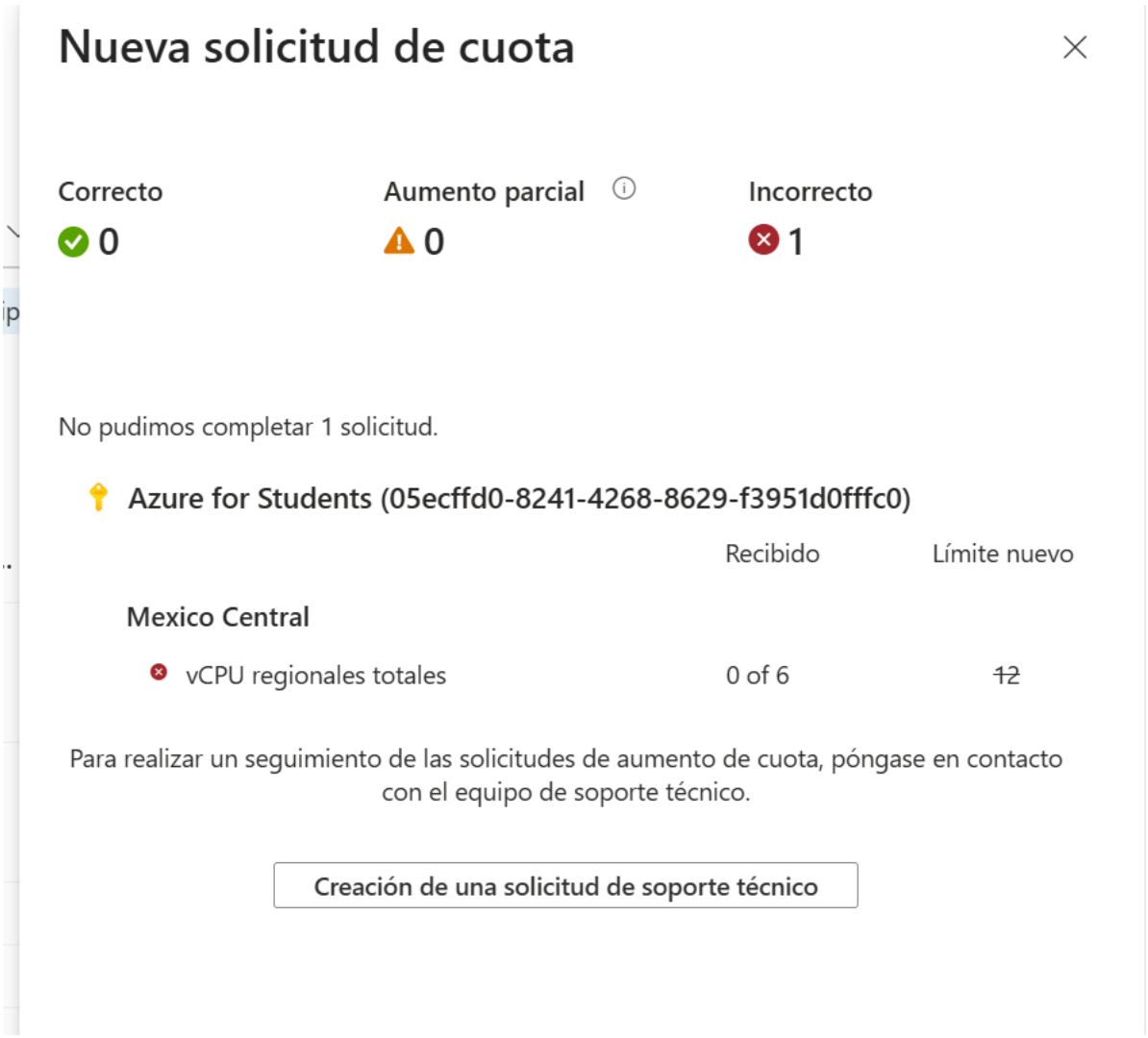


Figura 31. Solicitud de aumento de cuota rechazada.

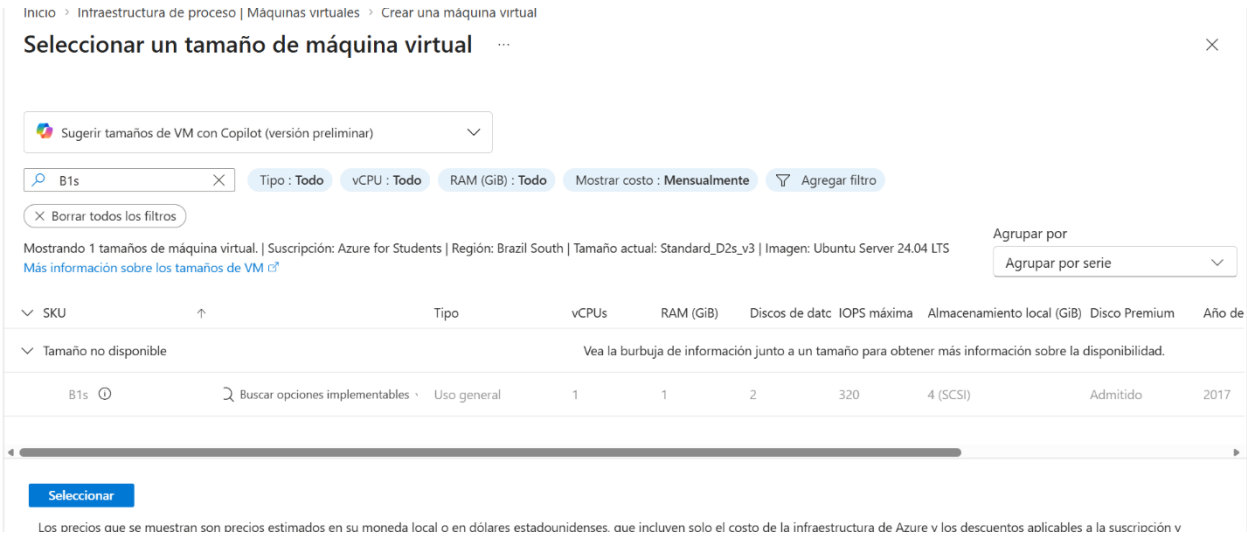


Figura 32. El tamaño B1s no está disponible en Brazil South.

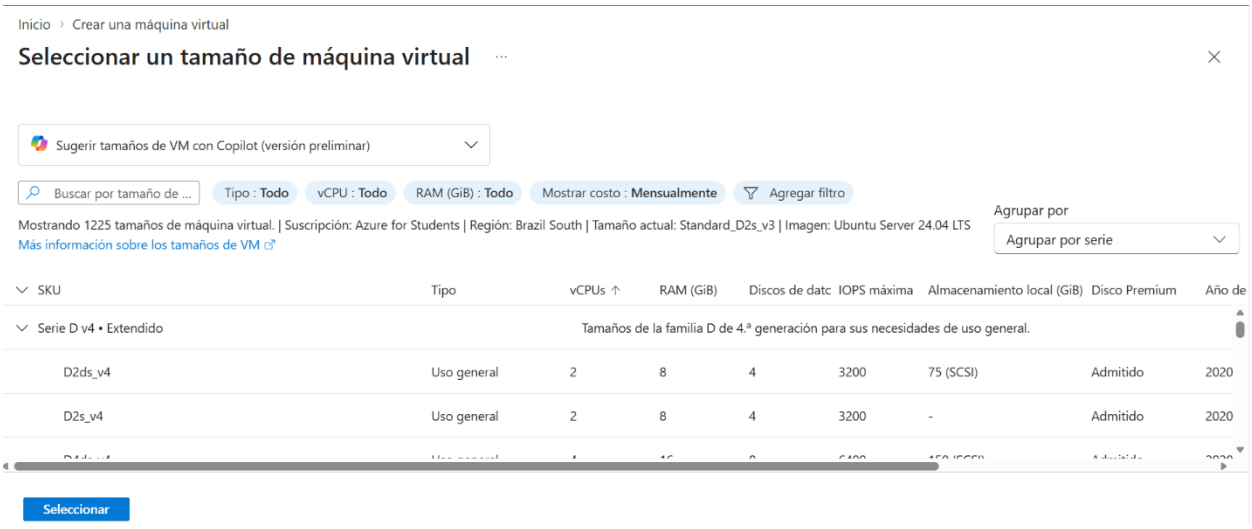


Figura 33. En Brazil South solo había tamaños de la serie D.

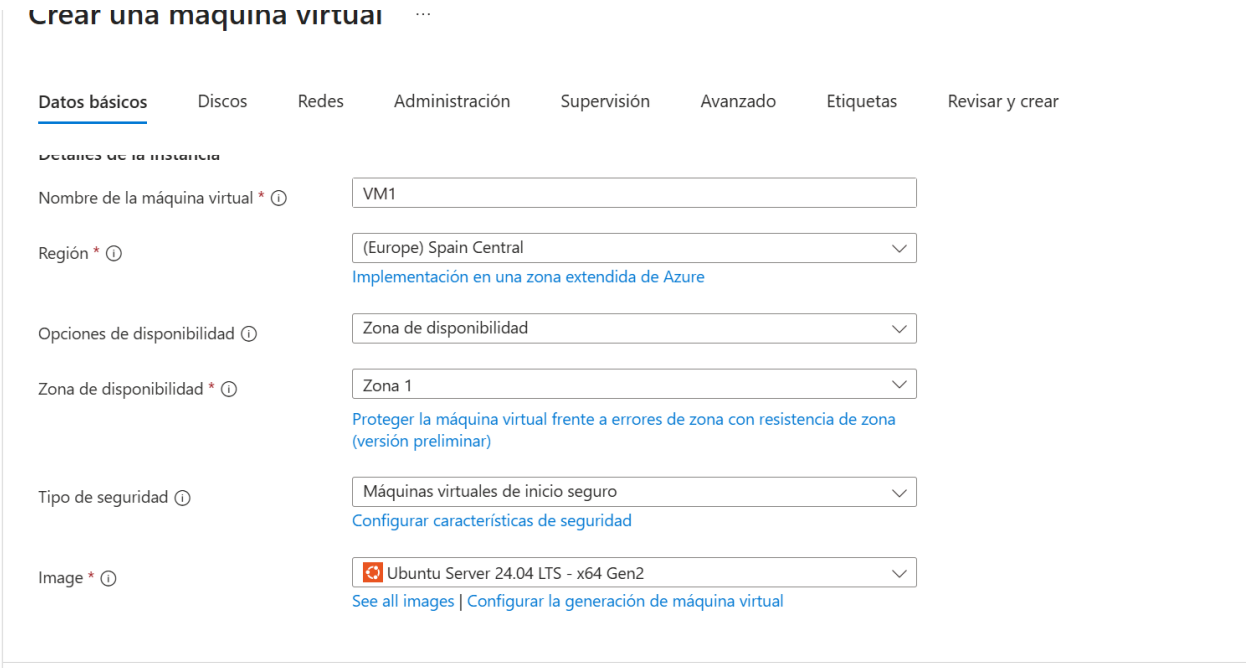


Figura 34. Región Spain Central con zonas de disponibilidad.

3.2 Grupo de recursos y red virtual

Se creó el grupo de recursos SCALABILITY_LAB_HORIZONTAL en Spain Central y dentro de él la red virtual SCALABILITY_LAB_VNET, con espacio de direcciones 10.0.0.0/16 y la subred default 10.0.0.0/24. La red virtual se creó antes que el balanceador porque el portal actual la necesita para definir el backend pool.

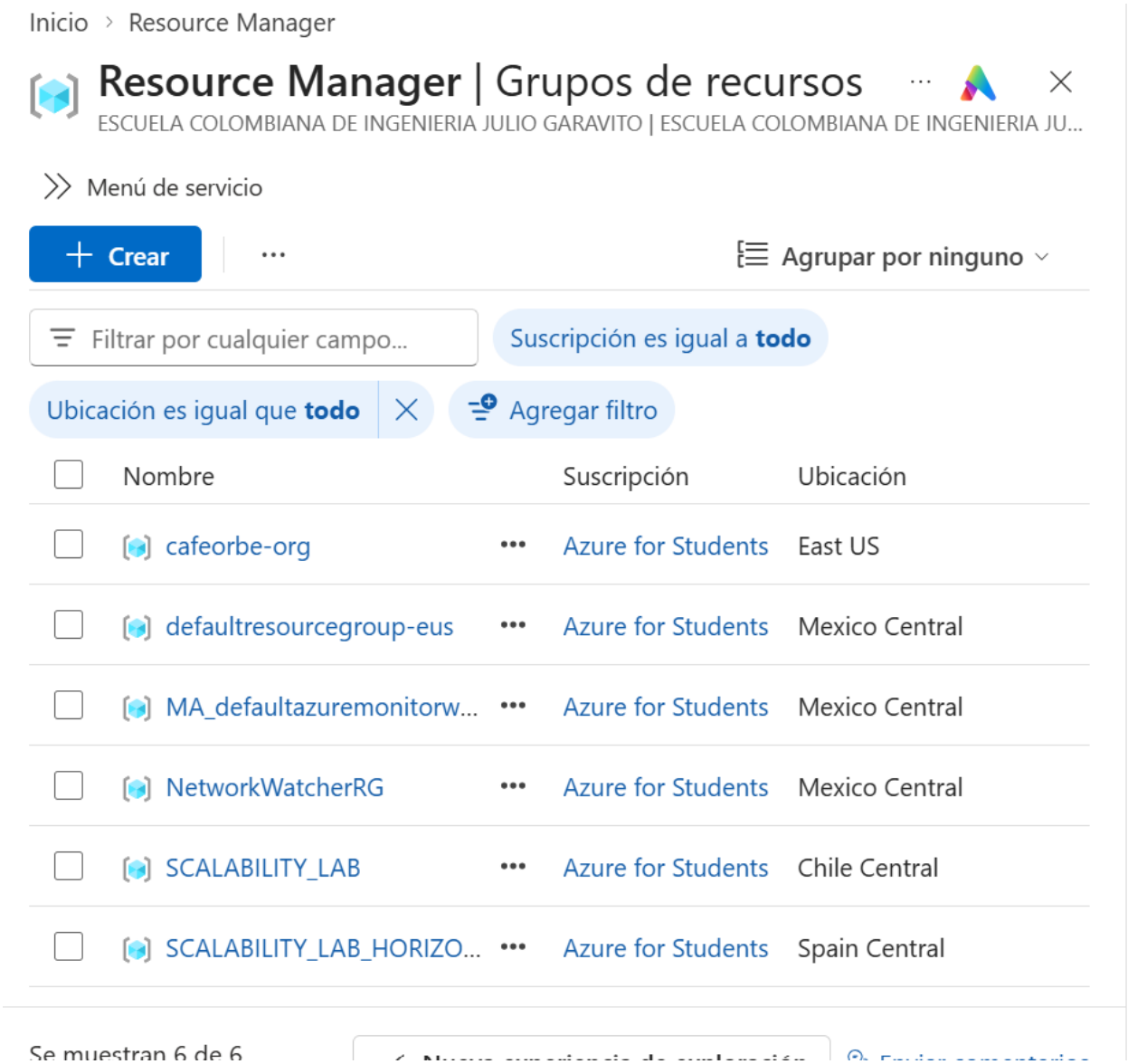


Figura 35. Grupo de recursos SCALABILITY_LAB_HORIZONTAL en Spain Central.



Figura 36. Datos básicos de la red virtual SCALABILITY_LAB_VNET.

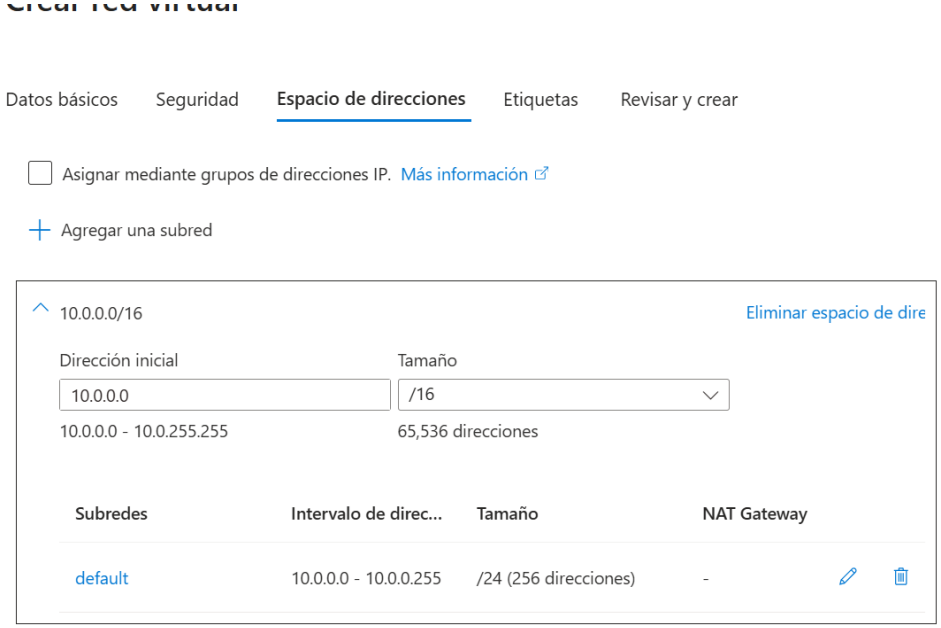


Figura 37. Espacio de direcciones 10.0.0.0/16 y subred default 10.0.0.0/24.

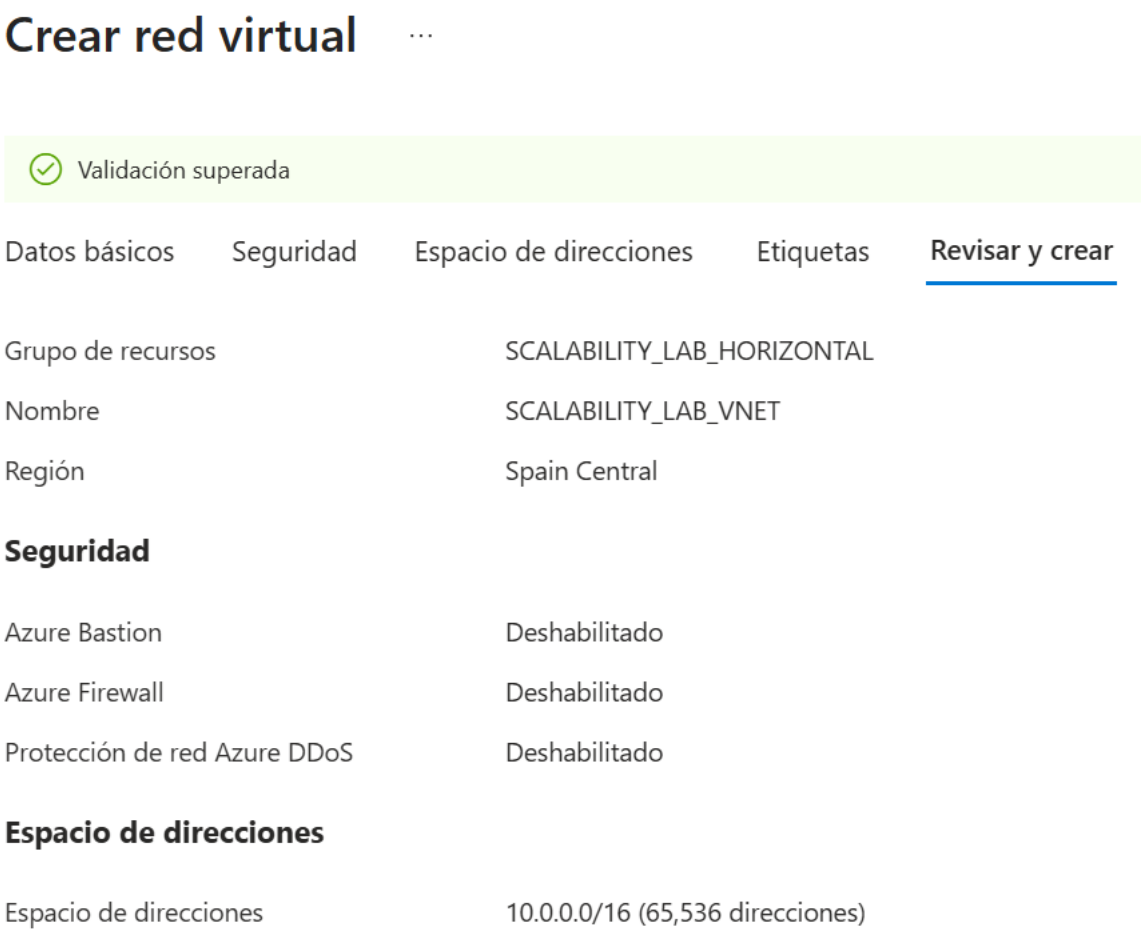


Figura 38. Revisión de la configuración de la red virtual antes de crearla.



Figura 39. Red virtual creada.

3.3 Balanceador de carga

Se creó el balanceador SCALABILITY_LAB_LB con SKU Estándar, tipo público y nivel regional. Su configuración de front-end LB_FRONTEND usa la IP pública SCALABILITY_LAB_LB_IP (68.221.5.170), con redundancia de zona. Se creó el backend pool SCALABILITY_LAB_BACKEND_POOL asociado a la red virtual y la regla FIBONACCI_LB_RULE, que recibe tráfico TCP en el puerto 80 y lo reenvía al puerto 3000 de las VMs, sin persistencia de sesión. El sondeo de estado FIBONACCI_HEALTH_PROBE revisa el puerto 3000 por TCP cada 5 segundos. Se eligió TCP en lugar de HTTP porque, mientras Node.js calcula un Fibonacci, no puede responder peticiones HTTP, y un sondeo HTTP marcaría la VM como caída aunque estuviera trabajando.

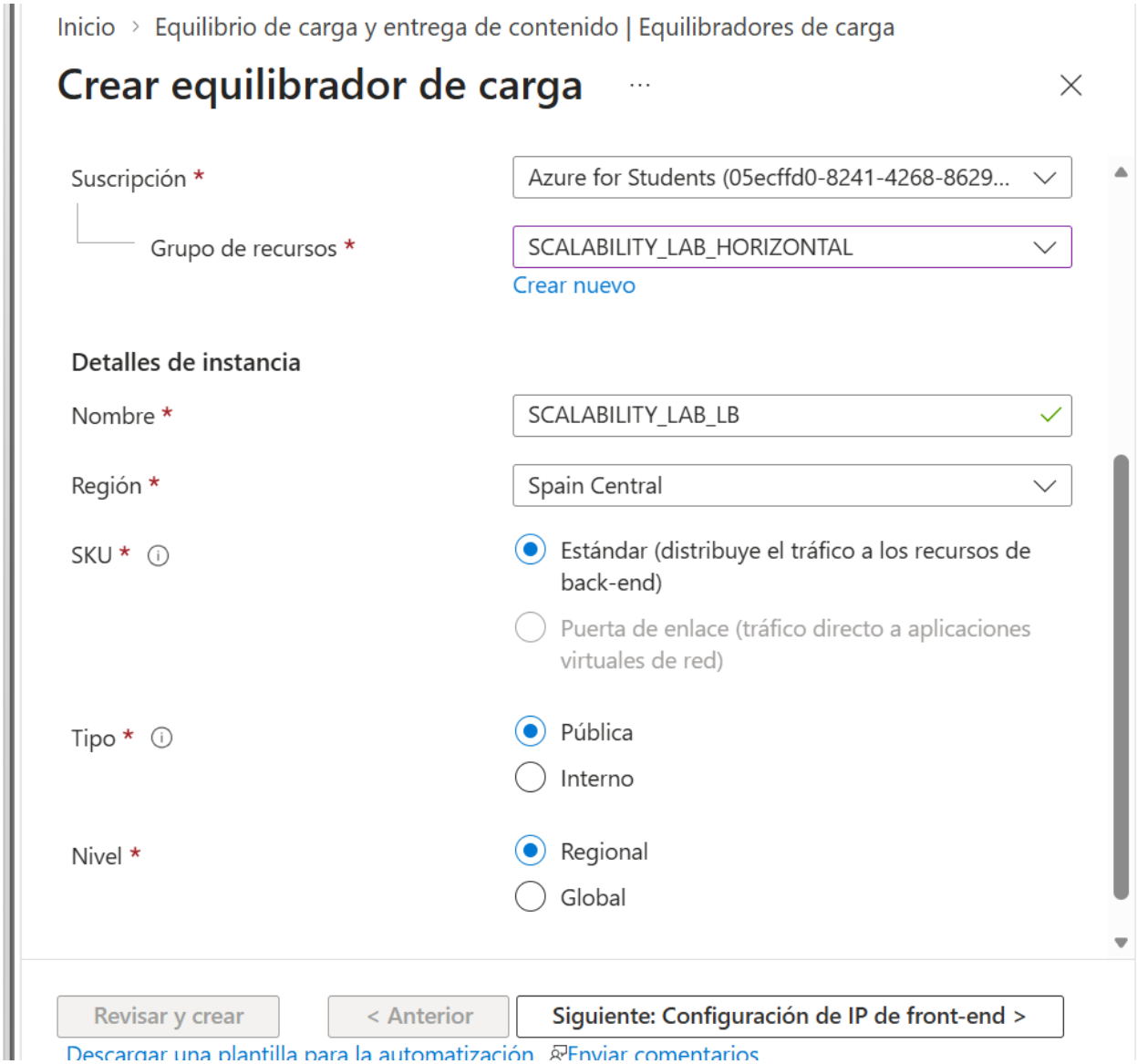


Figura 40. Datos básicos del balanceador: SKU Estándar, público y regional.

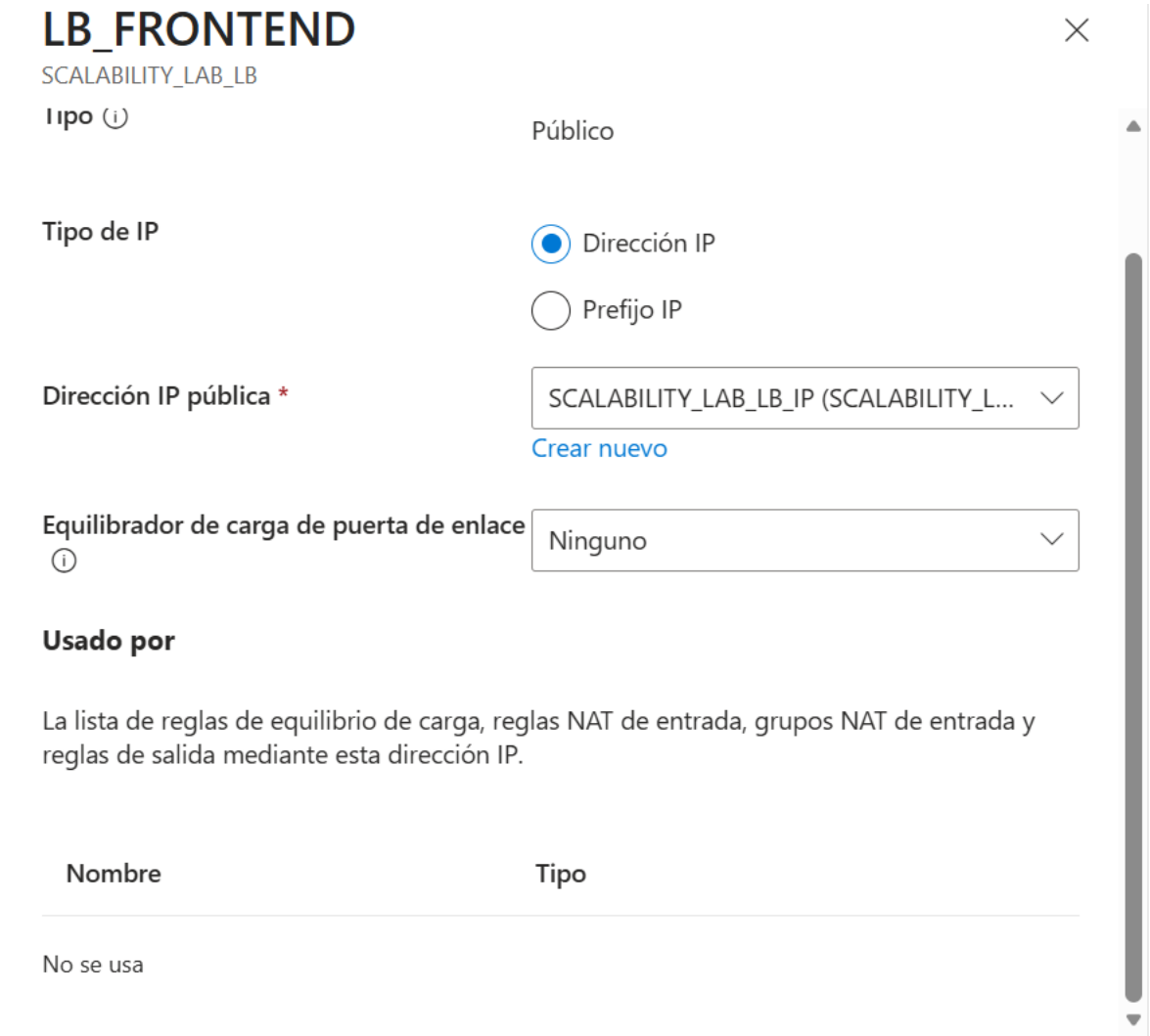


Figura 41. Configuración de IP de front-end LB_FRONTEND.

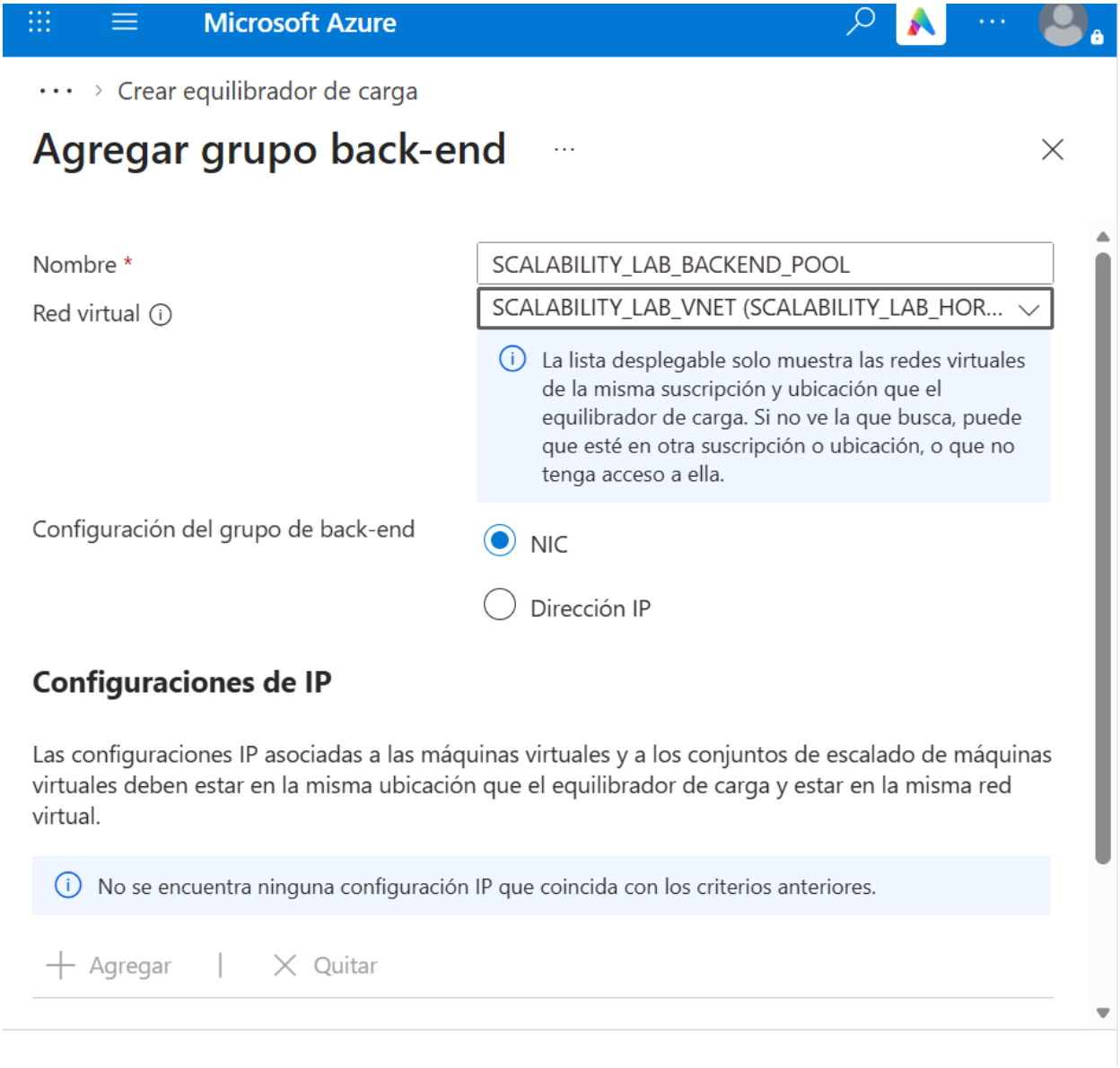


Figura 42. Backend pool SCALABILITY_LAB_BACKEND_POOL.

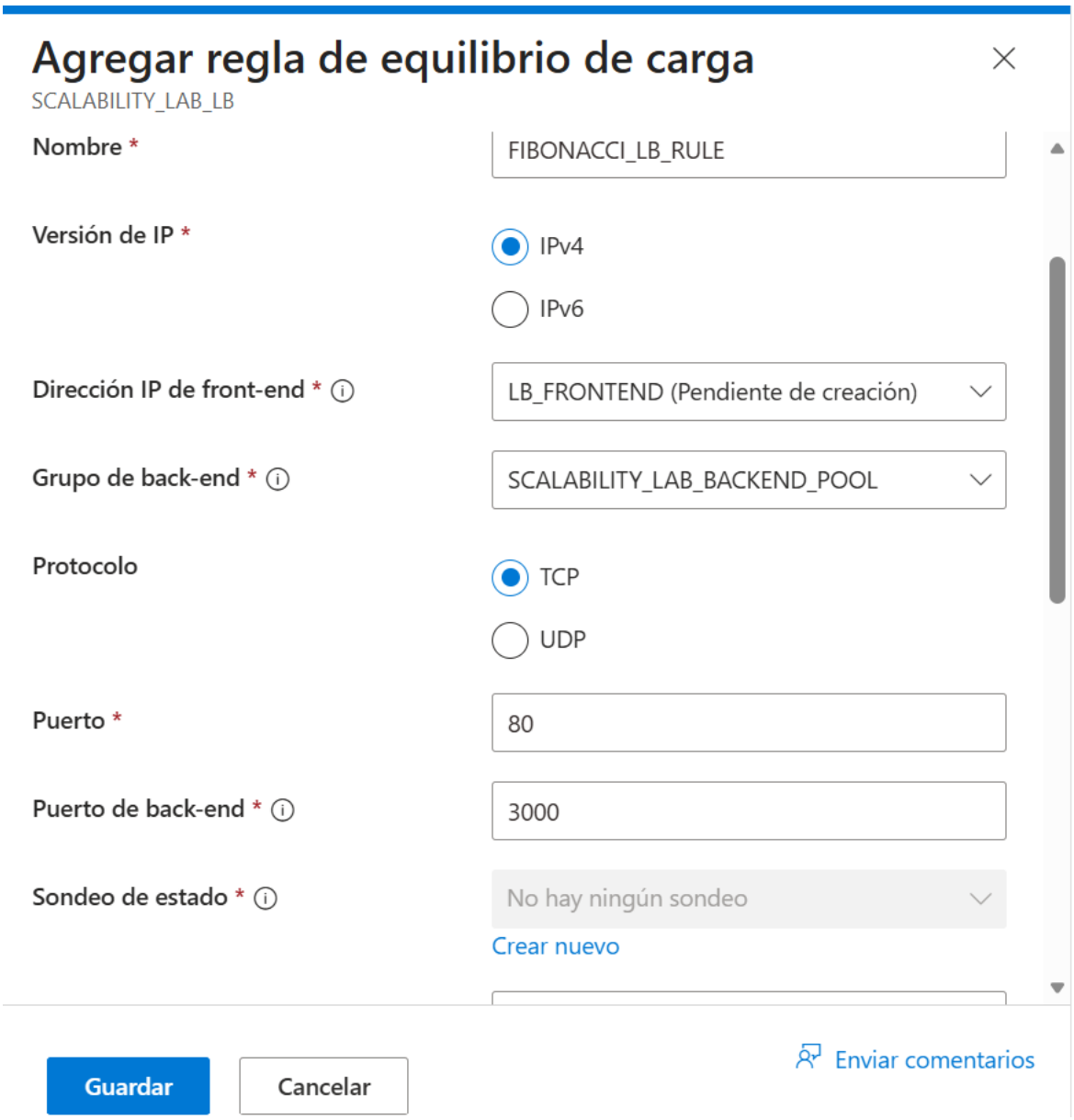


Figura 43. Regla de equilibrio de carga FIBONACCI_LB_RULE (puerto 80 a 3000).

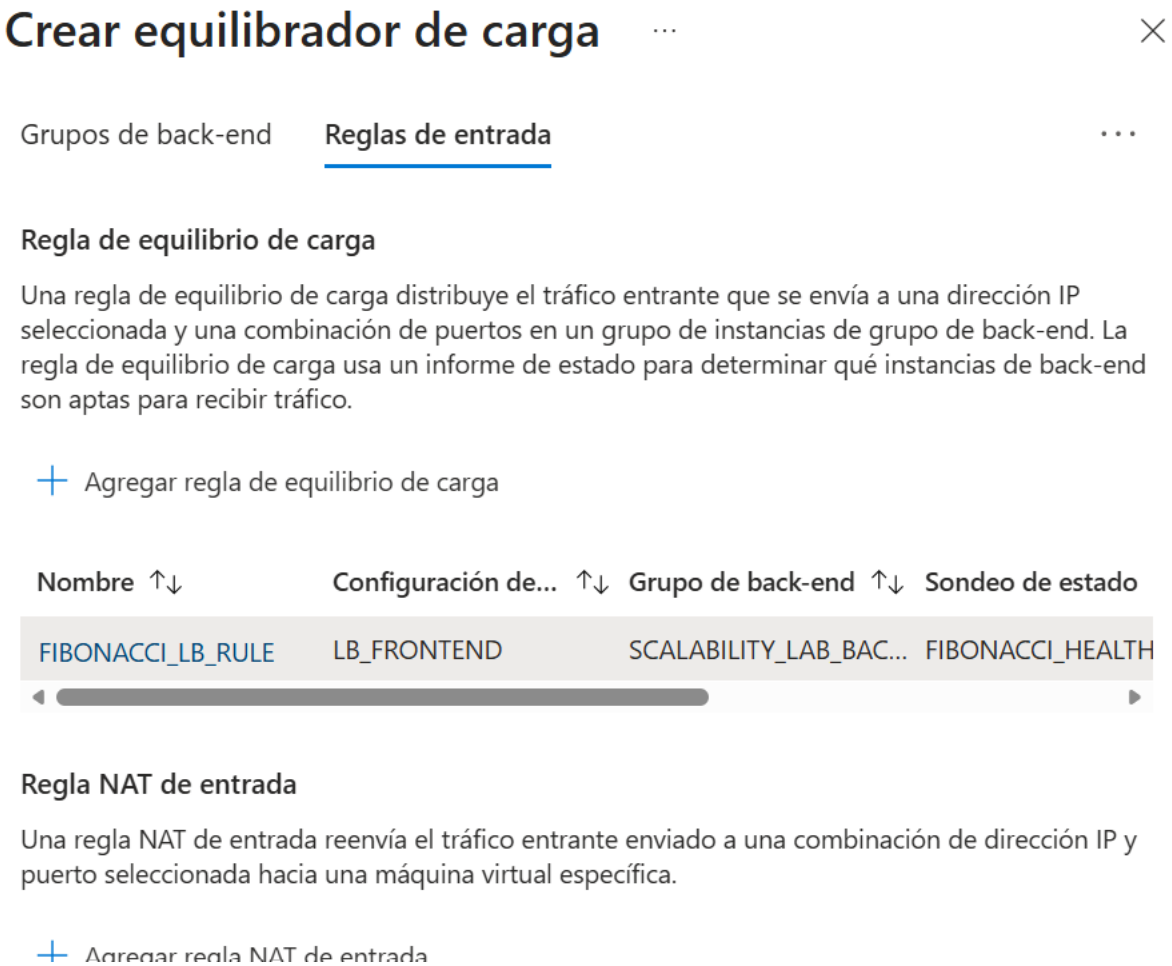


Figura 44. Sondeo de estado y persistencia de sesión de la regla.

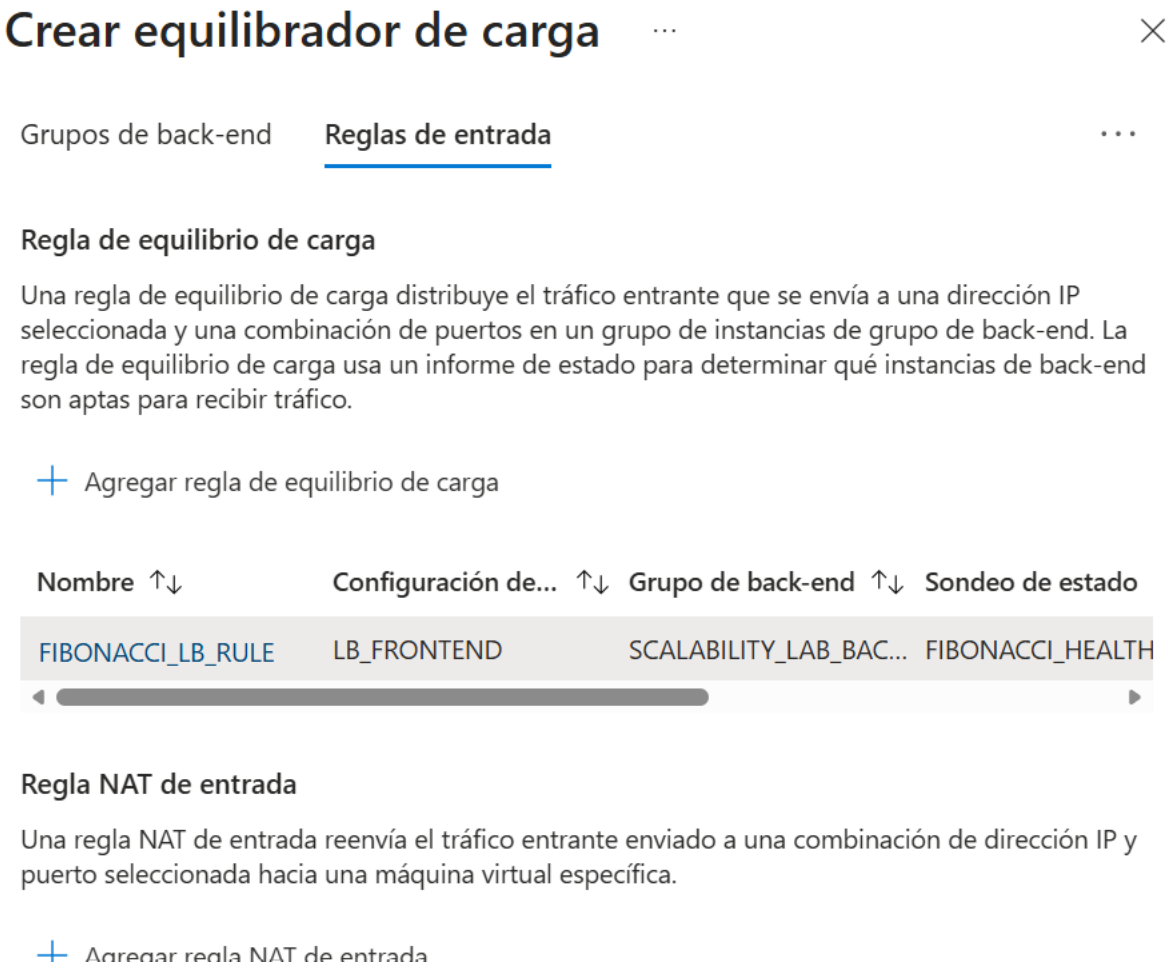


Figura 45. Regla FIBONACCI_LB_RULE guardada en las reglas de entrada.

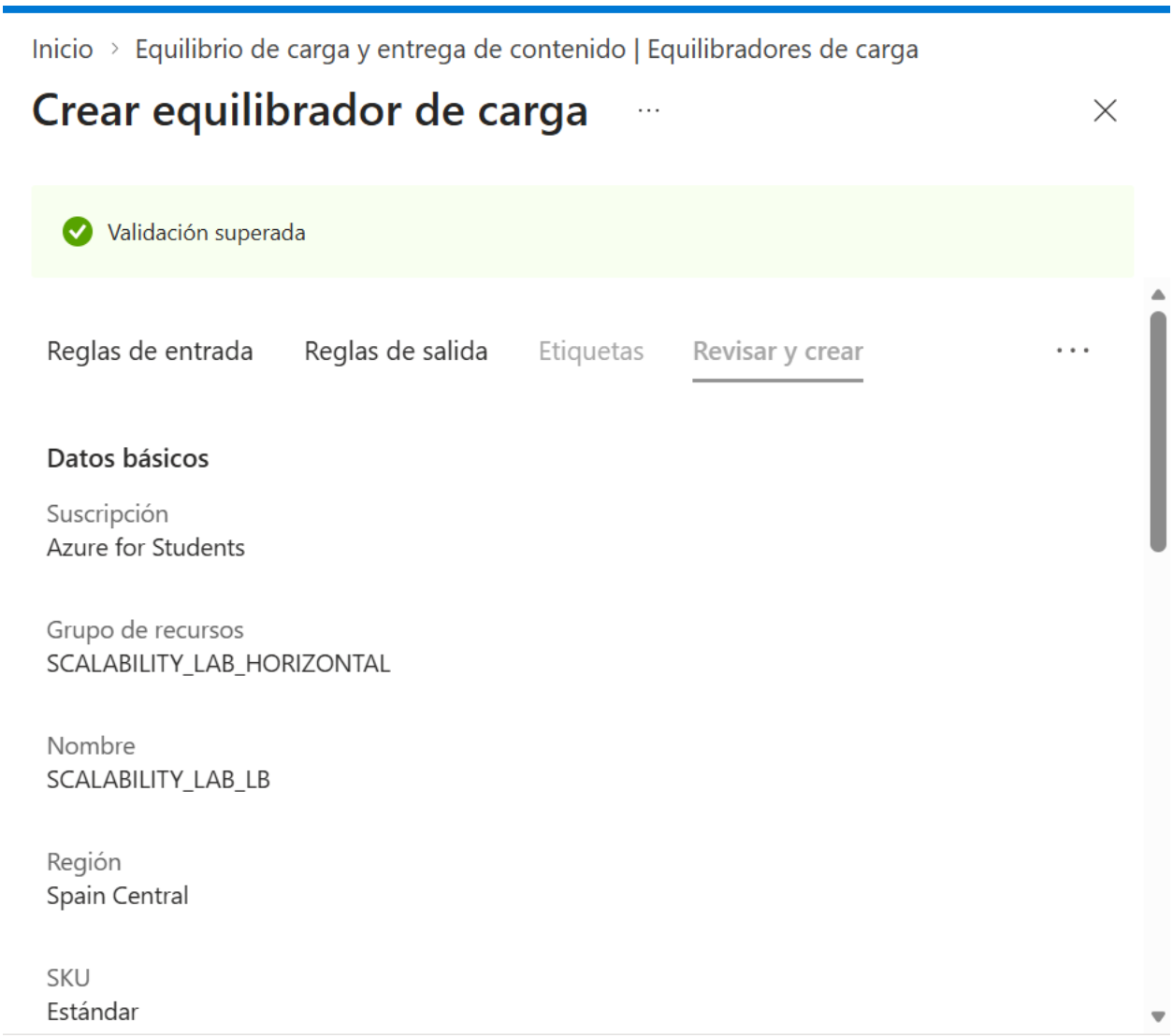


Figura 46. Revisión de la configuración del balanceador antes de crearlo.

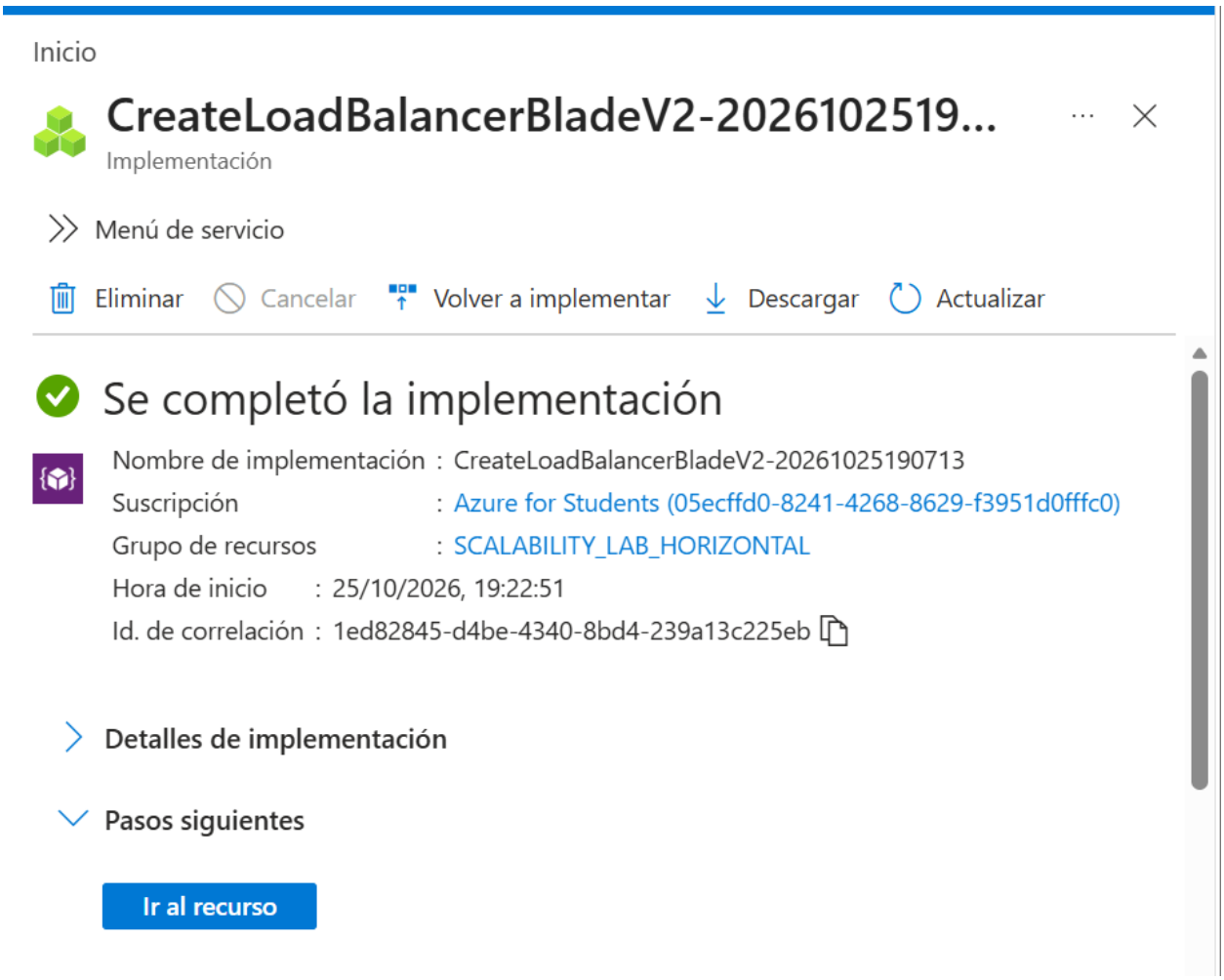


Figura 47. Balanceador de carga creado.

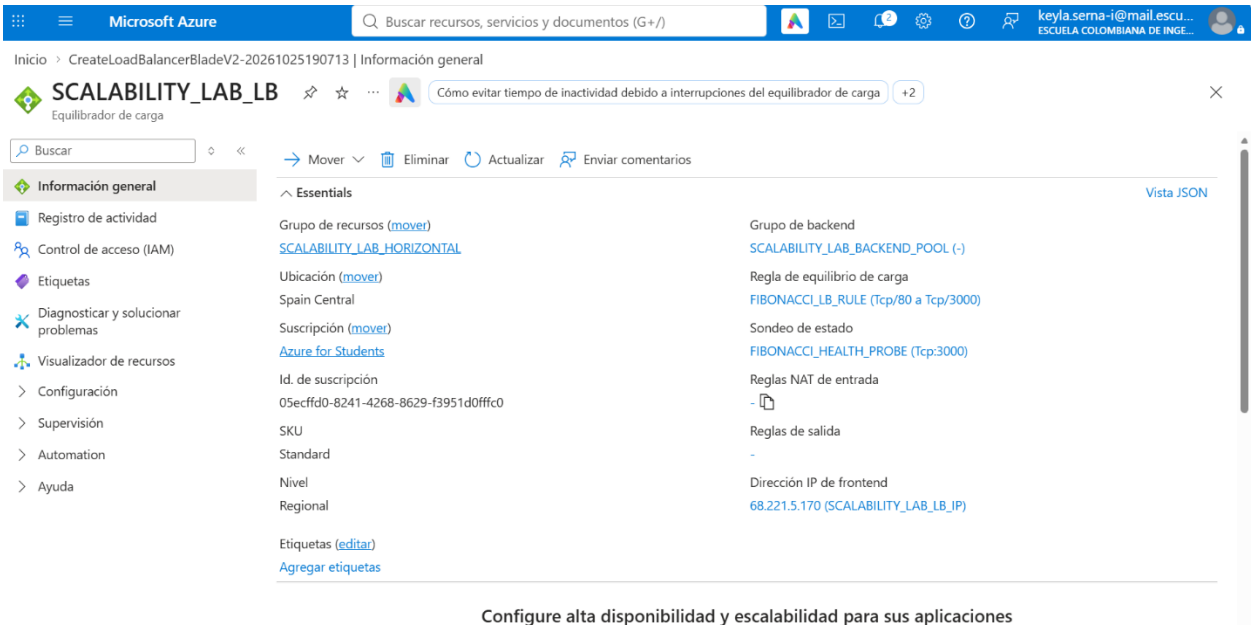


Figura 48. IP pública del balanceador: 68.221.5.170.

3.4 Máquinas virtuales

Se crearon tres VMs con tamaño B2ats_v2, Ubuntu 24.04, el usuario scalability_lab y la misma llave SSH, conectadas a la red SCALABILITY_LAB_VNET, al grupo de seguridad de red SCALABILITY_LAB_NSG (que permite los puertos 22 y 3000) y al backend pool del balanceador. Con esto se usó el total de la cuota de 6 vCPU de la región.

No fue posible ubicar cada VM en una zona distinta: en Spain Central el tamaño B2ats_v2 solo está disponible en las zonas 1 y 3, y la zona 2 no ofrece tamaños para la suscripción (error NotAvailableForSubscription). Por eso VM1 y VM3 quedaron en la zona 1 y VM2 en la zona 3. Además, la suscripción solo permite 3 IPs públicas por región, ya ocupadas por el balanceador, VM1 y VM2, así que VM3 se creó sin IP pública; no la necesita, porque el tráfico de los usuarios le llega a través del balanceador.

VM1

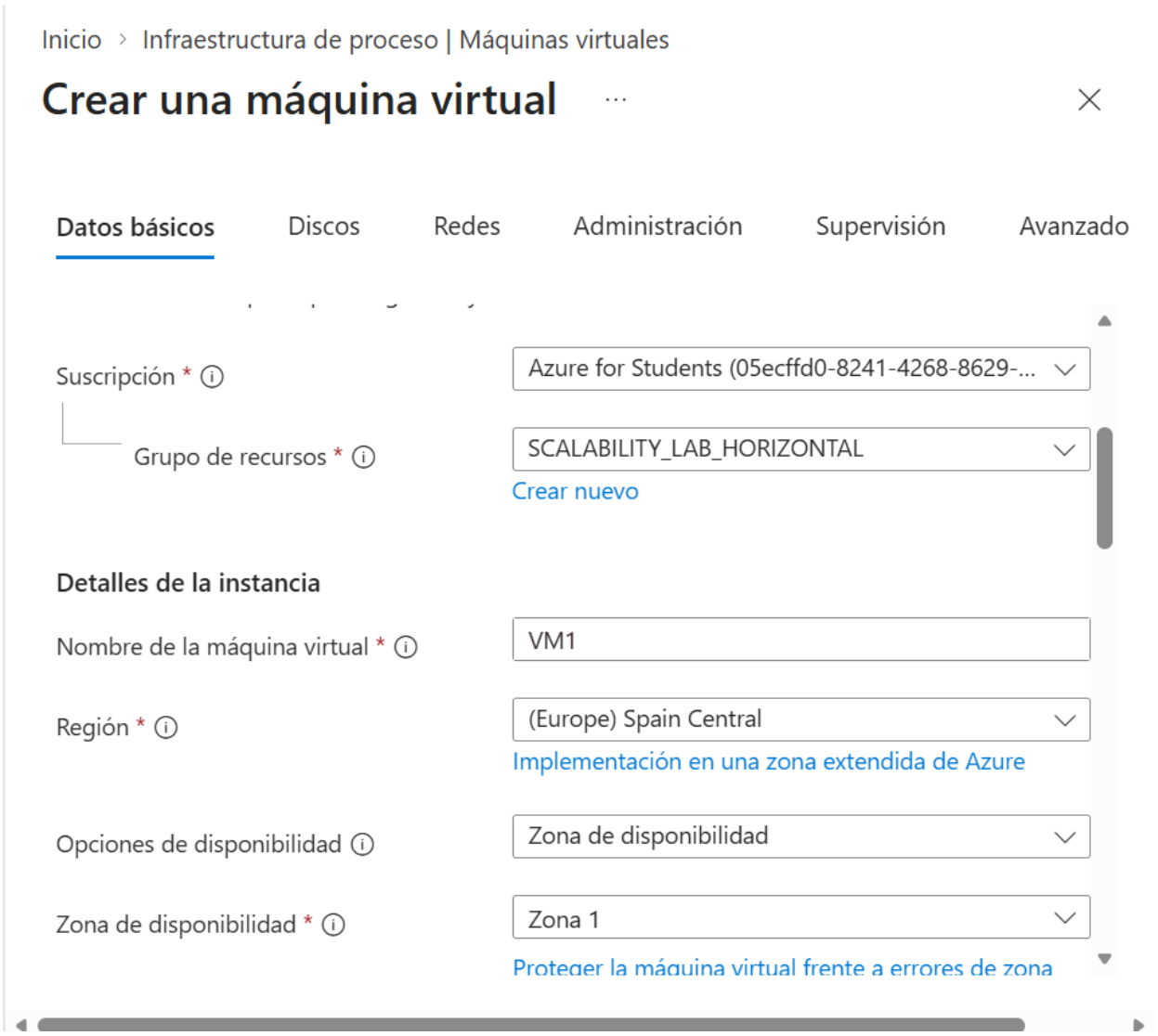


Figura 49. Datos básicos de VM1 (zona 1, B2ats_v2).

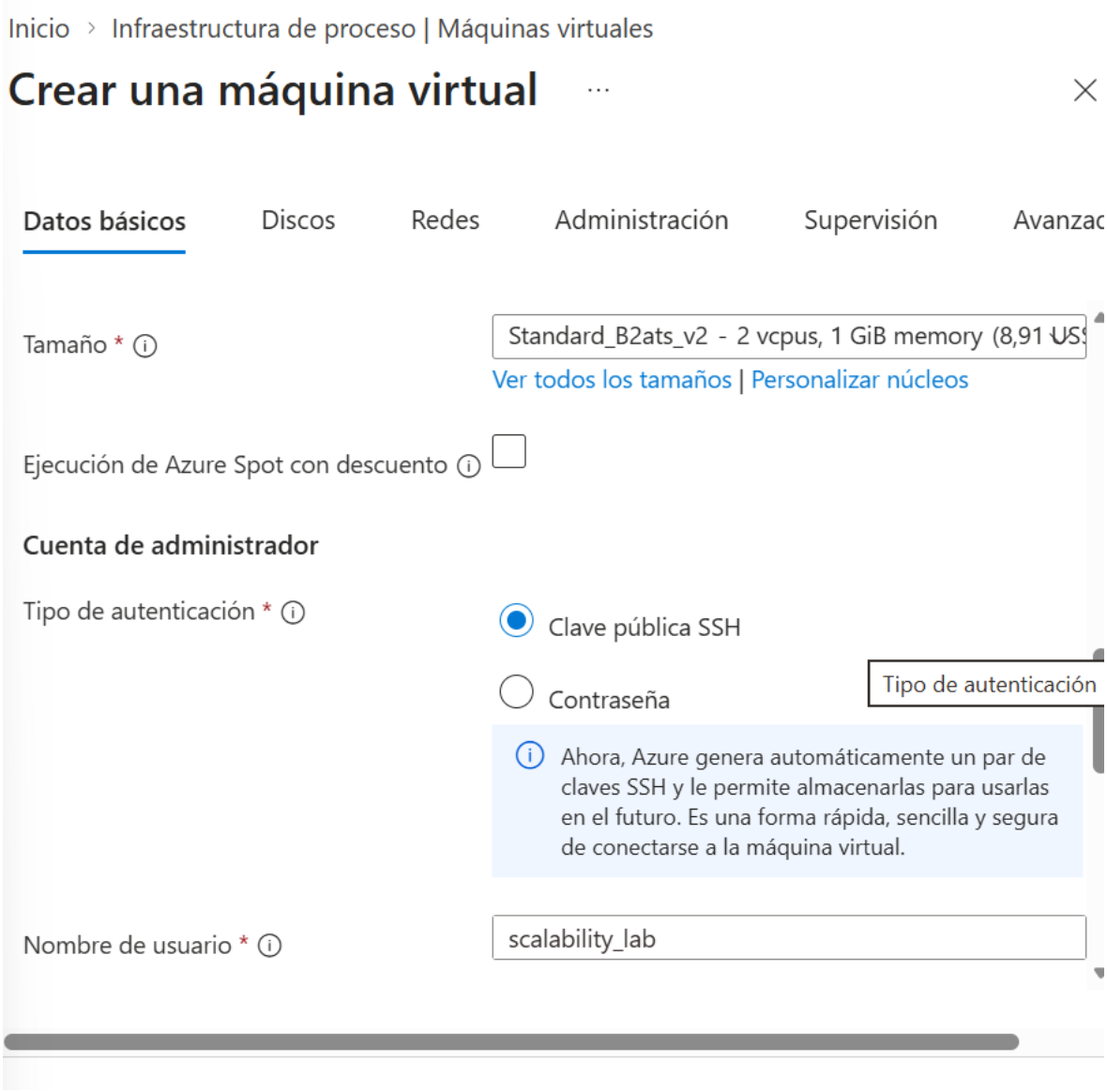


Figura 50. Cuenta de administrador de VM1 con la llave SSH existente.

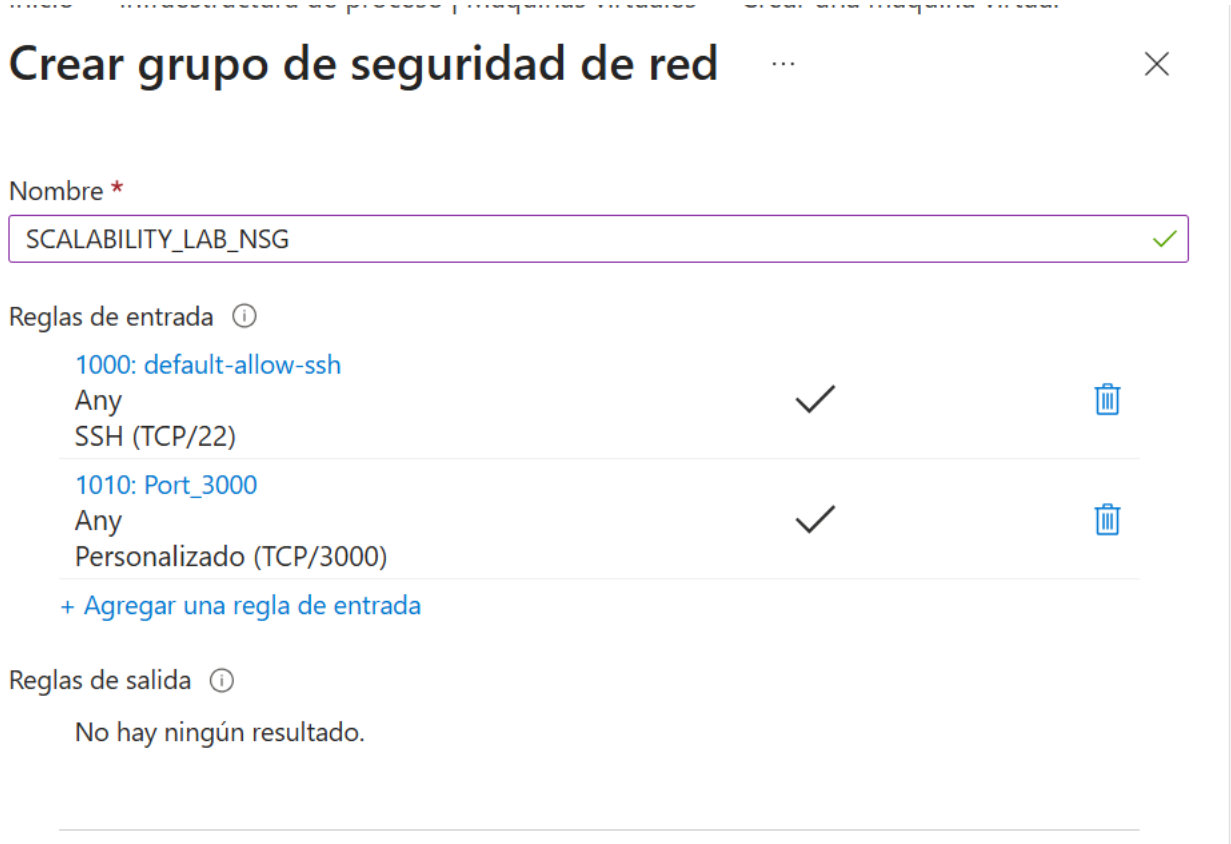


Figura 51. Grupo de seguridad de red SCALABILITY_LAB_NSG con las reglas de los puertos 22 y 3000.

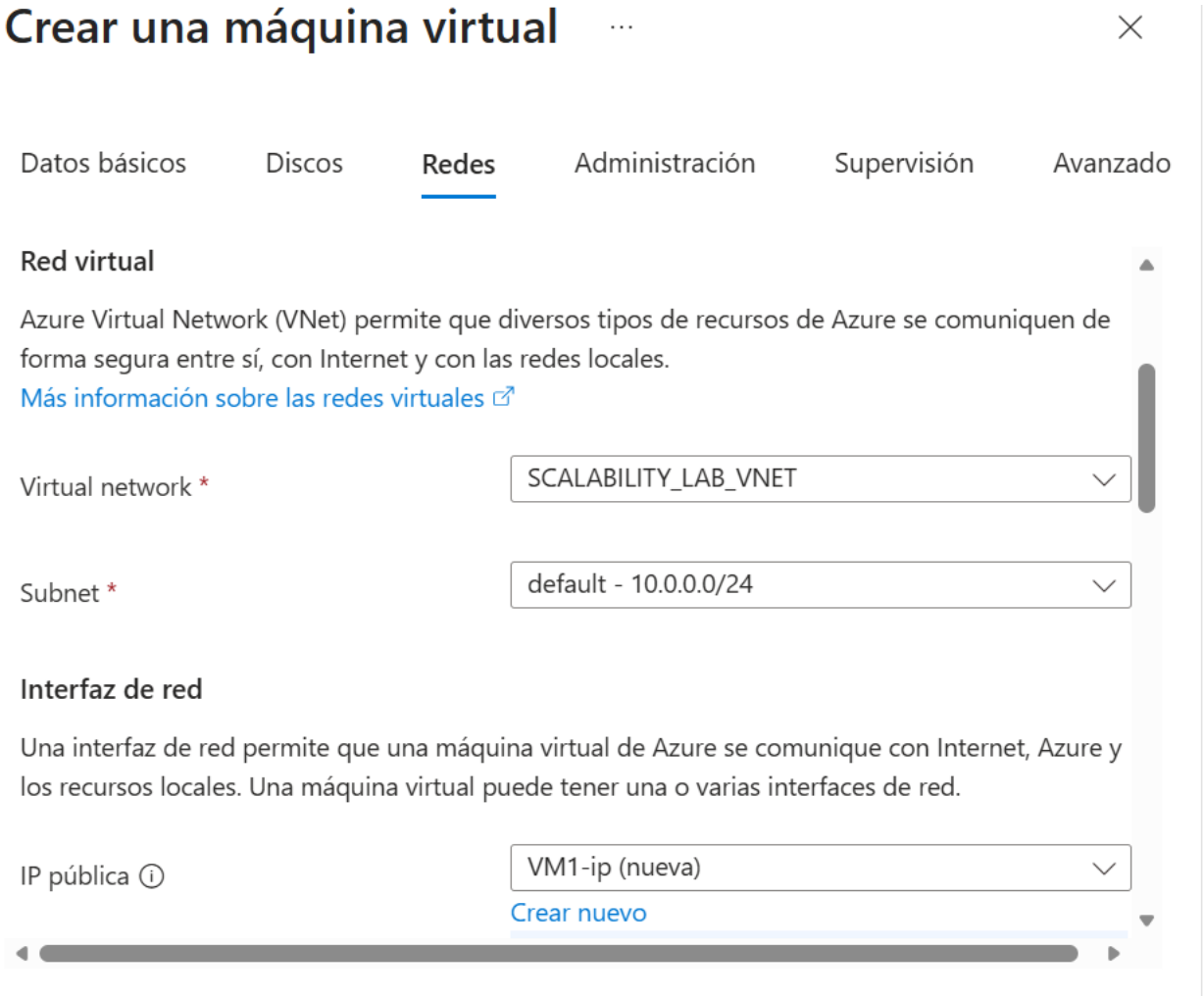


Figura 52. Configuración de red de VM1 (SCALABILITY_LAB_VNET, subred default).



Figura 53. VM1 asociada al balanceador y al backend pool.

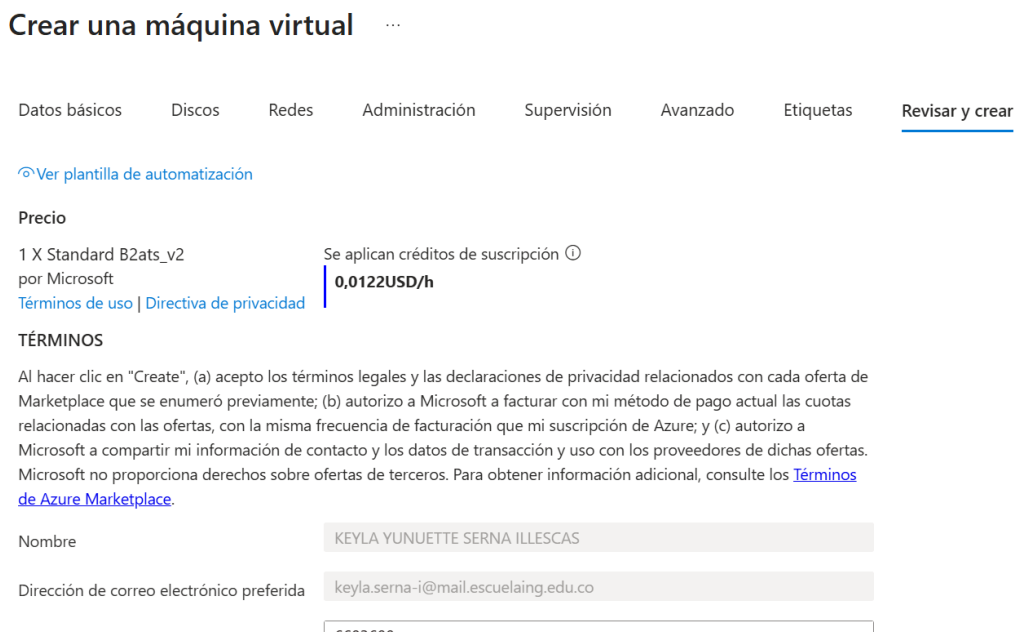


Figura 54. Revisión de la configuración de VM1 antes de crearla.

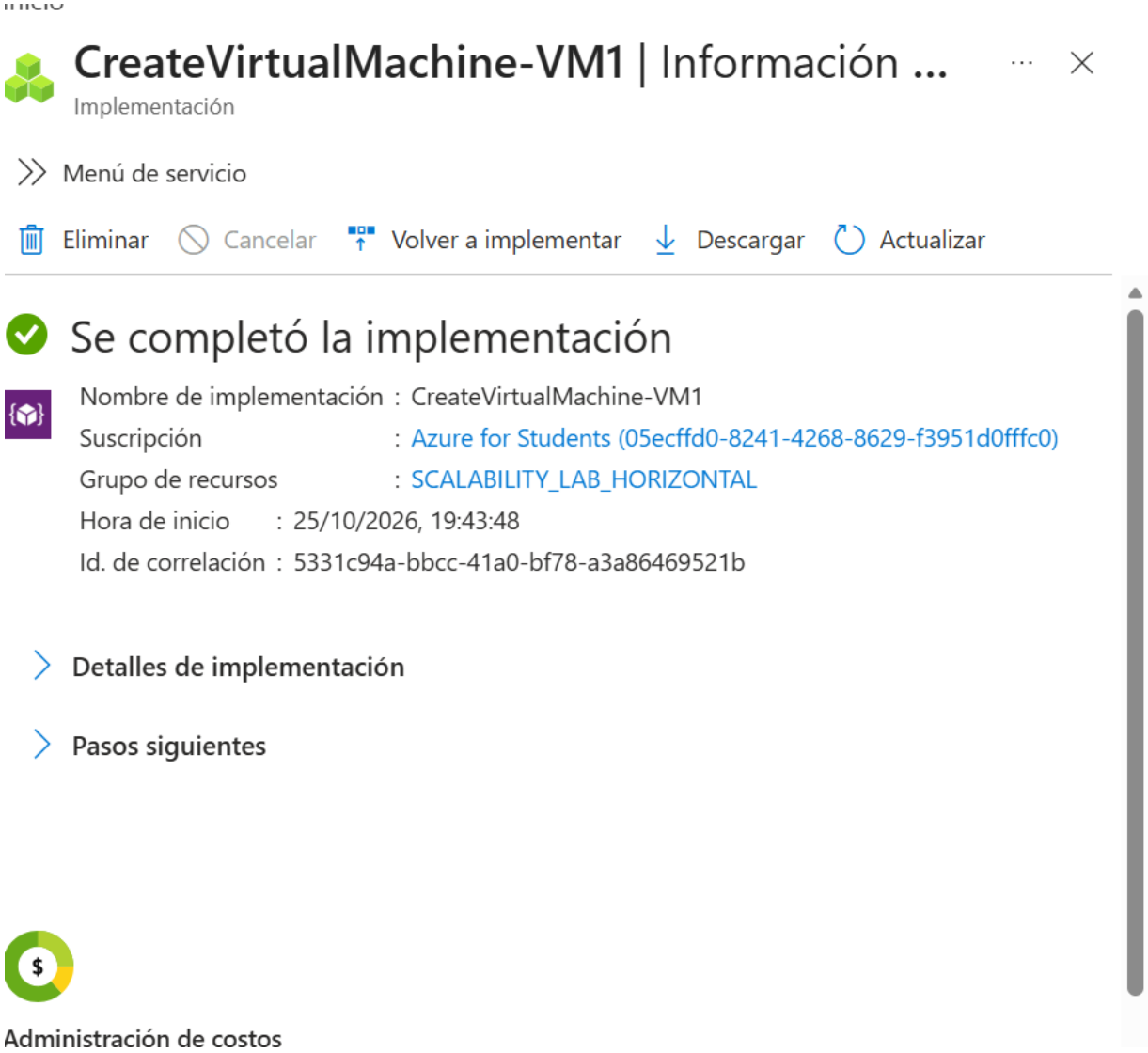


Figura 55. Implementación de VM1 completada.

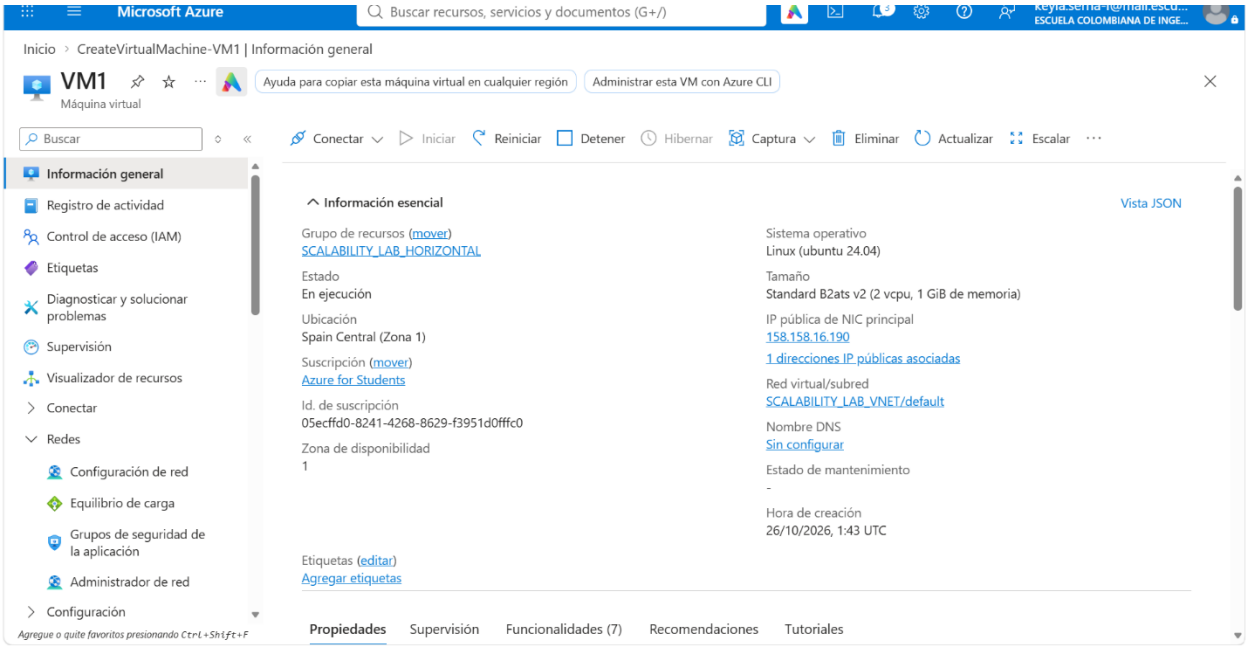


Figura 56. Información general de VM1.

VM2

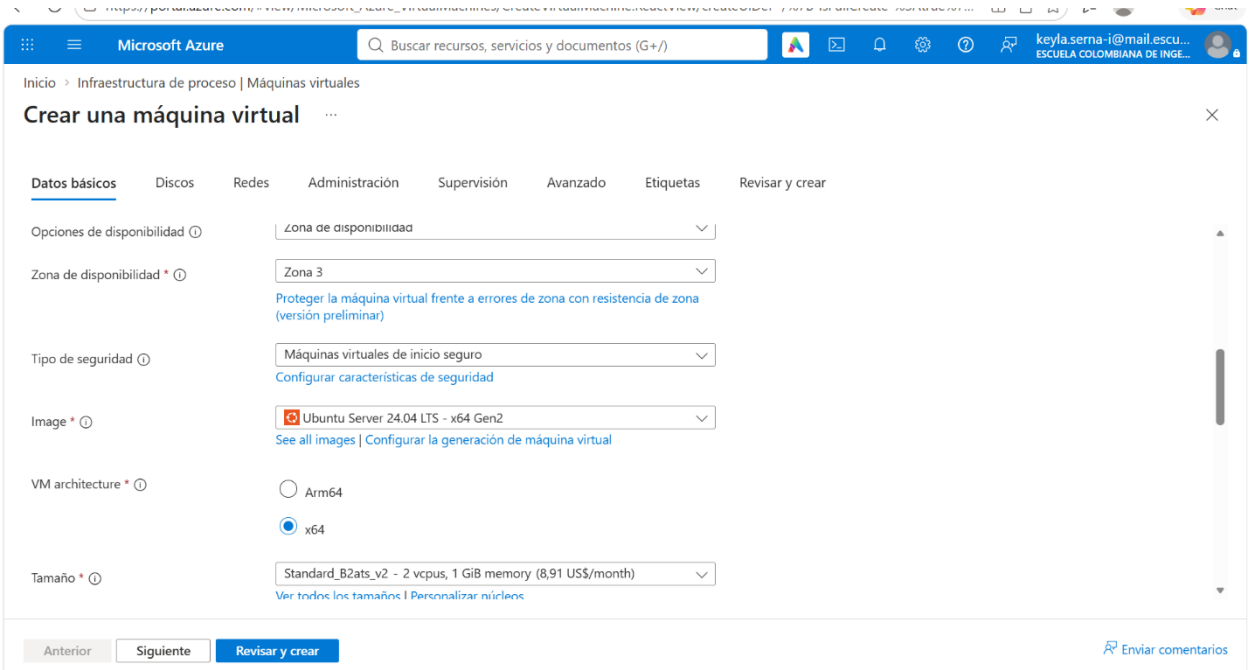


Figura 57. Datos básicos de VM2.

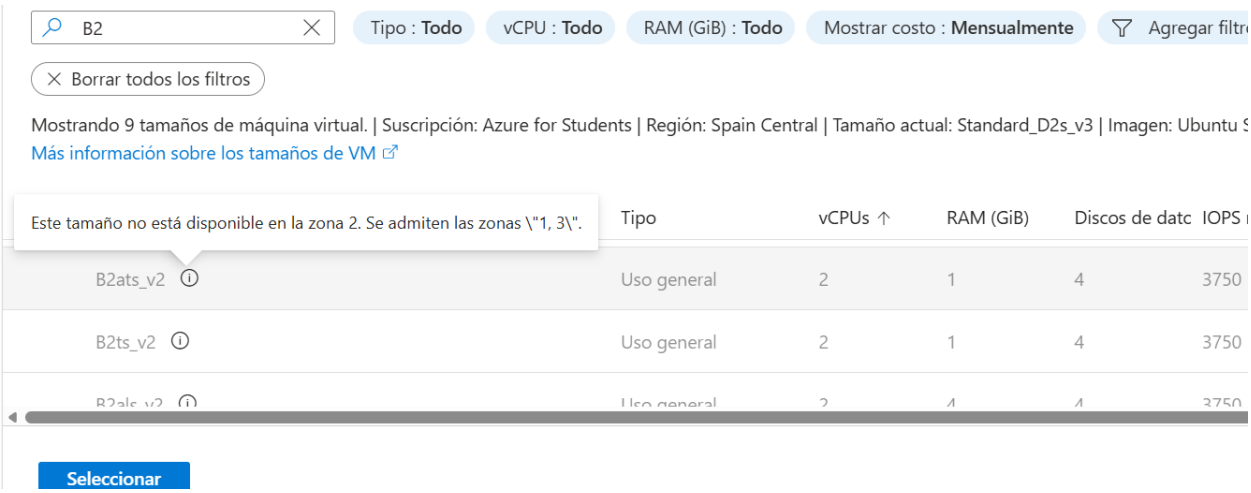


Figura 58. El tamaño B2ats_v2 solo está disponible en las zonas 1 y 3.

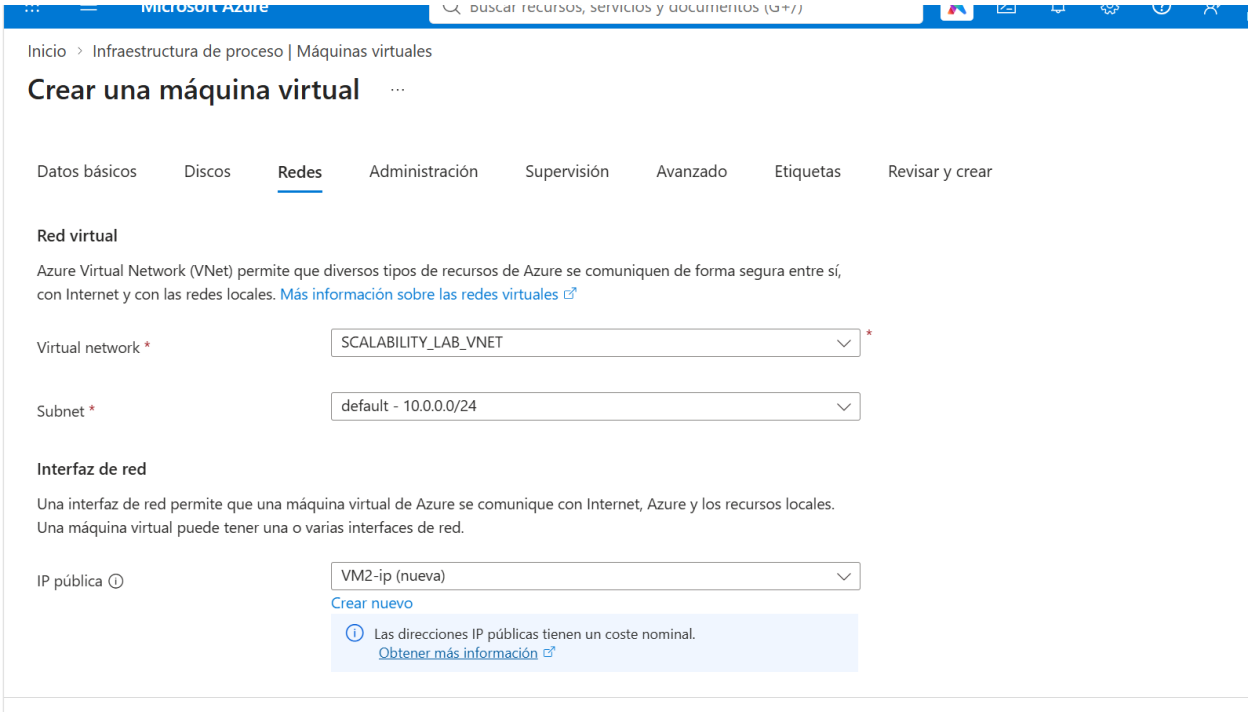


Figura 59. Configuración de red de VM2 con el NSG compartido SCALABILITY_LAB_NSG.

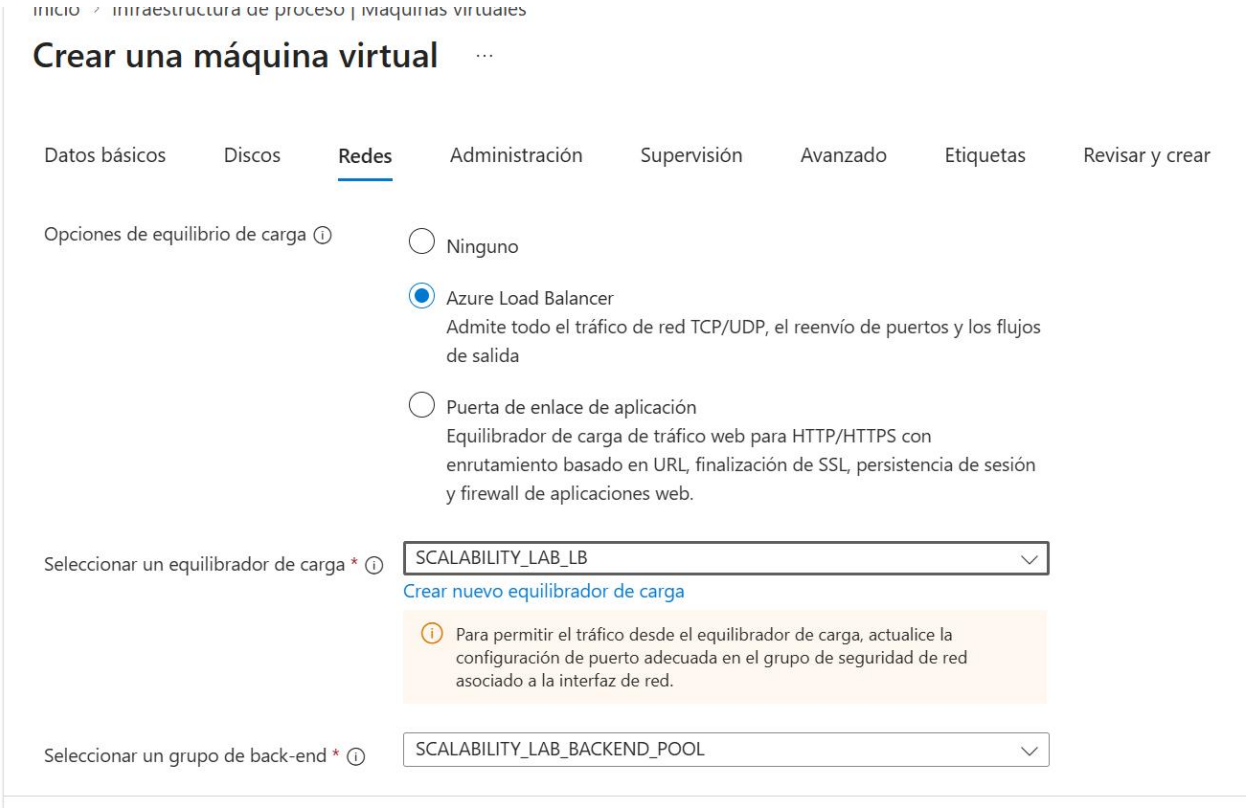


Figura 60. VM2 asociada al balanceador y al backend pool.



Figura 61. Revisión de la configuración de VM2 antes de crearla.

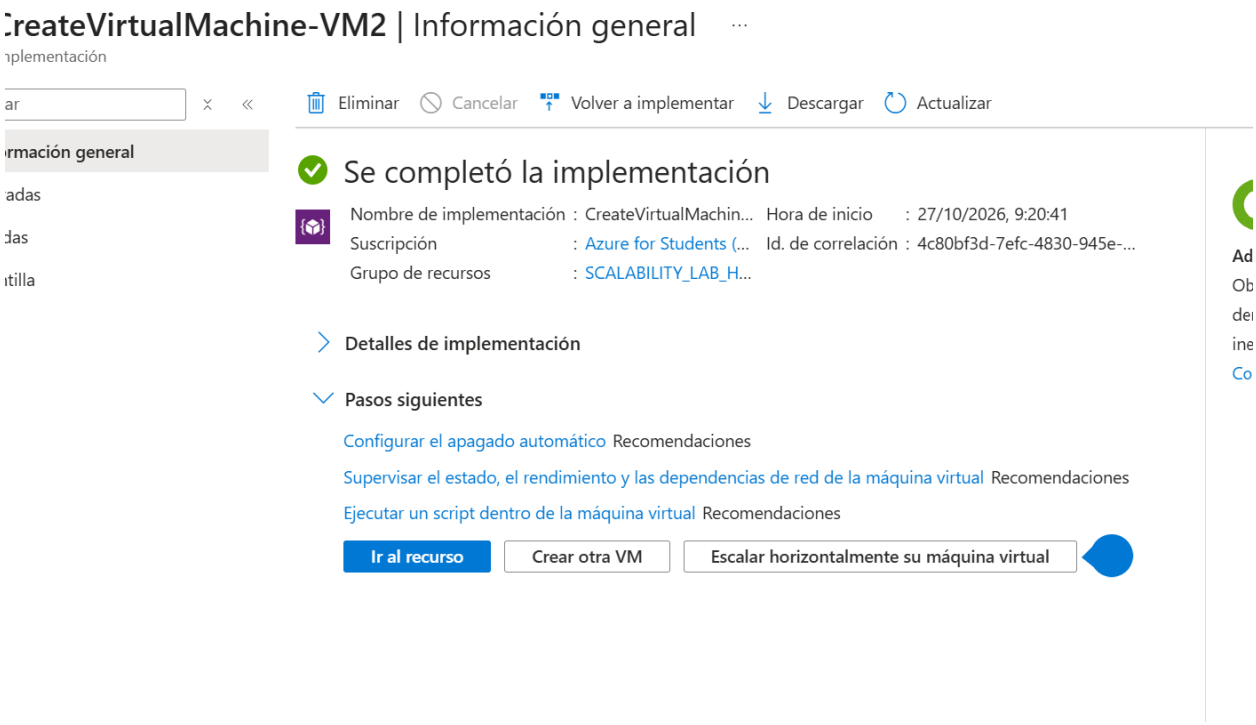


Figura 62. Implementación de VM2 completada.

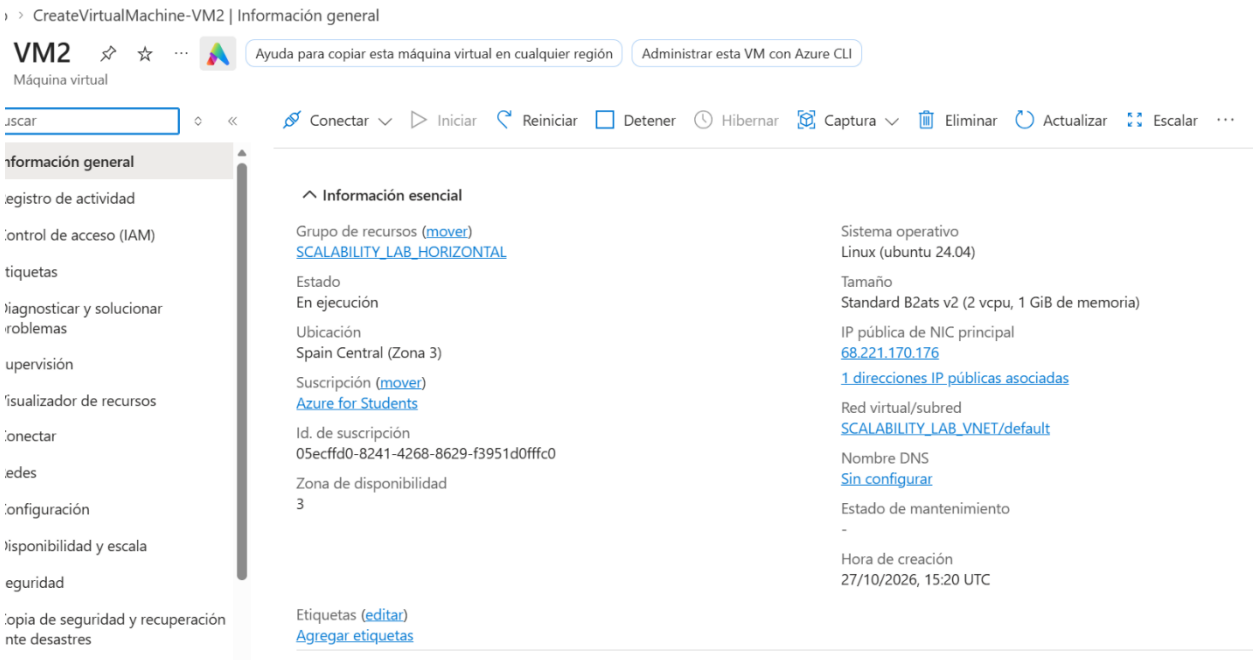


Figura 63. Información general de VM2 (zona 3).

VM3

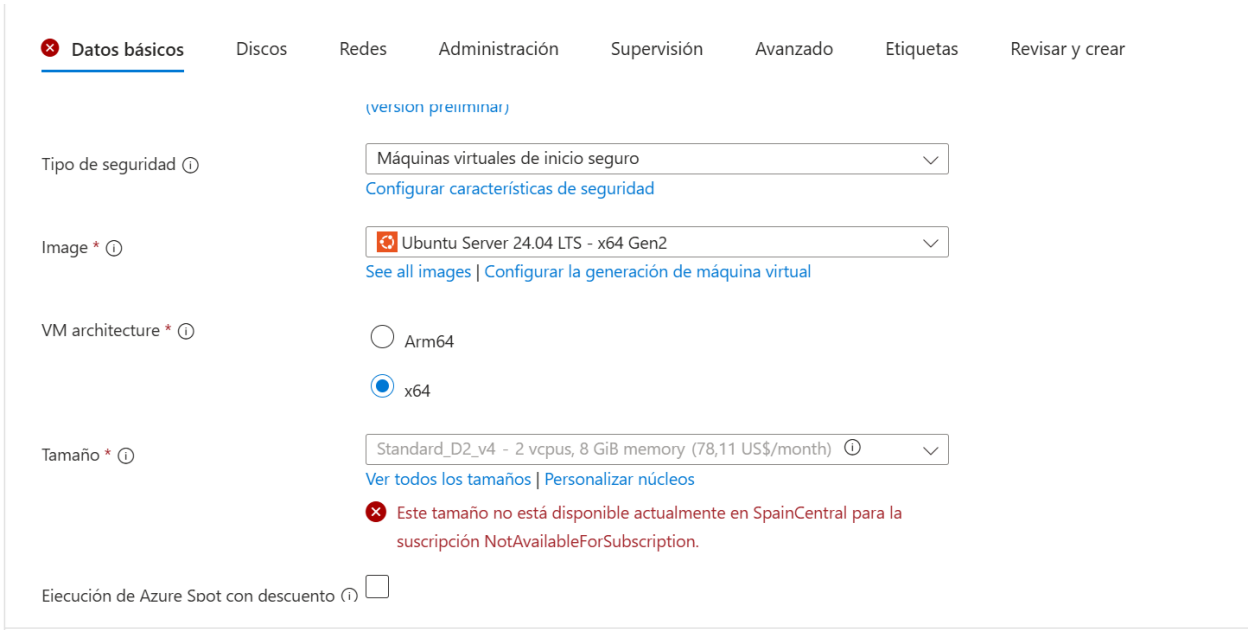


Figura 64. La zona 2 no tiene tamaños disponibles para la suscripción.

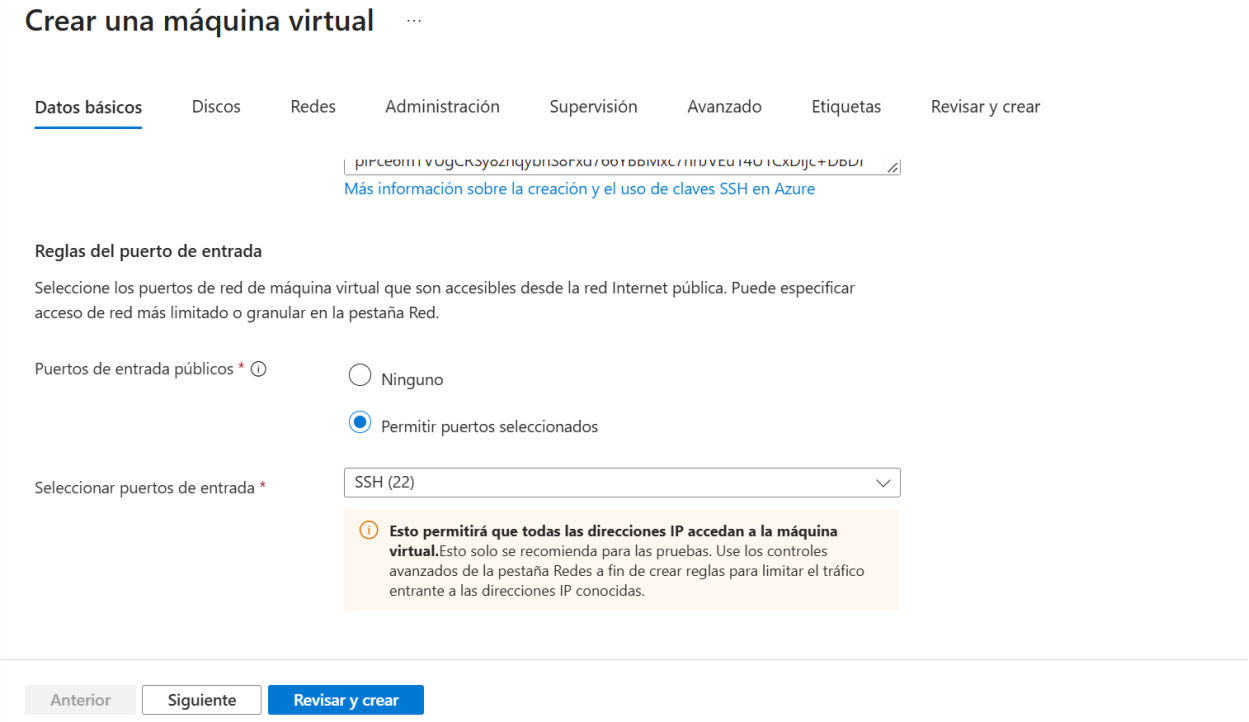


Figura 65. Cuenta de administrador de VM3 con la llave SSH existente.

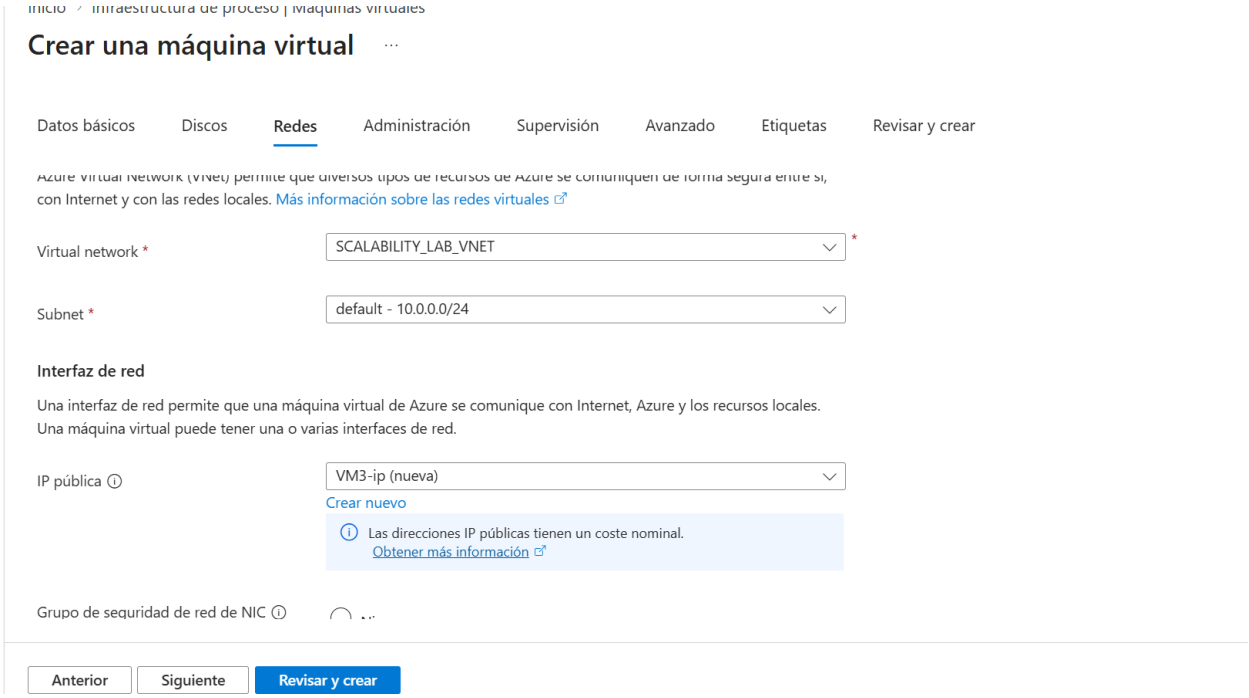


Figura 66. Primera configuración de red de VM3, con IP pública.

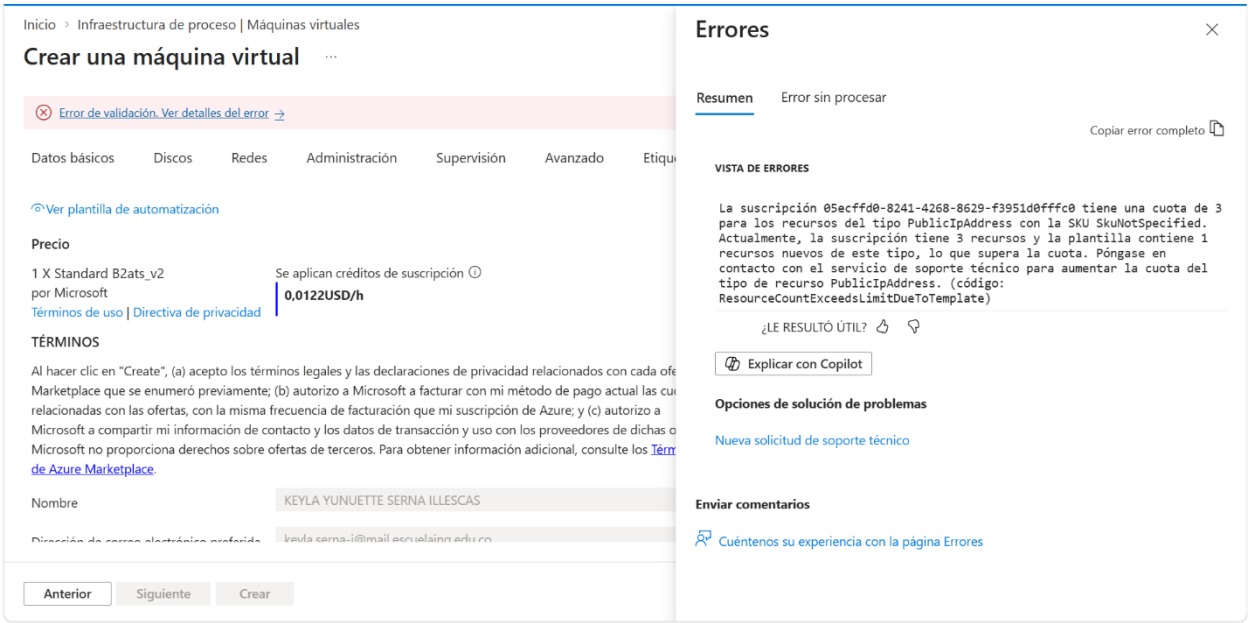


Figura 67. Límite de 3 IPs públicas por región al intentar crear VM3 con IP pública.

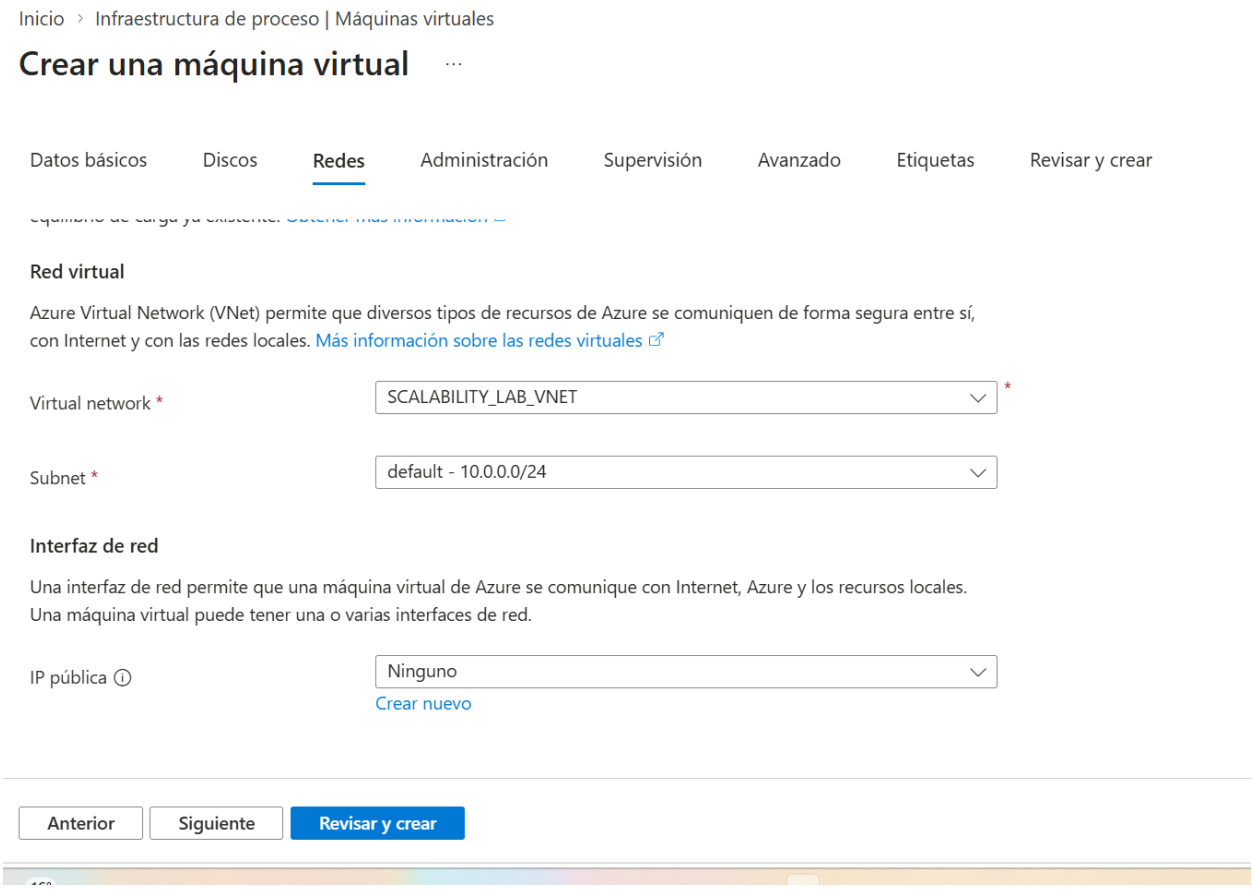


Figura 68. Configuración de red de VM3 sin IP pública.

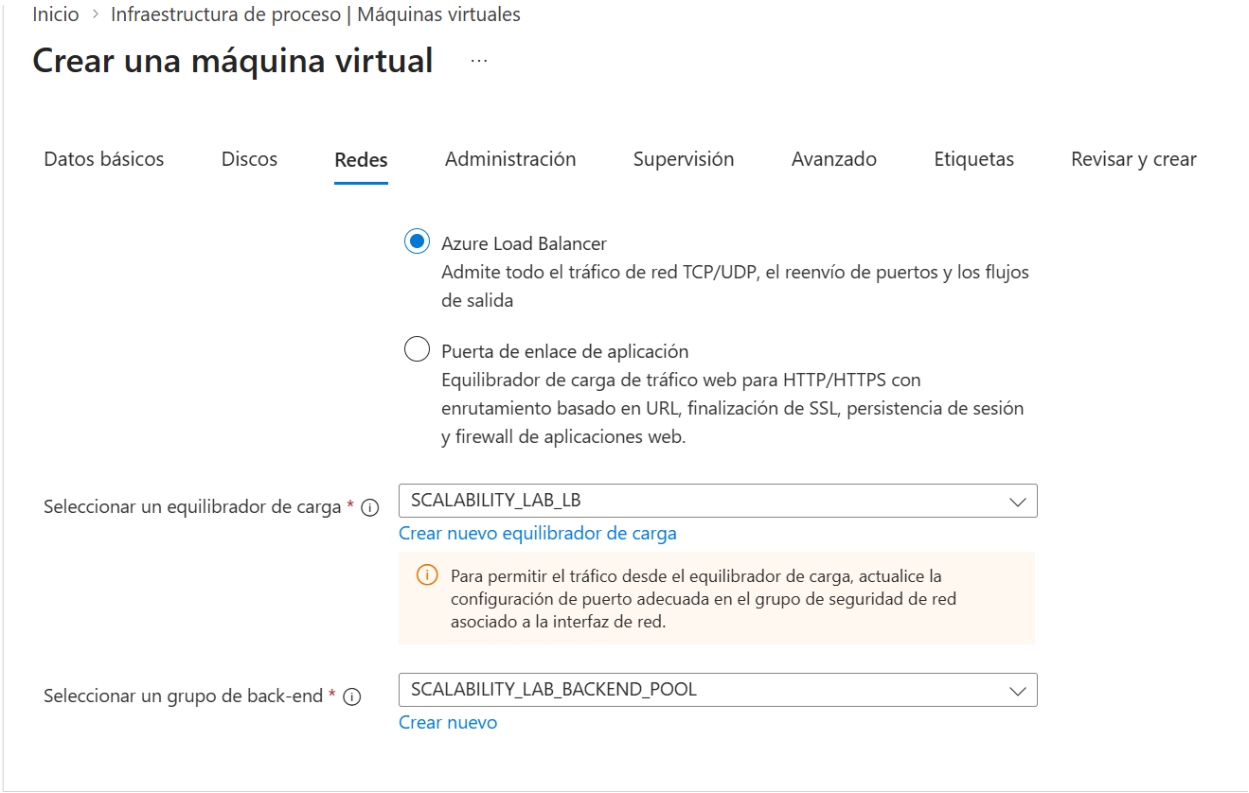


Figura 69. VM3 asociada al balanceador y al backend pool.



Figura 70. Revisión de la configuración de VM3 antes de crearla.

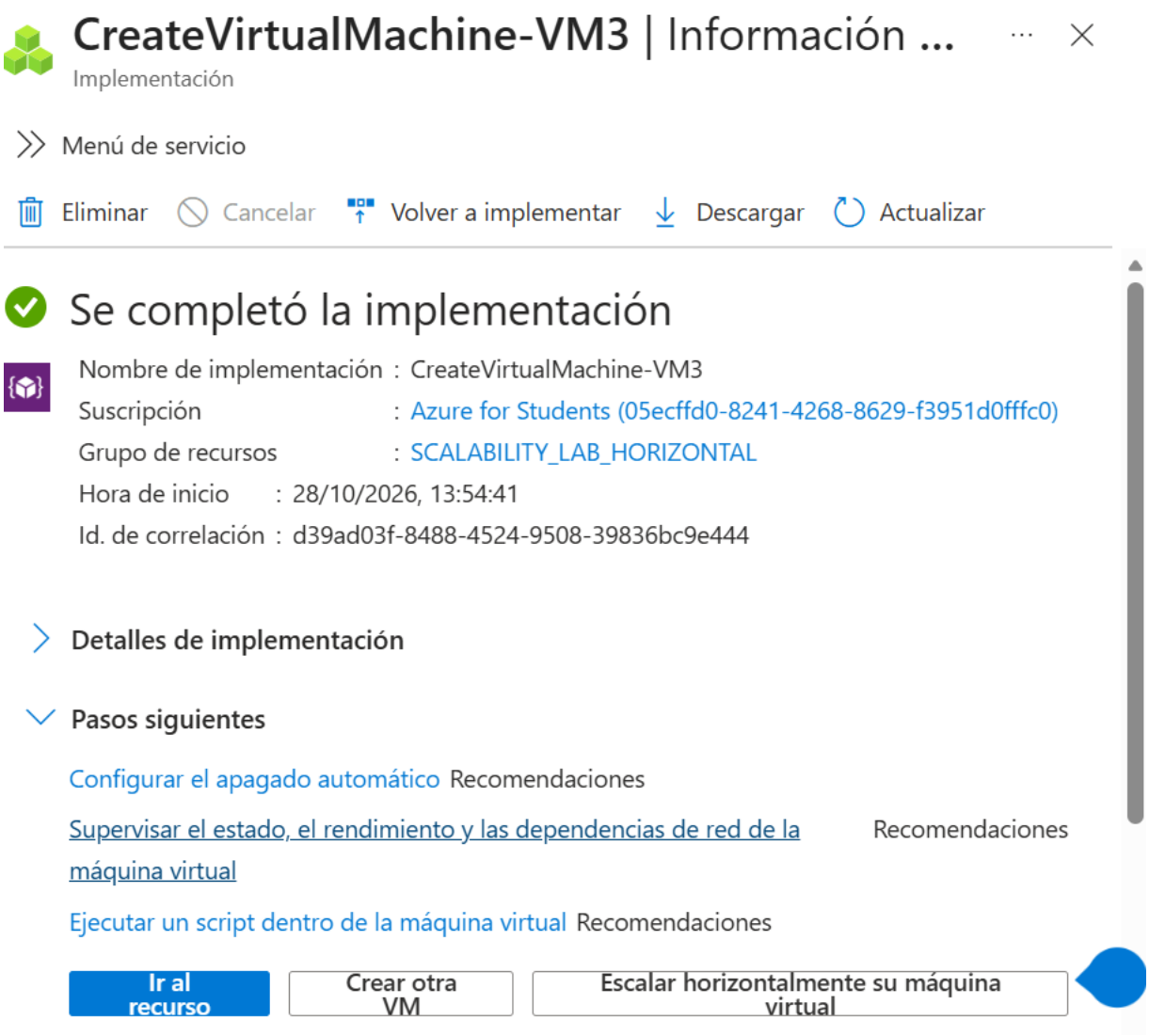


Figura 71. Implementación de VM3 completada.

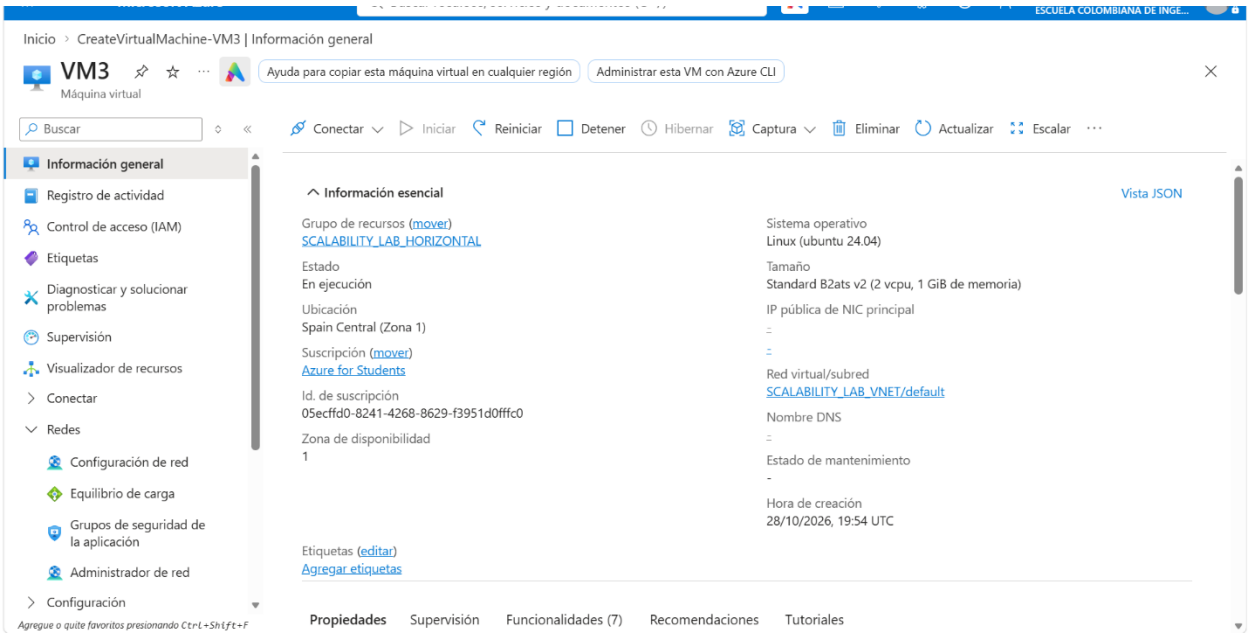


Figura 72. Información general de VM3 (zona 1).

3.5 Instalación de la aplicación

En VM1 y VM2 la aplicación se instaló por SSH, igual que en la Parte 1. En VM2, nvm instaló por defecto Node.js v26, que falló por una librería faltante (libatomic), así que se instaló Node.js v24 para que todas las máquinas tuvieran la misma versión. VM3, al no tener IP pública, no tenía salida a internet para descargar Node.js ni el repositorio. Para resolverlo se empaquetaron Node.js, forever y la aplicación en VM1 y se compartieron por la red privada; VM3 los descargó desde la IP privada de VM1 (10.0.0.4) usando la opción Ejecutar comando del portal, y la aplicación se inició con forever.

```
scalability_lab@VM1:~/ARSW_LOAD-BALANCING_AZURE/FibonacciApp$ forever stop
1
(node:2394) Warning: Accessing non-existent property 'padLevels' of module
exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:2394) Warning: Accessing non-existent property 'padLevels' of module
exports inside circular dependency
(node:2394) [DEP0060] DeprecationWarning: The `util._extend` API is depreca
ted. Please use Object.assign() instead.
error:   Forever cannot find process with id: 1
scalability_lab@VM1:~/ARSW_LOAD-BALANCING_AZURE/FibonacciApp$ forever list
(node:2406) Warning: Accessing non-existent property 'padLevels' of module
exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was created)
(node:2406) Warning: Accessing non-existent property 'padLevels' of module
exports inside circular dependency
(node:2406) [DEP0060] DeprecationWarning: The `util._extend` API is depreca
ted. Please use Object.assign() instead.
info:    Forever processes running
data:        uid  command
  script           forever pid  id logfile                                 u
ptime
data:    [0] 0NZT /home/scalability_lab/.nvm/versions/node/v24.22.1/bin/nod
e FibonacciApp.js 2303    2311    /home/scalability_lab/.forever/0NZT.log 0
:0:5:11.129999999999995
scalability_lab@VM1:~/ARSW_LOAD-BALANCING_AZURE/FibonacciApp$
```

Figura 73. FibonacciApp ejecutándose con forever en VM1.

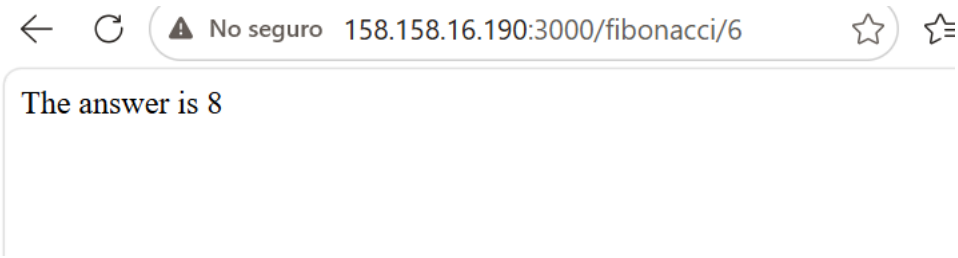


Figura 74. Respuesta de VM1 en /fibonacci/6: The answer is 8.

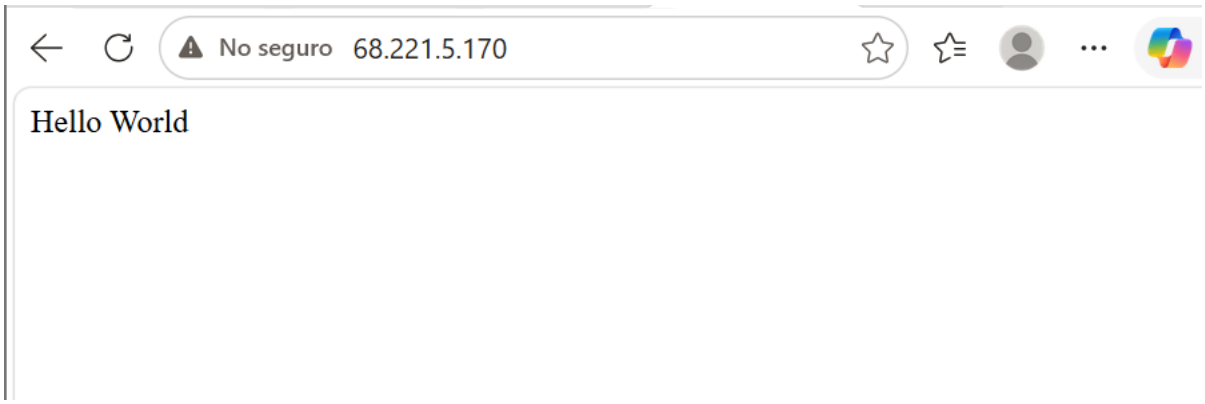


Figura 75. Primera prueba del balanceador con VM1: Hello World.

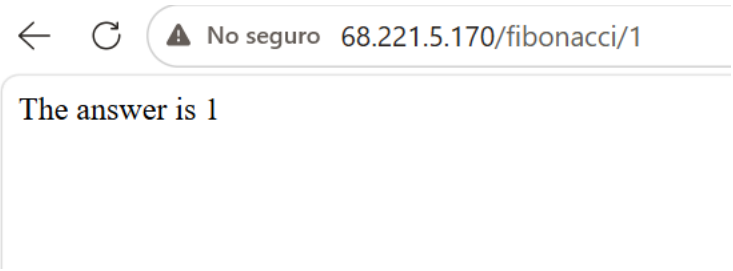


Figura 76. Primera prueba del balanceador con VM1: /fibonacci/1.



Figura 77. Revisión del NSG de VM2 al no poder conectarse por SSH desde la red de la universidad.



Figura 78. Conexión SSH a VM2.

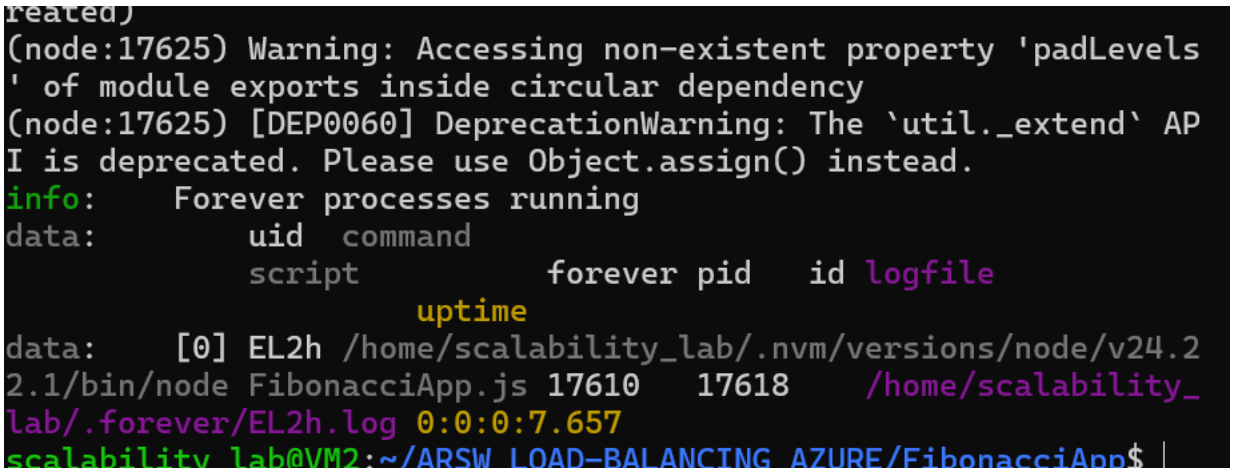


Figura 79. FibonacciApp ejecutándose con forever en VM2.



Figura 80. Respuesta de VM2 en /fibonacci/6: The answer is 8.

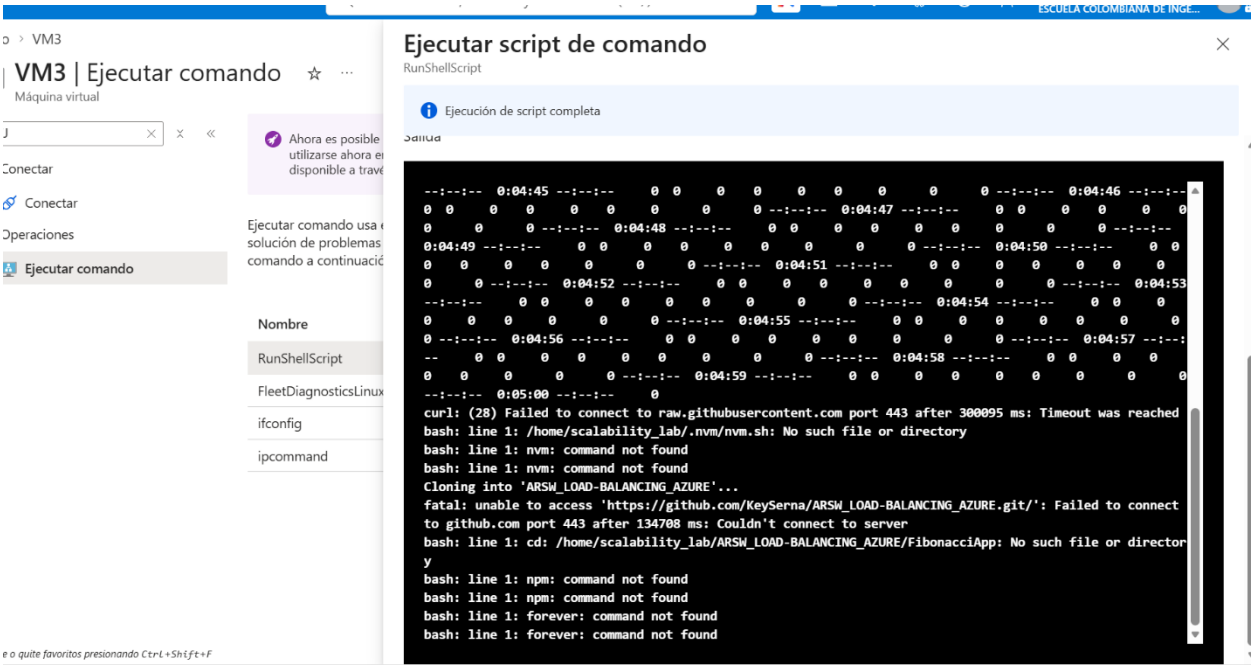


Figura 81. VM3 sin salida a internet: la descarga desde GitHub falla por tiempo de espera.

Figua 82. M1 compartiendo Node.js y la aplicación por la red privada.

```
See https://ubuntu.com/esm or run: sudo pro status


Last login: Mon Oct 26 01:55:09 2026 from 186.84.20.146
To run a command as administrator (user "root"), use "sudo <comm
and>".
See "man sudo_root" for details.

scalability_lab@VM1:~$ cd ~
scalability_lab@VM1:~$ tar czf /tmp/bundle.tgz .nvm ARSW_LOAD-BA
LANCING_AZURE
scalability_lab@VM1:~$ cd /tmp && python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Figura 83. Instalación en VM3 con Ejecutar comando: forever list y respuesta The answer is 8.

3.6 Prueba de la infraestructura

Con las tres VMs en el backend pool se consumieron los endpoints del balanceador. La raíz respondió Hello World y /fibonacci/1 respondió The answer is 1. El acceso se hace por el puerto 80, sin indicar el 3000, porque el balanceador hace la traducción de puertos.

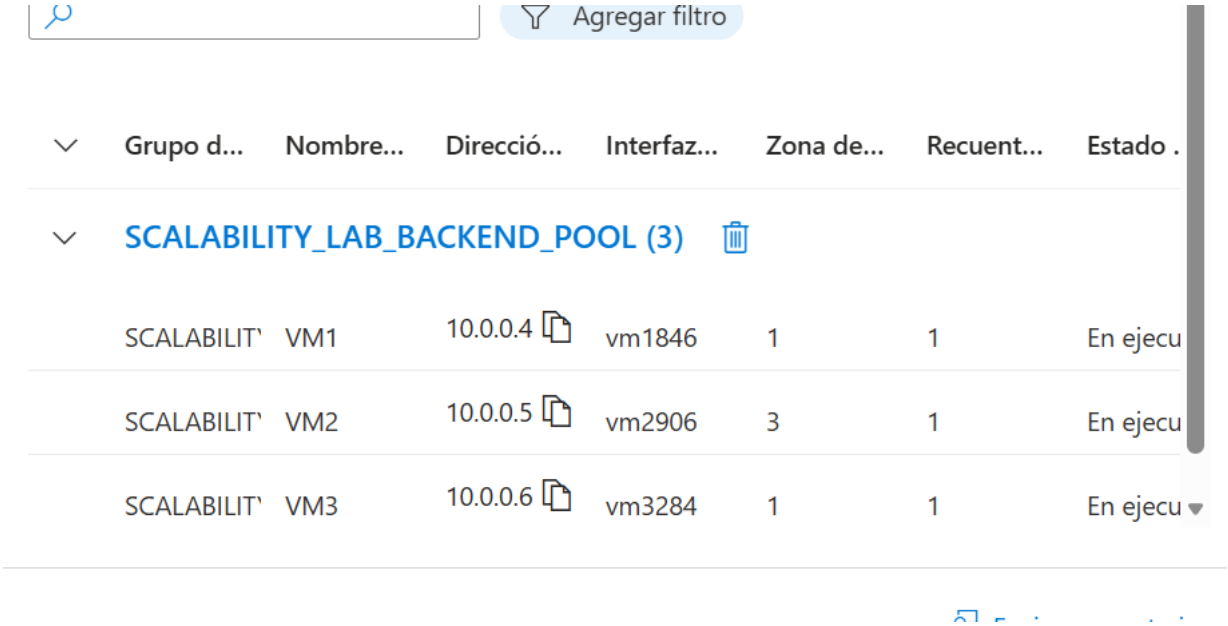


Figura 84. Backend pool con VM1, VM2 y VM3.



Figura 85. Respuesta del balanceador en http://68.221.5.170/.



Figura 86. Respuesta del balanceador en http://68.221.5.170/fibonacci/**1.**

3.7 Pruebas de carga con 2 ejecuciones en paralelo

Se repitió la prueba de Newman de la Parte 1, ahora contra la IP del balanceador. Las 20 peticiones fueron exitosas y el promedio de respuesta fue de 6.9 s y 6.7 s por ventana, prácticamente el mismo tiempo de una petición individual. Cada ejecución de Newman usa una sola conexión y el balanceador asigna cada conexión a una VM, así que cada ventana fue atendida por una máquina distinta y ninguna petición tuvo que esperar en fila. En las métricas se ve que en cada corrida trabajaron dos VMs, con un máximo de CPU de 46.28 %.

```
PS C:\Proyectos\ARSW_LOAD-BALANCING_AZURE\FibonacciApp\postman\part2> newman run A
RSW_LOAD-BALANCING_AZURE.postman_collection.json -e "[ARSW_LOAD-BALANCING_AZURE].p
ostman_environment.json" --env-var "loadbalancer=68.221.5.170" -n 1
(node:3584) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.
F_OK instead
(Use `node --trace-deprecation ...` to show where the warning was created)
newman

ARSW_LOAD-BALANCING_AZURE

→ fibonacci
  GET http://68.221.5.170/fibonacci/1000000 [200 OK, 209.24kB, 6.3s]

┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│                requests │                  1 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│            test-scripts │                  1 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│      prerequest-scripts │                  0 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│              assertions │                  0 │                  0 │
├─────────────────────────┴────────────────────┴────────────────────┤
│ total run duration: 6.4s                                          │
├───────────────────────────────────────────────────────────────────┤
│ total data received: 209kB (approx)                               │
├───────────────────────────────────────────────────────────────────┤
│ average response time: 6.3s [min: 6.3s, max: 6.3s, s.d.: 0µs]     │
└───────────────────────────────────────────────────────────────────┘
PS C:\Proyectos\ARSW_LOAD-BALANCING_AZURE\FibonacciApp\postman\part2>
```

Figura 87. Prueba de Newman con una sola petición contra el balanceador.



Figura 88. Resumen de Newman contra el balanceador con 2 ejecuciones en paralelo.

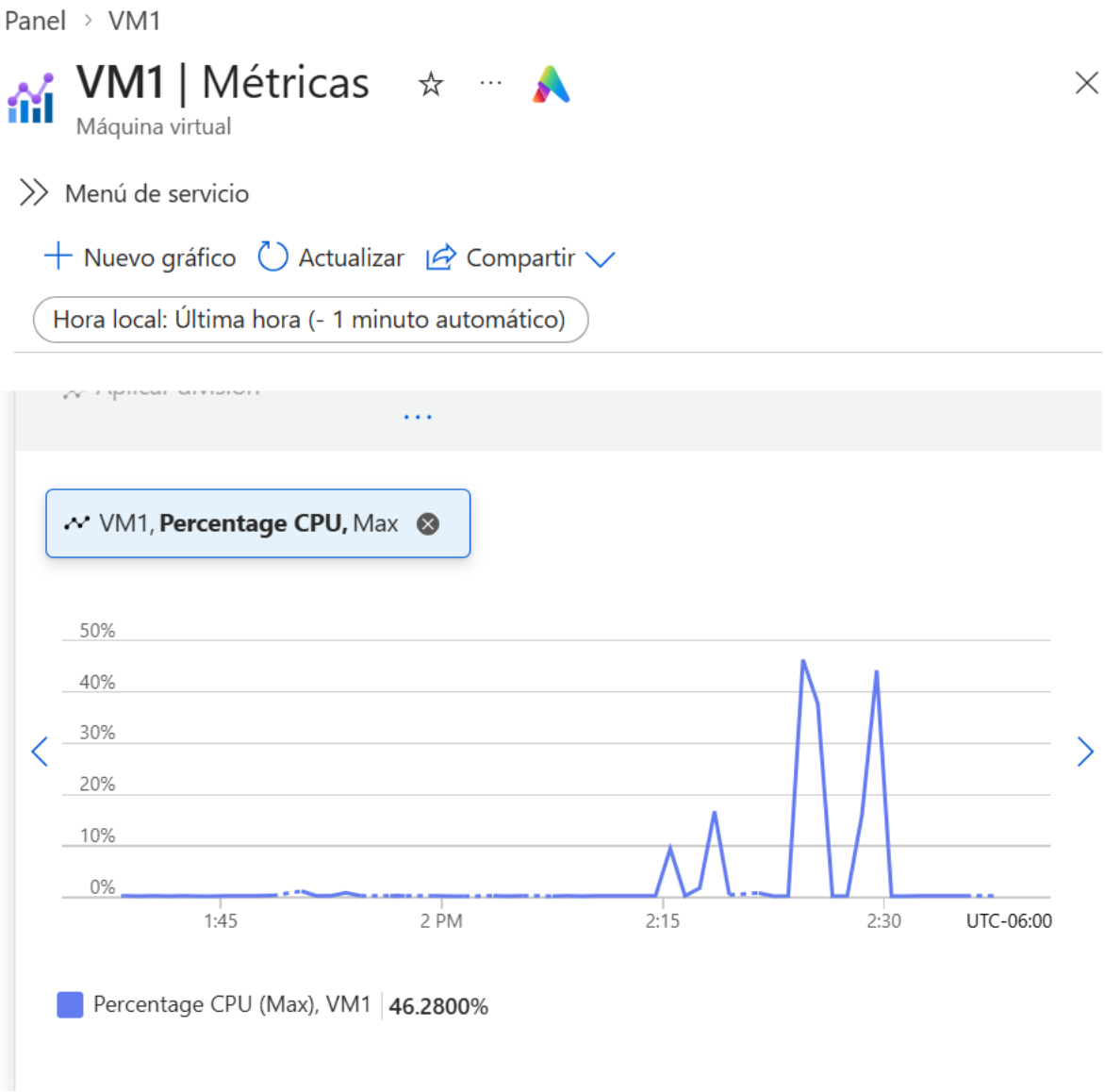


Figura 89. Consumo de CPU de VM1 (46.28 % y 44.19 % en las dos corridas).

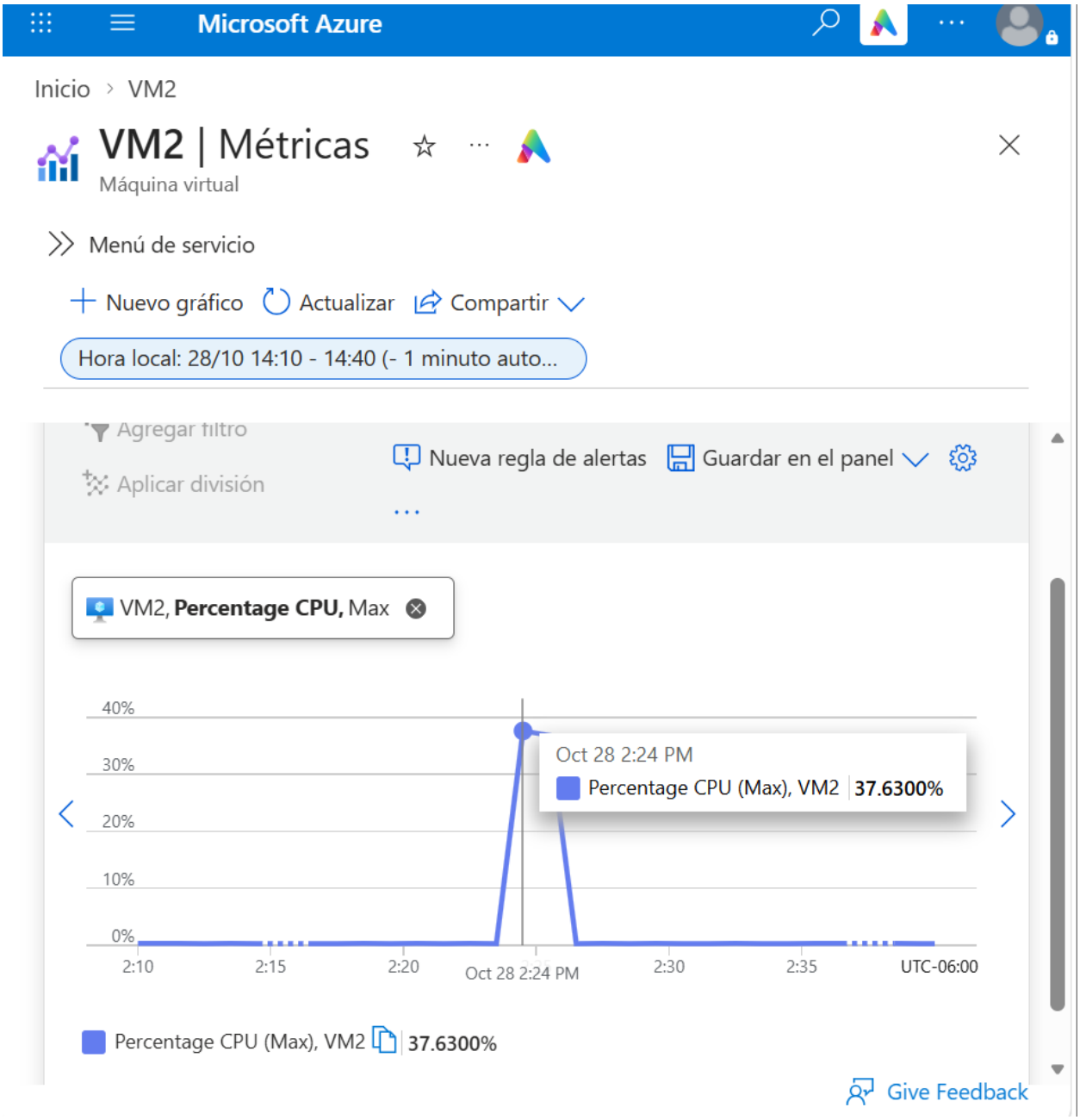


Figura 90. Consumo de CPU de VM2 (37.63 %).

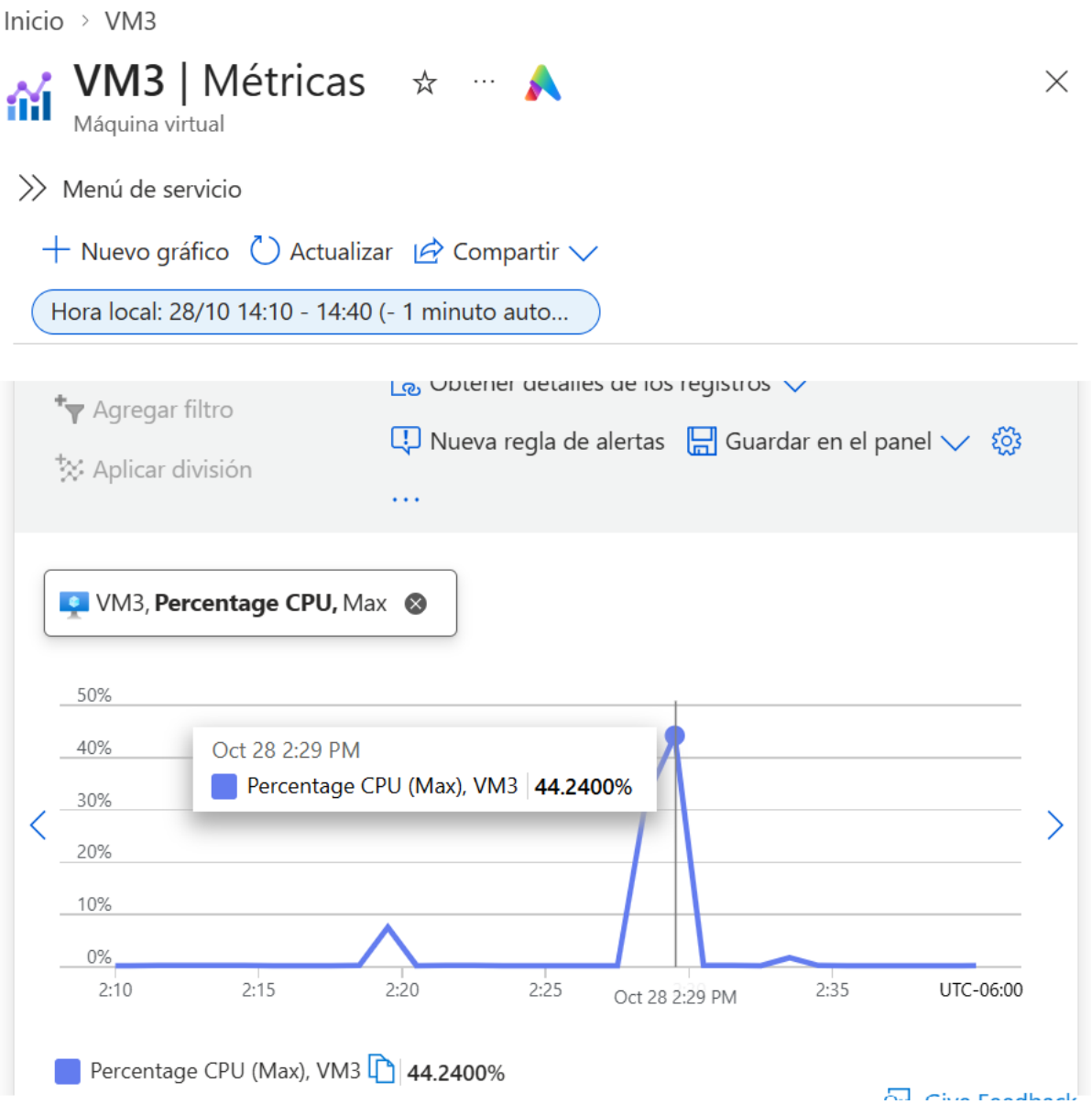


Figura 91. Consumo de CPU de VM3 (44.24 %).

3.8 Cuarta máquina virtual y 4 ejecuciones en paralelo

No fue posible crear la cuarta VM porque la suscripción ya usaba los 6 vCPU y las 3 IPs públicas permitidas en la región, y la solicitud de aumento de cuota había sido rechazada. Aun así se hizo la prueba con 4 ejecuciones de Newman en paralelo contra las 3 VMs. Fallaron 3 de 40 peticiones (7.5 %) y el promedio fue de unos 9 s. Dos de las ventanas no tuvieron fallos y respondieron en 6 a 8 s, porque su conexión quedó sola en una VM; las otras dos compartieron VM y tuvieron algunas esperas en fila que produjeron los ECONNRESET. Las tres VMs trabajaron y ninguna superó el 48.21 % de CPU.

La tasa de éxito aumentó frente a la Parte 1 (de 52.5 % a 92.5 % con 4 ejecuciones en paralelo) porque la carga ya no la atiende un solo proceso de Node.js: cada VM tiene su propio proceso y su propio núcleo, y el balanceador reparte las conexiones entre ellas. Con una cuarta VM cada conexión tendría su propia máquina y los fallos deberían desaparecer.

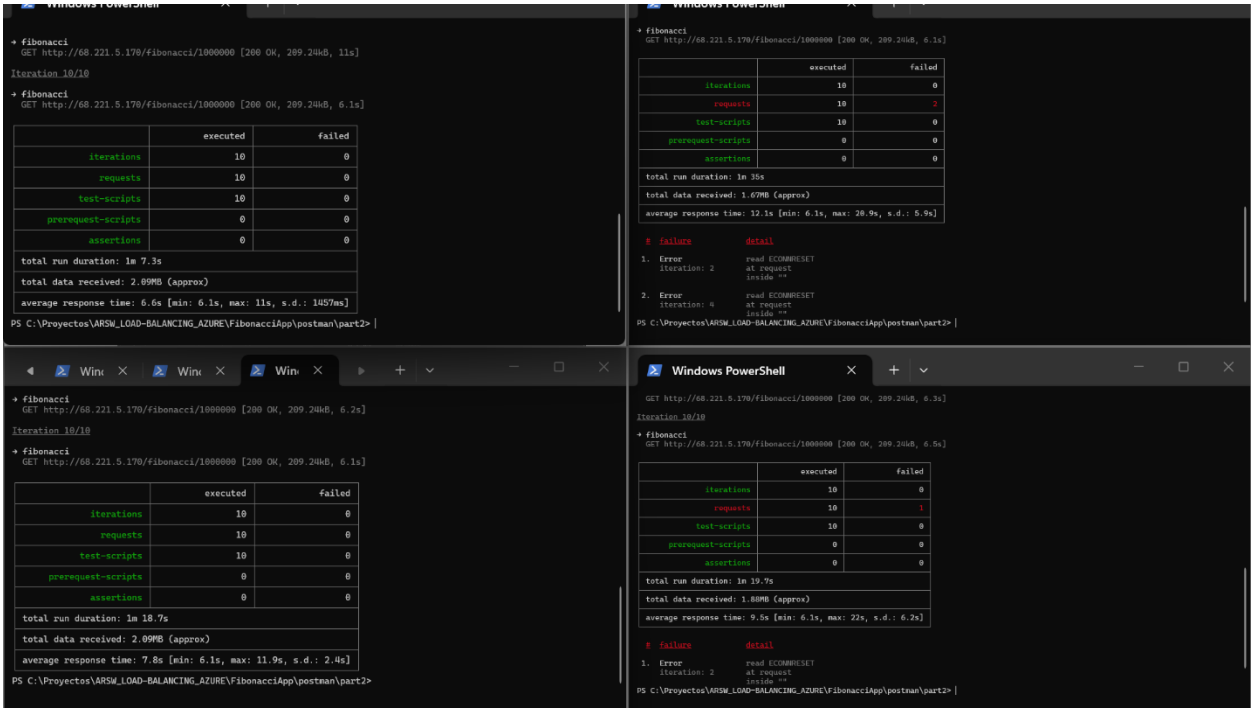


Figura 92. Resumen de Newman contra el balanceador con 4 ejecuciones en paralelo.

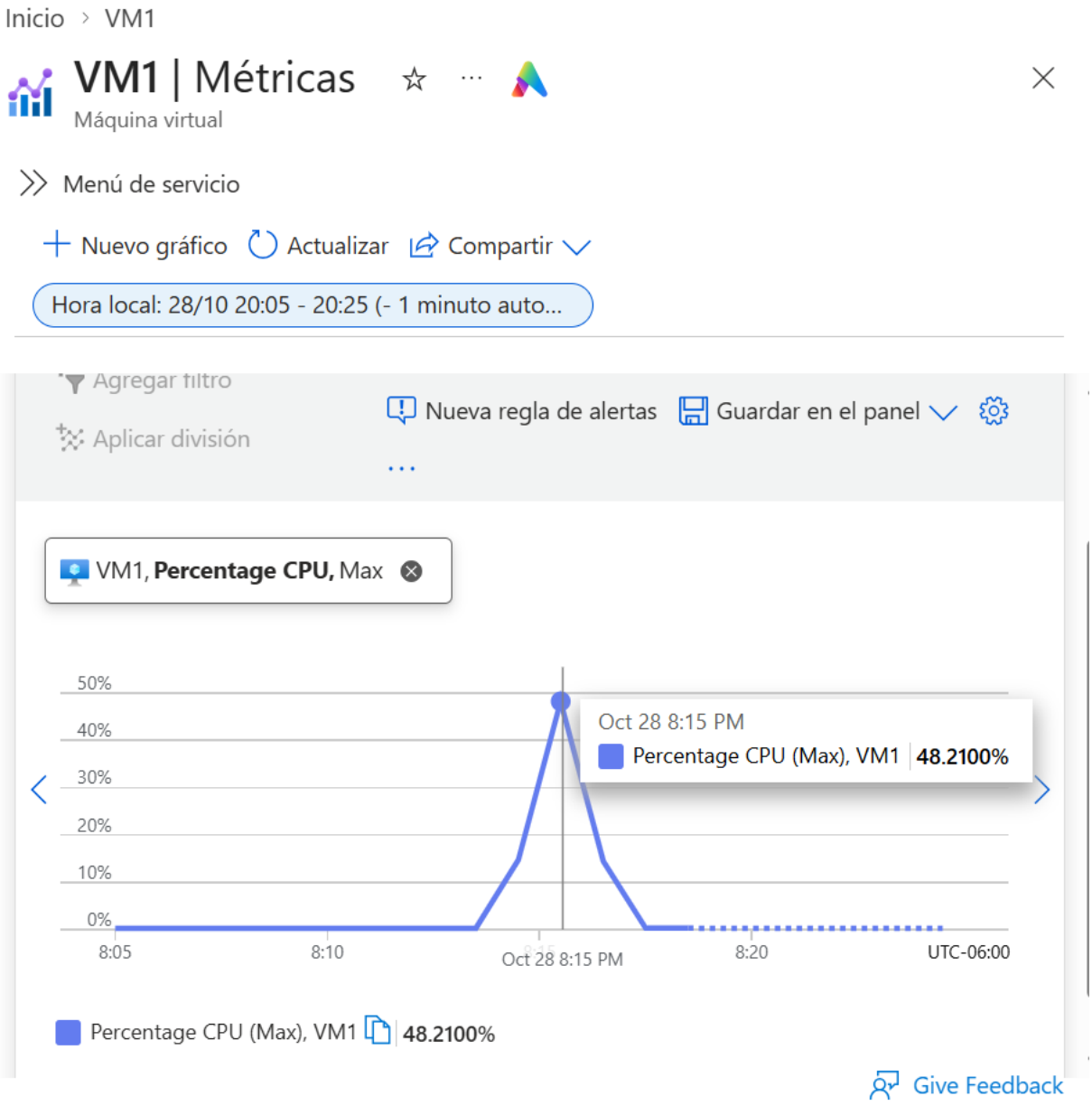


Figura 93. Consumo de CPU de VM1 con 4 ejecuciones en paralelo (48.21 %).

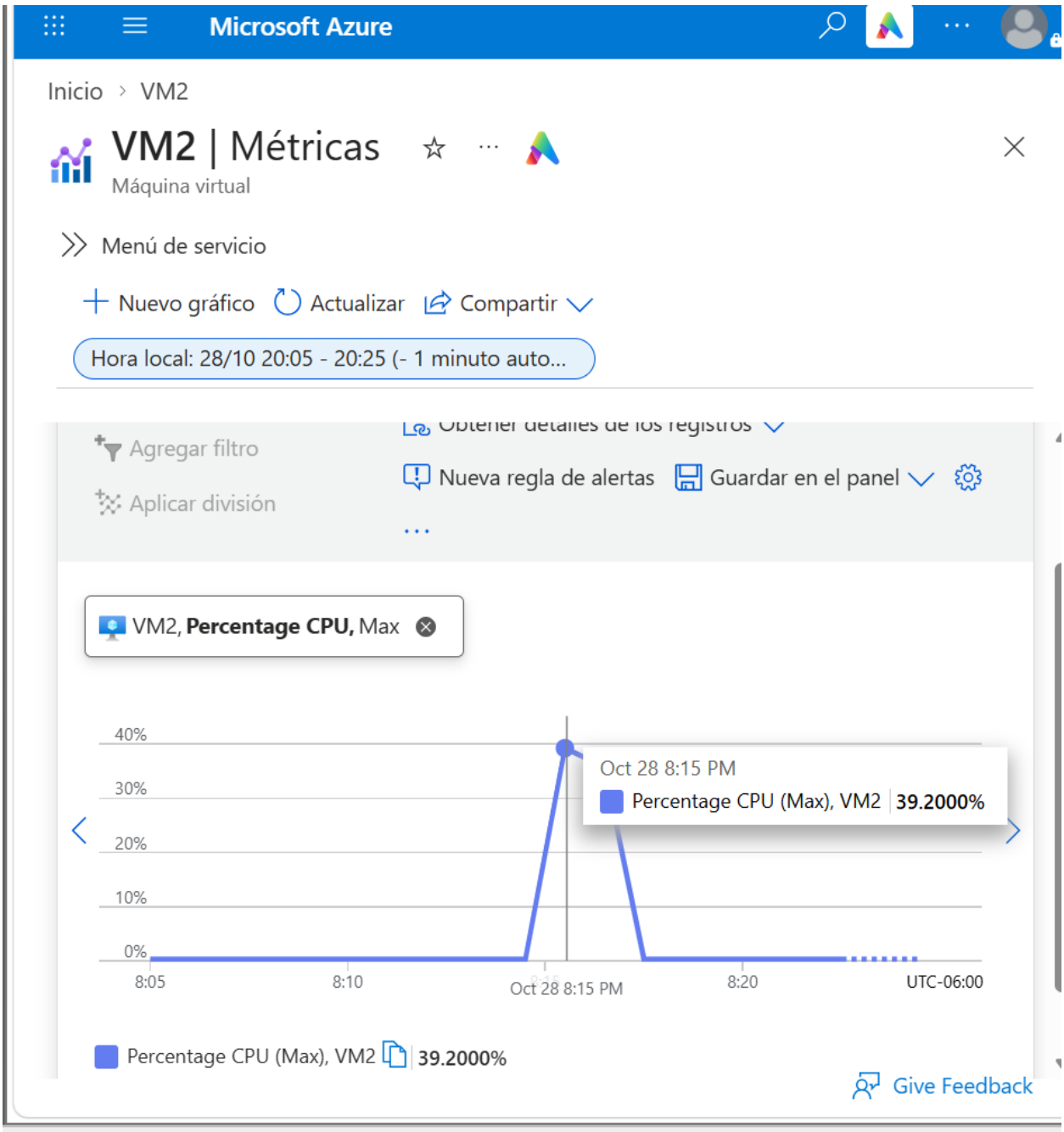


Figura 94. Consumo de CPU de VM2 con 4 ejecuciones en paralelo (39.20 %).

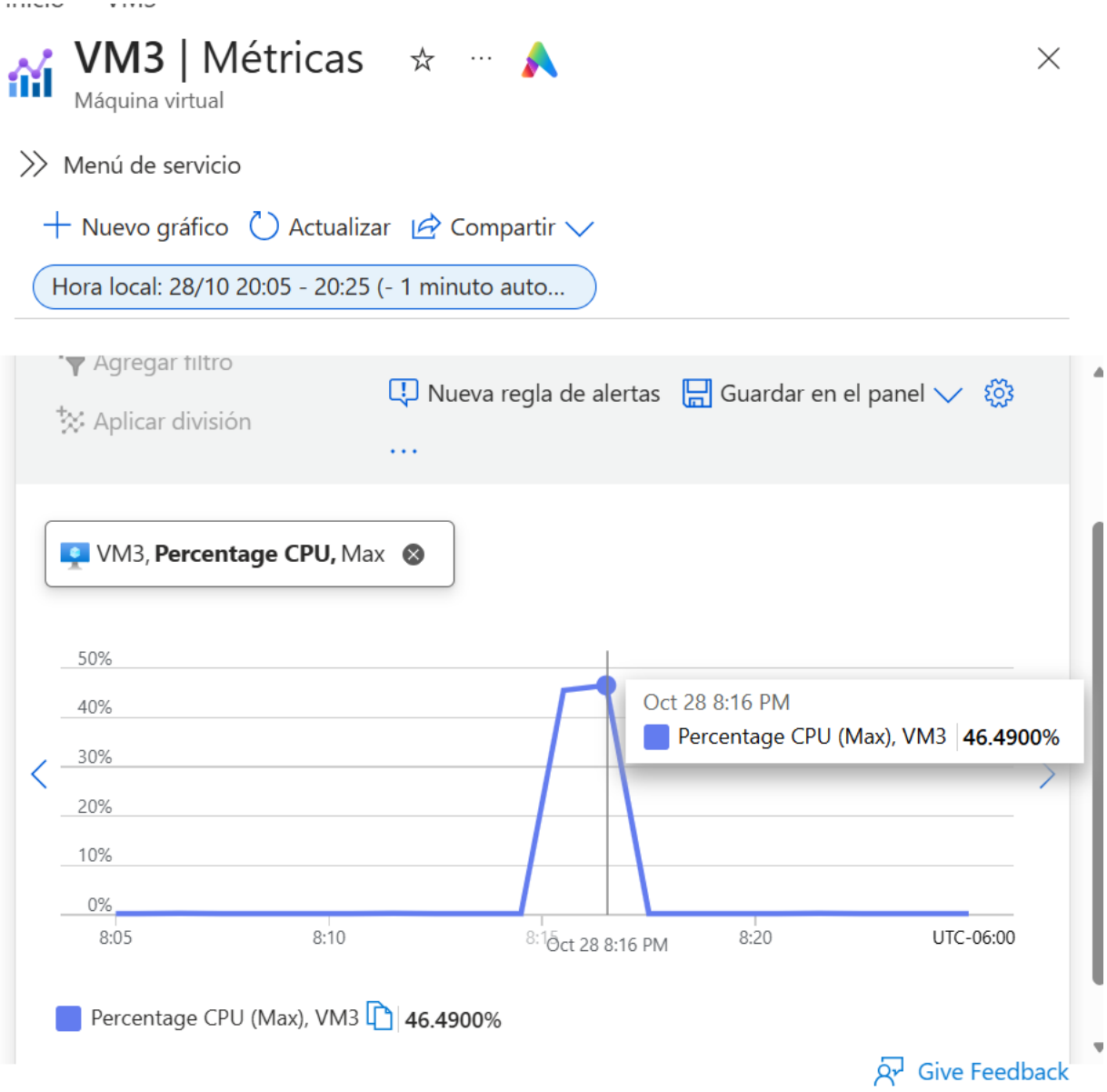


Figura 95. Consumo de CPU de VM3 con 4 ejecuciones en paralelo (46.49 %).

3.9 Informe comparativo de Newman (punto 2)

La siguiente tabla compara las tres infraestructuras probadas con la misma carga.

Prueba	1 VM B2ats_v2	1 VM D4s_v4	Balanceador + 3 VMs B2ats_v2
2 en paralelo: éxito	20 de 20 (100 %)	11 de 20 (55 %)	20 de 20 (100 %)
2 en paralelo: promedio	10.9 s	11.5 s	6.8 s
4 en paralelo: éxito	No se probó	21 de 40 (52.5 %)	37 de 40 (92.5 %)
4 en paralelo: promedio	No se probó	25.6 s	9.0 s
CPU máxima	50.02 %	25.24 %	48.21 % por VM
Costo aproximado	7,52 US$/mes	154,03 US$/mes	~56 US$/mes

Tabla 2. Comparación de resultados y costos entre las infraestructuras.

El costo de la infraestructura horizontal incluye las tres VMs B2ats_v2 (26,73 US$ al mes), el balanceador Estándar (unos 18 US$ al mes por las reglas) y las tres IPs públicas Estándar (unos 3,65 US$ al mes cada una), sin contar el tráfico procesado. Aunque es más cara que una sola VM pequeña, cuesta casi tres veces menos que la VM grande y es la única opción que respondió todas las peticiones con 2 usuarios concurrentes y la mayoría con 4, con tiempos cercanos a los de una petición individual.

3.10 Respuestas a las preguntas de la Parte 2

1. ¿Cuáles son los tipos de balanceadores de carga en Azure y en qué se diferencian? ¿Qué es SKU, qué tipos hay y en qué se diferencian? ¿Por qué el balanceador de carga necesita una IP pública?

Azure ofrece varios servicios de balanceo. Azure Load Balancer trabaja en la capa 4 (TCP y UDP) y reparte conexiones entre VMs; puede ser público, si recibe tráfico de internet, o interno, si solo reparte tráfico dentro de la red privada, y puede ser regional o global. Application Gateway trabaja en la capa 7 (HTTP y HTTPS), puede enrutar según la URL y tiene firewall de aplicaciones web. Front Door es un balanceador global de capa 7 que distribuye tráfico entre regiones del mundo y acelera la entrega de contenido. Traffic Manager balancea a nivel de DNS, dirigiendo a cada usuario al punto de acceso más adecuado.

El SKU es la versión o nivel del servicio, y define sus características y su costo. El SKU Básico era gratuito, sin soporte de zonas de disponibilidad ni SLA, y fue retirado en septiembre de 2025. El SKU Estándar soporta zonas de disponibilidad, tiene SLA del

99.99 %, es seguro por defecto (bloquea el tráfico que no se permita explícitamente), admite hasta 1000 instancias en el backend y ofrece métricas y reglas de salida. El SKU Gateway sirve para encadenar aplicaciones virtuales de red de terceros, como firewalls. En el laboratorio se usó el SKU Estándar.

El balanceador necesita una IP pública porque es el único punto de entrada al sistema desde internet: los clientes se conectan a una sola dirección (68.221.5.170) y no necesitan conocer las IPs de las VMs. Sin IP pública solo podría recibir tráfico desde dentro de la red privada.

2. ¿Cuál es el propósito del Backend Pool?

El backend pool es el conjunto de máquinas (identificadas por su interfaz de red o su IP privada) entre las que el balanceador reparte el tráfico. Agregar o quitar VMs del pool es la forma de escalar horizontalmente: en este laboratorio el pool tuvo primero una VM y después tres, sin cambiar la IP pública que usan los clientes.

3. ¿Cuál es el propósito del Health Probe?

El sondeo de estado revisa periódicamente cada VM del backend pool para saber si está disponible. Si una VM deja de responder, el balanceador deja de enviarle tráfico nuevo hasta que se recupere, y así evita que los usuarios reciban errores por una máquina caída. En este laboratorio se configuró un sondeo TCP al puerto 3000 cada 5 segundos.

4. ¿Cuál es el propósito de la Load Balancing Rule? ¿Qué tipos de sesión persistente existen, por qué esto es importante y cómo puede afectar la escalabilidad del sistema?

La regla de equilibrio de carga define cómo se reparte el tráfico: qué IP de front-end y puerto lo reciben (puerto 80), a qué backend pool y a qué puerto se envía (3000), con qué protocolo y qué sondeo de estado se usa para decidir qué VMs están disponibles.

Existen tres opciones de persistencia de sesión. Con Ninguna, el balanceador usa un hash de cinco valores (IP y puerto de origen, IP y puerto de destino, y protocolo), así que cada conexión nueva puede ir a cualquier VM. Con IP del cliente, todas las conexiones que vienen de la misma IP van a la misma VM. Con IP del cliente y protocolo, se agrega el protocolo a esa condición. La persistencia es importante cuando la aplicación guarda el estado de la sesión en la memoria de una VM, por ejemplo un carrito de compras, porque el usuario debe volver siempre a la misma máquina. Sin embargo, perjudica la escalabilidad, porque la carga deja de repartirse de forma uniforme: en este laboratorio todas las pruebas salen de una sola IP, así que con persistencia por IP todas las peticiones habrían caído en una sola VM. Por eso se eligió Ninguna. Aun así, como cada

ejecución de Newman mantiene abierta una sola conexión, todas sus peticiones fueron a la misma VM, y por eso con 2 ejecuciones en paralelo solo trabajaron dos de las tres máquinas.

5. ¿Qué es una Virtual Network? ¿Qué es una Subnet? ¿Para qué sirven el address space y el address range?

Una red virtual es una red privada y aislada dentro de Azure, en la que los recursos se comunican entre sí con direcciones IP privadas. Una subred es una división de esa red que agrupa recursos y permite aplicarles reglas de seguridad y enrutamiento comunes. El address space es el rango total de direcciones privadas que puede usar la red virtual; en este caso 10.0.0.0/16, con 65 536 direcciones. El address range de una subred es la porción de ese espacio que se le asigna; en este caso 10.0.0.0/24, con 256 direcciones, de las cuales las VMs recibieron 10.0.0.4, 10.0.0.5 y 10.0.0.6 (Azure reserva las primeras cuatro y la última).

6. ¿Qué son las Availability Zones y por qué seleccionamos 3 diferentes zonas? ¿Qué significa que una IP sea zone-redundant?

Las zonas de disponibilidad son centros de datos físicamente separados dentro de una misma región, con energía, refrigeración y red independientes. Se distribuyen las VMs en zonas distintas para que, si una zona completa falla, las VMs de las otras zonas sigan atendiendo y el sistema continúe disponible. En este laboratorio no fue posible usar tres zonas porque el tamaño B2ats_v2 solo existía en las zonas 1 y 3 y la zona 2 no tenía tamaños para la suscripción; las VMs quedaron en dos zonas, lo que igualmente protege ante la falla de una de ellas. Una IP zone-redundant está replicada en todas las zonas de la región, así que sigue funcionando aunque una zona falle; por eso la IP del balanceador se configuró de esa forma.

7. ¿Cuál es el propósito del Network Security Group?

El grupo de seguridad de red funciona como un firewall a nivel de red: contiene reglas que permiten o deniegan el tráfico de entrada y de salida según el origen, el destino, el puerto y el protocolo. En este laboratorio el NSG SCALABILITY_LAB_NSG, compartido por las tres VMs, permite el puerto 22 para administrar por SSH y el puerto 3000 para que el balanceador entregue las peticiones a la aplicación. Sus reglas por defecto permiten además el tráfico interno de la red virtual, lo que hizo posible copiar la aplicación de VM1 a VM3, y el tráfico del sondeo de estado del balanceador.

8. Informe de Newman 1 (punto 2).

El informe comparativo de tiempos de respuesta, cantidad de peticiones exitosas y costos se presenta en la sección 3.9 y en la Tabla 2.

9. Presente el diagrama de despliegue de la solución.

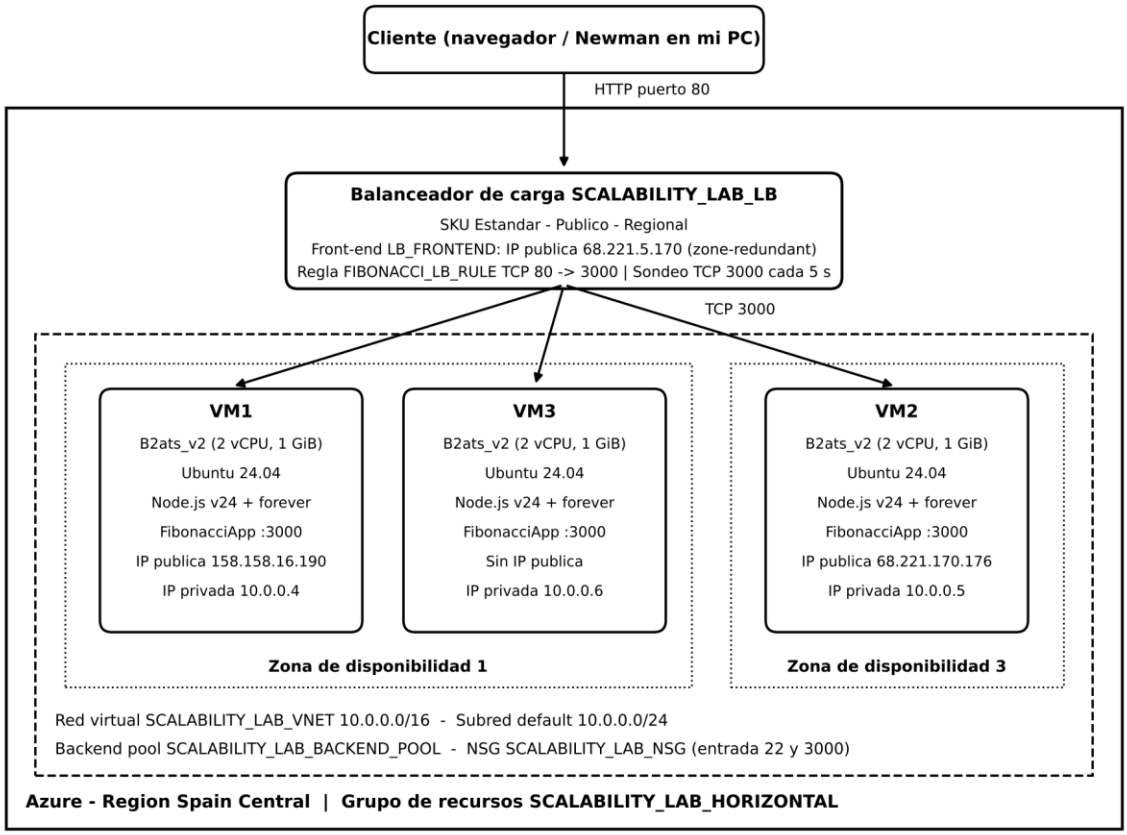


Figura 96. Diagrama de despliegue de la solución con balanceador de carga en Spain Central.

4. Conclusiones

El escalamiento vertical no resolvió el problema de esta aplicación. Agregar núcleos a una sola máquina no ayuda cuando el programa solo puede usar un hilo: la VM de 4 vCPU usó apenas el 25 % de su CPU, fue más lenta porque sus núcleos eran más lentos, falló en casi la mitad de las peticiones concurrentes, costó unas 20 veces más y obligó a reiniciar el servicio para cambiar su tamaño.

El escalamiento horizontal sí mejoró el sistema. Al repartir las conexiones entre varias VMs, cada petición se atendió en un tiempo cercano al de una petición individual. Con 2 usuarios concurrentes se respondieron todas las peticiones con una CPU máxima de 46 %, cumpliendo el escenario de calidad, y con 4 usuarios la tasa de éxito pasó del 52.5 % al 92.5 %. Además, el sistema se puede seguir escalando agregando VMs al backend pool sin cambiar la dirección que usan los clientes y sin detener el servicio.

El laboratorio también mostró que las decisiones de arquitectura dependen de las restricciones reales de la plataforma: los límites de la suscripción obligaron a cambiar de región, de tamaños y de distribución por zonas, a crear una VM sin IP pública y a instalarle la aplicación a través de la red privada. Finalmente, el límite de 6 vCPU impidió crear la cuarta VM, que habría permitido eliminar los fallos restantes con 4 usuarios concurrentes.